Program No: 01	**Date:** 17-11-2025

Program Title: Program to display two numbers using command line arguments, add those numbers, and display the result.

```
/*
Usage of Command Line Arguments
@ Ashbi Saju
Roll no: 16
Date: 17-11-2025
*/

import java.io.*;
class PG1SumCLA {
   public static void main(String args[]) {
      // Check if at least two arguments are provided
      if (args.length < 2) {
         System.out.println("Usage: java PG1SumCLA <num1> <num2>");
         return;
      }

      // Convert first command-line argument from String to int
      int a = Integer.parseInt(args[0]);

      // Convert second command-line argument from String to int
      int b = Integer.parseInt(args[1]);

      // Display the sum of two numbers
      System.out.println(a + " + " + b + " = " + (a + b));
   }
}
```

Output

```
C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>javac PG1SumCLA.java

C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>java PG1SumCLA
Usage: java PG1SumCLA <num1> <num2>

C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>java PG1SumCLA 10 5
10 + 5 = 15
```

Program No: 02	**Date:** 17-11-2025

Program Title: Program to display two numbers using command line arguments, add those numbers, and display the result.

```
/*
Arithmetic Operations using Command Line Arguments
@ Ashbi Saju
Roll no: 16
Date: 17-11-2025
*/

import java.io.*;
class PG2ArithmeticCLA {
   public static void main(String args[]) {
      // Check if three arguments are provided (two numbers and operation code)
      if (args.length < 3) {
         System.out.println("Usage: java ArithmeticCLA <a> <b> <operation>");
         return;
      }

      // Convert first argument to integer (first operand)
      int a = Integer.parseInt(args[0]);

      // Convert second argument to integer (second operand)
      int b = Integer.parseInt(args[1]);

      // Convert third argument to integer (operation code: 0-Add, 1-Sub, 2-Mul, 3-Div, 4-Mod)
      int o = Integer.parseInt(args[2]);

      // Perform operation based on operation code
      switch (o) {
         case 0:
            // Addition
            System.out.println(a + " + " + b + " = " + (a + b));
            break;

         case 1:
            // Subtraction
            System.out.println(a + " - " + b + " = " + (a - b));
            break;
```

```
        case 2:
            // Multiplication
            System.out.println(a + " * " + b + " = " + (a * b));
            break;

        case 3:
            // Division with zero check
            if (b == 0) {
                System.out.println("Error: Division by zero");
            } else {
                System.out.println(a + " / " + b + " = " + (a / b));
            }
            break;

        case 4:
            // Modulus with zero check
            if (b == 0) {
                System.out.println("Error: Modulus by zero");
            } else {
                System.out.println(a + " % " + b + " = " + (a % b));
            }
            break;

        default:
            // Invalid operation code
            System.out.println("Invalid operation code.");
    }
  }
}
```

Output

```
C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>javac PG2ArithmeticCLA.java

C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>java PG2ArithmeticCLA
Usage: java ArithmeticCLA <a> <b> <operation>

C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>java PG2ArithmeticCLA 10 5 0
10 + 5 = 15

C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>java PG2ArithmeticCLA 10 5 1
10 - 5 = 5

C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>java PG2ArithmeticCLA 10 5 2
10 * 5 = 50

C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>java PG2ArithmeticCLA 10 5 3
10 / 5 = 2

C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>java PG2ArithmeticCLA 10 5 4
10 % 5 = 0
```

Program No: 03	**Date:** 29-11-2025

Program Title: Keep a secret number between 20 and 30 in your program. Ask the user to predict a number between 20 and 30. Use a do-while loop until the user predicts your secret number and displays the count of attempts.

```
/*
Secret Number Game
@ Ashbi Saju
Roll no: 16
Date: 29-11-2025
*/

import java.util.Scanner;

public class PG3GuessNumber {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      // Set the secret number
      int secretNumber = 25;
      int guess;
      int attempts = 0;

      // Display instructions to the user
      System.out.println("I have a secret number between 20 and 30.");
      System.out.println("Try to guess it!");

      // Do-while loop continues until correct guess
      do {
         // Prompt user for input
         System.out.print("\nEnter your guess (20-30): ");
         guess = sc.nextInt();

         // Increment attempt counter
         attempts++;

         // Provide hints based on user's guess
         if (guess < secretNumber) {
            System.out.println("Too low! Try again.");
         } else if (guess > secretNumber) {
            System.out.println("Too high! Try again.");
         } else {
```

```
            // Display success message with attempt count
            System.out.println("\nCongratulations! You guessed the secret number " +
secretNumber);
            System.out.println("Number of attempts: " + attempts);
        }

    } while (guess != secretNumber); // Loop until correct guess

    sc.close();
  }
}
```

Output

```
C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>javac PG3GuessNumber.java

C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>java PG3GuessNumber
I have a secret number between 20 and 30.
Try to guess it!

Enter your guess (20-30): 22
Too low! Try again.

Enter your guess (20-30): 28
Too high! Try again.

Enter your guess (20-30): 25

Congratulations! You guessed the secret number 25
Number of attempts: 3
```

Program No: 04	**Date:** 29-11-2025

Program Title: Implement searching and sorting elements in an Array.

```
/*
Array Search & Sort
@ Ashbi Saju
Roll no: 16
Date: 29-11-2025
*/

import java.util.Scanner;

public class PG4SearchAndSort {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      // Input array size
      System.out.print("Enter the size of array: ");
      int size = sc.nextInt();

      // Declare array
      int[] arr = new int[size];

      // Input array elements
      System.out.print("Enter " + size + " elements:");
      for (int i = 0; i < size; i++) {
         arr[i] = sc.nextInt();
      }

      // Display original array
      System.out.print("\nOriginal array: ");
      for (int i = 0; i < size; i++) {
         System.out.print(arr[i] + " ");
      }

      // Linear Search on original array
      System.out.print("\n\nEnter element to search: ");
      int searchElement = sc.nextInt();

      boolean found = false;
```

```
int position = -1;

// Traverse array to find element
for (int i = 0; i < size; i++) {
   if (arr[i] == searchElement) {
      found = true;
      position = i;
      break;
   }
}

// Display linear search result
if (found) {
   System.out.println("Element " + searchElement + " found at index " + position);
} else {
   System.out.println("Element " + searchElement + " not found in the array");
}

// Bubble Sort - Sort array in ascending order
for (int i = 0; i < size - 1; i++) {
   for (int j = 0; j < size - i - 1; j++) {
      // Swap if current element is greater than next element
      if (arr[j] > arr[j + 1]) {
         int temp = arr[j];
         arr[j] = arr[j + 1];
         arr[j + 1] = temp;
      }
   }
}

// Display sorted array
System.out.print("\nSorted array (Ascending): ");
for (int i = 0; i < size; i++) {
   System.out.print(arr[i] + " ");
}

// Binary Search on sorted array
System.out.print("\n\nEnter element to search in sorted array: ");
searchElement = sc.nextInt();
```

```
        // Initialize binary search variables
        int left = 0;
        int right = size - 1;
        found = false;
        position = -1;

        // Binary search algorithm
        while (left <= right) {
            // Calculate middle index
            int mid = left + (right - left) / 2;

            // Check if element is at middle
            if (arr[mid] == searchElement) {
                found = true;
                position = mid;
                break;
            }

            // If element is greater, ignore left half
            if (arr[mid] < searchElement) {
                left = mid + 1;
            } else {
                // If element is smaller, ignore right half
                right = mid - 1;
            }
        }

        // Display binary search result
        if (found) {
            System.out.println("Element " + searchElement + " found at index " + position + " using
Binary Search");
        } else {
            System.out.println("Element " + searchElement + " not found in the array");
        }

        sc.close();
    }
}
```

Output

```
C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>java PG4SearchAndSort
Enter the size of array: 5
Enter 5 elements:10 4 7 2 9

Original array: 10 4 7 2 9

Enter element to search: 7
Element 7 found at index 2

Sorted array (Ascending): 2 4 7 9 10

Enter element to search in sorted array: 9
Element 9 found at index 3 using Binary Search
```



Program No: 05	**Date:** 29-11-2025

Program Title: Implement a Java program to pass an array in different ways.

```
/*
Array Passing Methods
@ Ashbi Saju
Roll no: 16
Date: 29-11-2025
*/

import java.util.Scanner;

class Arraypassing {

   static void printarray(int arr[]) {
      System.out.println("array elements :");
      for (int x : arr) {
         System.out.print(x + " ");
      }
      System.out.println();
   }

   static void printarraywithsize(int arr[], int n) {
      System.out.println("array using size :");
      for (int i = 0; i < n; i++) {
         System.out.print(arr[i] + " ");
      }
      System.out.println();
   }

   static void printelement(int element) {
      System.out.println("Individual element :" + element);
   }

   public static void main(String args[]) {
      Scanner sc = new Scanner(System.in);

      System.out.println("Enter the array size :");
      int n = sc.nextInt();
```

```
        int arr[] = new int[n];

        System.out.println("Enter " + n + " elements :");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        printarray(arr);

        printarraywithsize(arr, n);

        System.out.println("Passing elements one by one:");
        for (int i = 0; i < n; i++) {
            printelement(arr[i]);
        }

        sc.close();
    }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG5PassArray.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG5PassArray
Enter the array size: 5
Enter 5 elements: 2 4 6 8 10
Array elements:
2 4 6 8 10
Array using size:
2 4 6 8 10
Passing elements one by one:
Individual element: 2
Individual element: 4
Individual element: 6
Individual element: 8
Individual element: 10
```

Program No: 06	**Date:** 01-12-2025

Program Title: Write a Java program to add and multiply two matrices.

```
/*
Matrix Operations
@ Ashbi Saju
Roll no: 16
Date: 01-12-2025
*/

import java.util.Scanner;

public class PG6MatrixOperations {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      // Input dimensions for first matrix
      System.out.print("Enter rows of first matrix: ");
      int r1 = sc.nextInt();
      System.out.print("Enter columns of first matrix: ");
      int c1 = sc.nextInt();

      // Input dimensions for second matrix
      System.out.print("Enter rows of second matrix: ");
      int r2 = sc.nextInt();
      System.out.print("Enter columns of second matrix: ");
      int c2 = sc.nextInt();

      // Declare matrices
      int[][] matrix1 = new int[r1][c1];
      int[][] matrix2 = new int[r2][c2];

      // Input elements of first matrix
      System.out.println("\nEnter elements of first matrix:");
      for (int i = 0; i < r1; i++) {
         for (int j = 0; j < c1; j++) {
            matrix1[i][j] = sc.nextInt();
         }
      }
```

```
// Input elements of second matrix
System.out.println("Enter elements of second matrix:");
for (int i = 0; i < r2; i++) {
   for (int j = 0; j < c2; j++) {
      matrix2[i][j] = sc.nextInt();

   }
}

// Display first matrix
System.out.println("\nFirst Matrix:");
for (int i = 0; i < r1; i++) {
   for (int j = 0; j < c1; j++) {
      System.out.print(matrix1[i][j] + " ");
   }
   System.out.println();
}

// Display second matrix
System.out.println("\nSecond Matrix:");
for (int i = 0; i < r2; i++) {
   for (int j = 0; j < c2; j++) {
      System.out.print(matrix2[i][j] + " ");
   }
   System.out.println();
}

// Matrix Addition
// Check if addition is possible (dimensions must be same)
if (r1 == r2 && c1 == c2) {
   int[][] sum = new int[r1][c1];

   // Add corresponding elements
   for (int i = 0; i < r1; i++) {
      for (int j = 0; j < c1; j++) {
         sum[i][j] = matrix1[i][j] + matrix2[i][j];
      }
   }

   // Display sum matrix
   System.out.println("\nSum of matrices:");
```

```
    for (int i = 0; i < r1; i++) {
        for (int j = 0; j < c1; j++) {
            System.out.print(sum[i][j] + " ");
        }
        System.out.println();
    }
} else {
    System.out.println("\nMatrix addition not possible! Dimensions must be same.");
}



// Matrix Multiplication
// Check if multiplication is possible (columns of first = rows of second)
if (c1 == r2) {
    int[][] product = new int[r1][c2];

    // Multiply matrices
    for (int i = 0; i < r1; i++) {
        for (int j = 0; j < c2; j++) {
            product[i][j] = 0;
            // Calculate dot product of row i and column j
            for (int k = 0; k < c1; k++) {
                product[i][j] += matrix1[i][k] * matrix2[k][j];
            }
        }
    }

    // Display product matrix
    System.out.println("\nProduct of matrices:");
    for (int i = 0; i < r1; i++) {
        for (int j = 0; j < c2; j++) {
            System.out.print(product[i][j] + " ");
        }
        System.out.println();
    }
} else {
    System.out.println("\nMatrix multiplication not possible!");
    System.out.println("Columns of first matrix must equal rows of second matrix.");
}
```

```
        sc.close();
    }
}
```

Output

```
C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>javac PG6MatrixOperations.java

C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>java PG6MatrixOperations
Enter rows of first matrix: 2
Enter columns of first matrix: 2
Enter rows of second matrix: 2
Enter columns of second matrix: 2

Enter elements of first matrix:
1 2
3 4
Enter elements of second matrix:
5 6
7 8

First Matrix:
1 2
3 4

Second Matrix:
5 6
7 8

Sum of matrices:
6 8
10 12

Product of matrices:
19 22
43 50
```

Program No: 07	**Date:** 01-12-2025

Program Title: A function takes two integer arguments and returns the maximum. Use this function to find the maximum of three numbers.

```
/*
Max of Three
@ Ashbi Saju
Roll no: 16
Date: 01-12-2025
*/

import java.util.Scanner;

// Program to find maximum using function overloading
public class PG7MaxOfThree {

   // Function to find maximum of two integers
   public static int findMax(int a, int b) {
      // Return the larger of two numbers
      if (a > b) {
         return a;
      } else {
         return b;
      }
   }

   // Overloaded function to find maximum of three integers
   // Uses the two-parameter findMax function
   public static int findMax(int a, int b, int c) {
      // Find max of first two numbers
      int maxOfTwo = findMax(a, b);

      // Find max of result and third number
      return findMax(maxOfTwo, c);
   }

   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      // Input two numbers
      System.out.print("Enter first number: ");
```

```
        int num1 = sc.nextInt();

        System.out.print("Enter second number: ");
        int num2 = sc.nextInt();

        // Find maximum of two numbers using two-parameter function
        int maxTwo = findMax(num1, num2);
        System.out.println("\nMaximum of " + num1 + " and " + num2 + " is: " + maxTwo);

        // Input third number
        System.out.print("\nEnter third number: ");
        int num3 = sc.nextInt();

        // Find maximum of three numbers using three-parameter overloaded function
        int maxThree = findMax(num1, num2, num3);
        System.out.println("\nMaximum of " + num1 + ", " + num2 + ", and " + num3 + " is: " +
maxThree);

        sc.close();
    }
}
```

Output

```
C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>javac PG7MaxOfThree.java

C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>java PG7MaxOfThree
Enter first number: 25
Enter second number: 40

Maximum of 25 and 40 is: 40

Enter third number: 30

Maximum of 25, 40, and 30 is: 40
```

Program No: 08	**Date:** 01-12-2025

Program Title: A function takes an integer argument and returns the reverse of the same. Another function takes an integer argument and returns true or false if the number is a palindrome, using the above function.

```
/*
Reverse & Palindrome
@ Ashbi Saju
Roll no: 16
Date: 01-12-2025
*/

import java.util.Scanner;

// Program to reverse a number and check if it's a palindrome using functions
public class PG8ReversePalindrome {

   // Function to reverse an integer
   public static int reverseNumber(int num) {
      int reversed = 0;
      int original = num;

      // Handle negative numbers
      if (num < 0) {
         num = -num;
      }

      // Reverse the number digit by digit
      while (num > 0) {
         int digit = num % 10;
         reversed = reversed * 10 + digit;
         num = num / 10;
      }

      // Return negative if original was negative
      if (original < 0) {
         return -reversed;
      }

      return reversed;
   }
```

```
    // Function to check if a number is palindrome
    // Uses reverseNumber function to compare original and reversed
    public static boolean isPalindrome(int num) {
        // Get the reversed number using reverseNumber function
        int reversed = reverseNumber(num);

        // Check if original equals reversed
        if (num == reversed) {
            return true;
        } else {
            return false;
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Input a number
        System.out.print("Enter a number: ");
        int number = sc.nextInt();

        // Call reverseNumber function
        int reversed = reverseNumber(number);
        System.out.println("\nOriginal number: " + number);
        System.out.println("Reversed number: " + reversed);

        // Call isPalindrome function
        boolean palindrome = isPalindrome(number);

        // Display palindrome result
        if (palindrome) {
            System.out.println("\n" + number + " is a palindrome");
        } else {
            System.out.println("\n" + number + " is not a palindrome");
        }

        sc.close();
    }
}
```

Output

```
C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>javac PG8ReversePalindrome.java

C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>java PG8ReversePalindrome
Enter a number: 121

Original number: 121
Reversed number: 121

121 is a palindrome

C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>java PG8ReversePalindrome
Enter a number: 345

Original number: 345
Reversed number: 543

345 is not a palindrome
```



Program No: 09	**Date:** 03-12-2025

Program Title: The 15-puzzle (also known as the sliding puzzle) consists of a 4×4 grid containing 15 numbered tiles (from 1 to 15) and one empty space. The objective is to rearrange the tiles by sliding them horizontally or vertically into the empty space to achieve the correct ascending order, as shown below:

1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 _

```
/*
15-Puzzle Solver Game
@ Ashbi Saju
Roll no: 16
Date: 06-12-2025
*/

import java.util.Scanner;
class PG9FifteenPuzzle {
   int SIZE = 4;
   int[][] board;
   int emptyRow;
   int emptyCol;

   public PG9FifteenPuzzle() {
      board = new int[SIZE][SIZE];
      initializeBoard();
   }
   public void initializeBoard() {
      int num = 1;
      for (int i = 0; i < SIZE; i++) {
         for (int j = 0; j < SIZE; j++) {
            if (i == SIZE - 1 && j == SIZE - 1) {
               board[i][j] = 0; // Empty space
               emptyRow = i;
               emptyCol = j;
            } else {
               board[i][j] = num++;
            }
         }
```

```
    }
  }
  public void displayBoard() {
    for (int i = 0; i < SIZE; i++) {
      for (int j = 0; j < SIZE; j++) {
        if (board[i][j] == 0) {
          System.out.print("_ ");
        } else {
          System.out.print(board[i][j] + " ");
        }
      }
      System.out.println();
    }
  }
  public boolean moveTile(int tile) {
    int tileRow = -1;
    int tileCol = -1;

    // Find the position of the tile
    for (int i = 0; i < SIZE; i++) {
      for (int j = 0; j < SIZE; j++) {
        if (board[i][j] == tile) {
          tileRow = i;
          tileCol = j;
        }
      }
    }

    // Check if the tile is adjacent to the empty space
    if ((Math.abs(tileRow - emptyRow) == 1 && tileCol == emptyCol) ||
      (Math.abs(tileCol - emptyCol) == 1 && tileRow == emptyRow)) {
      // Swap the tile with the empty space
      board[emptyRow][emptyCol] = tile;
      board[tileRow][tileCol] = 0;
      emptyRow = tileRow;
      emptyCol = tileCol;
      return true;
    }
    return false;
  }
  public static void main(String[] args) {
```

```
        PG9FifteenPuzzle puzzle = new PG9FifteenPuzzle();
        Scanner sc = new Scanner(System.in);
        while (true) {
            puzzle.displayBoard();
            System.out.print("Enter tile number to move (or 0 to exit): ");
            int tile = sc.nextInt();
            if (tile == 0) {
                break;
            }
            if (!puzzle.moveTile(tile)) {
                System.out.println("Invalid move. Try again.");
            }
        }
        sc.close();
    }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG9FifteenPuzzle.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG9FifteenPuzzle
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 _
Enter tile number to move (or 0 to exit): 15
1 2 3 4
5 6 7 8
9 10 11 12
13 14 _ 15
Enter tile number to move (or 0 to exit): 14
1 2 3 4
5 6 7 8
9 10 11 12
13 _ 14 15
Enter tile number to move (or 0 to exit): 10
1 2 3 4
5 6 7 8
9 _ 11 12
13 10 14 15
Enter tile number to move (or 0 to exit): 0
```

Program No: 10	**Date:** 03-12-2025

Program Title: Display from n - 1 and 1-n using recursive functions.

```
/*
Recursive Number Display
@ Ashbi Saju
Roll no: 16
Date: 06-11-2025
*/

import java.util.Scanner;

// Program to display numbers in descending and ascending order using recursive functions
public class PG10RecursiveDisplay {

   // Recursive function to display from n to 1 (descending)
   public static void displayDescending(int n) {
      // Base case: if n becomes 0, stop recursion
      if (n == 0) {
         return;
      }

      // Print current number
      System.out.print(n + " ");

      // Recursive call with n-1
      displayDescending(n - 1);
   }

   // Recursive function to display from 1 to n (ascending)
   public static void displayAscending(int n) {
      // Base case: if n becomes 0, stop recursion
      if (n == 0) {
         return;
      }

      // Recursive call first with n-1
      displayAscending(n - 1);

      // Print current number after recursive call
```

```
        System.out.print(n + " ");
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Input the value of n
        System.out.print("Enter n: ");
        int n = sc.nextInt();

        // Display from n to 1 using recursive function
        System.out.print("\nNumbers from " + n + " to 1 (Descending): ");
        displayDescending(n);

        // Display from 1 to n using recursive function
        System.out.print("\n\nNumbers from 1 to " + n + " (Ascending): ");
        displayAscending(n);

        sc.close();
    }
}
```

Output

```
C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>javac PG10RecursiveDisplay.java

C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>java PG10RecursiveDisplay
Enter n: 5

Numbers from 5 to 1 (Descending): 5 4 3 2 1

Numbers from 1 to 5 (Ascending): 1 2 3 4 5
```

Program No: 11	**Date:** 03-12-2025

Program Title: Create a program for adding, subtracting, multiplying and dividing two numbers using object-oriented concepts.

```
/*
OOPs Calculator
@ Ashbi Saju
Roll no: 16
Date: 06-12-2025
*/

import java.util.Scanner;

// Calculator class with two numbers and arithmetic operations
class Calculator {
    // Instance variables to store two numbers
    private int num1;
    private int num2;

    // Constructor to initialize the numbers
    public Calculator(int num1, int num2) {
        this.num1 = num1;
        this.num2 = num2;
    }

    // Method to perform addition
    public int add() {
        return num1 + num2;
    }

    // Method to perform subtraction
    public int subtract() {
        return num1 - num2;
    }

    // Method to perform multiplication
    public int multiply() {
        return num1 * num2;
    }

    // Method to perform division
```

```
    public int divide() {
        // Check for division by zero
        if (num2 == 0) {
            System.out.println("Error: Division by zero!");
            return 0;
        }
        return num1 / num2;
    }
}

// Main class to test Calculator
public class PG11CalculatorOOPs {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Input two numbers
        System.out.print("Enter first number: ");
        int a = sc.nextInt();

        System.out.print("Enter second number: ");
        int b = sc.nextInt();

        // Create Calculator object with two numbers
        Calculator calc = new Calculator(a, b);

        // Display menu
        System.out.println("\n--- Menu ---");
        System.out.println("1. Addition");
        System.out.println("2. Subtraction");
        System.out.println("3. Multiplication");
        System.out.println("4. Division");
        System.out.print("\nEnter your choice: ");
        int choice = sc.nextInt();

        // Perform operation based on user choice
        switch (choice) {
            case 1:
                System.out.println("\nResult: " + a + " + " + b + " = " + calc.add());
                break;
            case 2:
                System.out.println("\nResult: " + a + " - " + b + " = " + calc.subtract());
```

```
            break;
        case 3:
            System.out.println("\nResult: " + a + " * " + b + " = " + calc.multiply());
            break;
        case 4:
            if (b != 0) {
                System.out.println("\nResult: " + a + " / " + b + " = " + calc.divide());
            } else {
                System.out.println("\nError: Cannot divide by zero");
            }
            break;
        default:
            System.out.println("\nInvalid choice!");
    }
    sc.close();
  }
}
```

Output

```
C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>javac PG11CalculatorOOPs.java

C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>java PG11CalculatorOOPs
Enter first number: 20
Enter second number: 5

--- Menu ---
1. Addition
2. Subtraction
3. Multiplication
4. Division

Enter your choice: 1

Result: 20 + 5 = 25

C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>java PG11CalculatorOOPs
Enter first number: 20
Enter second number: 5

--- Menu ---
1. Addition
2. Subtraction
3. Multiplication
4. Division

Enter your choice: 2

Result: 20 - 5 = 15
```

```
C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>java PG11CalculatorOOPs
Enter first number: 20
Enter second number: 5

--- Menu ---
1. Addition
2. Subtraction
3. Multiplication
4. Division

Enter your choice: 3

Result: 20 * 5 = 100

C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>java PG11CalculatorOOPs
Enter first number: 20
Enter second number: 5

--- Menu ---
1. Addition
2. Subtraction
3. Multiplication
4. Division

Enter your choice: 4

Result: 20 / 5 = 4
```

Program No: 12	**Date:** 06-12-2025

Program Title: Create a class complex having real and imaginary as data members and provide function for read, display, add and multiply.

```
/*
Complex Number Class
@ Ashbi Saju
Roll no: 16
Date: 06-12-2025
*/

import java.util.Scanner;

// Complex class with real and imaginary parts
class Complex {
    // Instance variables for real and imaginary parts
    private int real;
    private int imaginary;

    // Method to read complex number from user
    public void read(Scanner sc) {
        System.out.print("Enter real part: ");
        this.real = sc.nextInt();
        System.out.print("Enter imaginary part: ");
        this.imaginary = sc.nextInt();
    }

    // Method to display complex number
    public void display() {
        // Display in format: a + bi or a - bi
        if (imaginary >= 0) {
            System.out.println(real + " + " + imaginary + "i");
        } else {
            System.out.println(real + " - " + (-imaginary) + "i");
        }
    }

    // Method to add two complex numbers
    // (a + bi) + (c + di) = (a+c) + (b+d)i
    public Complex add(Complex c) {
        Complex result = new Complex();
```

```
        result.real = this.real + c.real;
        result.imaginary = this.imaginary + c.imaginary;
        return result;
    }

    // Method to multiply two complex numbers
    // (a + bi) * (c + di) = (ac - bd) + (ad + bc)i
    public Complex multiply(Complex c) {
        Complex result = new Complex();
        result.real = (this.real * c.real) - (this.imaginary * c.imaginary);
        result.imaginary = (this.real * c.imaginary) + (this.imaginary * c.real);
        return result;
    }
}

// Main class to test Complex number operations
public class PG12ComplexNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Create first complex number
        Complex c1 = new Complex();
        System.out.println("Enter first complex number:");
        c1.read(sc);

        // Create second complex number
        Complex c2 = new Complex();
        System.out.println("\nEnter second complex number:");
        c2.read(sc);

        // Display both complex numbers
        System.out.print("\nFirst complex number: ");
        c1.display();

        System.out.print("Second complex number: ");
        c2.display();

        // Add two complex numbers
        Complex sum = c1.add(c2);
        System.out.print("\nSum: ");
        sum.display();
```

```
        // Multiply two complex numbers
        Complex product = c1.multiply(c2);
        System.out.print("Product: ");
        product.display();

        sc.close();
    }
}
```

Output

```
C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>javac PG12ComplexNumber.java

C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>java PG12ComplexNumber
Enter first complex number:
Enter real part: 3
Enter imaginary part: 2

Enter second complex number:
Enter real part: 1
Enter imaginary part: 4

First complex number: 3 + 2i
Second complex number: 1 + 4i

Sum: 4 + 6i
Product: -5 + 14i
```

Program No: 13	**Date:** 06-12-2025

Program Title: Create a class for CString having a string data member and provide functions for read, display, compare(return Boolean value),add and concatenate.

```
/*
String Operations
@ Ashbi Saju
Roll no: 16
Date: 06-12-2025
*/

import java.util.Scanner;

// CString class with string data member and various operations
class CString {
    // Instance variable to store string
    private String str;

    // Method to read string from user
    public void read(Scanner sc) {
        System.out.print("Enter string: ");
        this.str = sc.nextLine();
    }

    // Method to display string
    public void display() {
        System.out.println(str);
    }

    // Method to compare two strings (returns boolean)
    public boolean compare(CString other) {
        // Returns true if strings are equal, false otherwise
        return this.str.equals(other.str);
    }

    // Method to add (concatenate with space) two strings
    public CString add(CString other) {
        CString result = new CString();
        result.str = this.str + " " + other.str;
        return result;
    }
```

```
    // Method to concatenate two strings (without space)
    public CString concatenate(CString other) {
        CString result = new CString();
        result.str = this.str + other.str;
        return result;
    }

}

// Main class to test CString operations
public class PG13StringOperations {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Create first CString object
        CString s1 = new CString();
        System.out.println("Enter first string:");
        s1.read(sc);

        // Create second CString object
        CString s2 = new CString();
        System.out.println("\nEnter second string:");
        s2.read(sc);

        // Display both strings
        System.out.print("\nFirst string: ");
        s1.display();

        System.out.print("Second string: ");
        s2.display();

        // Compare two strings
        boolean isEqual = s1.compare(s2);
        System.out.println("\nAre strings equal? " + isEqual);

        // Add two strings (with space)
        CString added = s1.add(s2);
        System.out.print("Added string (with space): ");
        added.display();
```

```
        // Concatenate two strings (without space)
        CString concatenated = s1.concatenate(s2);
        System.out.print("Concatenated string (without space): ");
        concatenated.display();

        sc.close();
    }
}
```

Output

```
C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>javac PG13StringOperations.java

C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>java PG13StringOperations
Enter first string:
Enter string: Hello World

Enter second string:
Enter string: Welcome

First string: Hello World
Second string: Welcome

Are strings equal? false
Added string (with space): Hello World Welcome
Concatenated string (without space): Hello WorldWelcome
```

Program No: 14	**Date:** 06-12-2025

Program Title: Create a class for a student having (sno, sname, sprogram) [example sno - 1, sname - sanjay, sprogram - MCA]. Provide functions for reading and displaying the same information. Also provide a function for comparing two student objects for equality.

```
/*
Student Class Model
@ Ashbi Saju
Roll no: 16
Date: 06-12-2025
*/

import java.util.Scanner;

// Student class with student number, name and program
class Student {
   // Instance variables for student details
   private int sno;
   private String sname;
   private String sprogram;

   // Method to read student details from user
   public void read(Scanner sc) {
      System.out.print("Enter student number: ");
      this.sno = sc.nextInt();
      sc.nextLine(); // Consume newline

      System.out.print("Enter student name: ");
      this.sname = sc.nextLine();

      System.out.print("Enter student program: ");
      this.sprogram = sc.nextLine();
   }

   // Method to display student details
   public void display() {
      System.out.println("Student Number: " + sno);
      System.out.println("Student Name: " + sname);
      System.out.println("Student Program: " + sprogram);
   }
```

```
    // Method to compare two student objects for equality
    // Two students are equal if all three attributes match
    public boolean compare(Student other) {
        // Check student number
        if (this.sno != other.sno) {
            return false;
        }

        // Check student name
        if (!this.sname.equals(other.sname)) {
            return false;
        }

        // Check student program
        if (!this.sprogram.equals(other.sprogram)) {
            return false;
        }

        // If all checks pass, students are equal
        return true;
    }
}

// Main class to test Student operations
public class PG14StudentCompare {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Create first student object
        Student student1 = new Student();
        System.out.println("Enter details of first student:");
        student1.read(sc);

        // Create second student object
        Student student2 = new Student();
        System.out.println("\nEnter details of second student:");
        student2.read(sc);

        // Display first student details
        System.out.println("\n--- First Student Details ---");
        student1.display();
```

```
        // Display second student details
        System.out.println("\n--- Second Student Details ---");
        student2.display();

        // Compare two student objects
        boolean isEqual = student1.compare(student2);
        System.out.println("\n--- Comparison Result ---");
        if (isEqual) {
            System.out.println("Both students are equal");
        } else {
            System.out.println("Students are not equal");
        }

        sc.close();
    }
}
```

Output

```
C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>javac PG14StudentCompare.java

C:\Users\ASUS\Documents\00 MCA\SEM 2\Java>java PG14StudentCompare
Enter details of first student:
Enter student number: 101
Enter student name: Ashbi
Enter student program: MCA

Enter details of second student:
Enter student number: 102
Enter student name: Ayush
Enter student program: Msc. CS

--- First Student Details ---
Student Number: 101
Student Name: Ashbi
Student Program: MCA

--- Second Student Details ---
Student Number: 102
Student Name: Ayush
Student Program: Msc. CS

--- Comparison Result ---
Students are not equal
```

Program No: 15	**Date:** 08-12-2025

Program Title: Program to implement the ISA and HASA relationship.

```
/*
Class Relationships
@ Ashbi Saju
Roll no: 16
Date: 08-12-2025
*/

import java.util.Scanner;

// HASA relationship class
class Address {
   String city;
   String state;

   Address(String city, String state) {
      this.city = city;
      this.state = state;
   }

   void displayAddress() {
      System.out.println("City: " + city);
      System.out.println("State: " + state);
   }
}

// Base class (ISA relationship)
class Person {
   String name;
   int age;

   Person(String name, int age) {
      this.name = name;
      this.age = age;
   }

   void displayPerson() {
      System.out.println("Name: " + name);
```

```
        System.out.println("Age: " + age);
    }
}

// Derived class (ISA + HASA)
class Student extends Person {   // ISA relationship
    int rollNo;
    Address address;           // HASA relationship

    Student(String name, int age, int rollNo, Address address) {
        super(name, age);
        this.rollNo = rollNo;
        this.address = address;
    }

    void displayStudent() {
        displayPerson();
        System.out.println("Roll No: " + rollNo);
        address.displayAddress();
    }
}

public class PG15ISAHASADemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter student details:");
        System.out.print("\nEnter student name: ");
        String name = sc.next();
        System.out.print("Enter student age: ");
        int age = sc.nextInt();
        System.out.print("Enter student roll number: ");
        int rollNo = sc.nextInt();

        System.out.println("\nEnter address details:");
        System.out.print("\nEnter City: ");
        String city = sc.next();
        System.out.print("Enter State: ");
        String state = sc.next();

        Address addr = new Address(city, state);
```

```
        Student s = new Student(name, age, rollNo, addr);

        s.displayStudent();
    }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG15ISAHASADemo.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG15ISAHASADemo
Enter student details:

Enter student name: Ashbi
Enter student age: 21
Enter student roll number: 016

Enter address details:

Enter City: Kochi
Enter State: Kerala

Student Details:
Name: Ashbi
Age: 21
Roll No: 16
City: Kochi
State: Kerala
```

Program No: 16	**Date:** 08-12-2025

Program Title: Program to overcome function overriding in Java.

```
/*
Method Overriding Control
@ Ashbi Saju
Roll no: 16
Date: 13-12-2025
*/

class Parent {
   void display() {
      System.out.println("This is Parent class display method");
   }
}

class Child extends Parent {
   void display() {
      System.out.println("This is Child class display method");

      // Calling parent class method
      super.display();
   }
}

public class PG16OverrideDemo {
   public static void main(String[] args) {

      Child obj = new Child();
      obj.display();
   }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG16OverrideDemo.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG16OverrideDemo
This is Child class display method
This is Parent class display method
```

Program No: 17	**Date:** 08-12-2025

Program Title: Demonstrate the use of a super keyword
a. To overcome function overriding
b. Passing arguments to the base class constructor

```
/*
Super Method Call
@ Ashbi Saju
Roll no: 16
Date: 13-12-2025
*/

import java.util.Scanner;
class Employee {
   String name;
   double salary;

   Employee(String name, double salary) {    // Base class constructor
      this.name = name;
      this.salary = salary;
   }

   void showDetails() {
      System.out.println("Employee Name: " + name);
      System.out.println("Salary: " + salary);
   }
}

class Manager extends Employee {
   String department;

   // Passing arguments to base class constructor
   Manager(String name, double salary, String department) {
      super(name, salary);   // calls Employee constructor
      this.department = department;
   }

   void showDetails() {            // Overcoming function overriding
      System.out.println("Department: " + department);
      super.showDetails();       // calls Employee method
   }
}
```

```
public class PG17SuperDemo {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter employee name: ");
      String name = sc.next();
      System.out.print("Enter employee salary: ");
      double salary = sc.nextDouble();
      System.out.print("Enter department: ");
      String department = sc.next();
      Manager m = new Manager(name, salary, department);
      m.showDetails();
   }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG17SuperDemo.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG17SuperDemo
Enter employee name: Ashbi
Enter employee salary: 50000
Enter department: IT

Employee Details:
Department: IT
Employee Name: Ashbi
Salary: 50000.0
```

Program No: 18	**Date:** 08-12-2025

Program Title: Create a class for an employee having eno, ename and esal as data members. Provide functions for reading, displaying, and comparing by eno. Accept n employees' information in the main function, display the same, and search for an employee whose eno is given.

```
/*
Employee Management System
@ Ashbi Saju
Roll no: 16
Date: 13-12-2025
*/

import java.util.Scanner;
class Employee1 {
   int eno;
   String ename;
   double esal;

   Employee1(int eno, String ename, double esal) {
      this.eno = eno;
      this.ename = ename;
      this.esal = esal;
   }

   void display() {
      System.out.println("Employee No: " + eno);
      System.out.println("Employee Name: " + ename);
      System.out.println("Employee Salary: " + esal);
   }

   boolean compareByEno(int searchEno) {
      return this.eno == searchEno;
   }
}
public class PG18EmployeeDemo {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter number of employees: ");
      int n = sc.nextInt();
```

```
        Employee1[] employees = new Employee1[n];

        for (int i = 0; i < n; i++) {
            System.out.println("\nEnter details for employee " + (i + 1) + ":");
            System.out.print("\nEnter employee no: ");
            int eno = sc.nextInt();
            System.out.print("Enter employee name: ");
            String ename = sc.next();
            System.out.print("Enter employee salary: ");
            double esal = sc.nextDouble();
            employees[i] = new Employee1(eno, ename, esal);
        }
        System.out.println("Employee Details:");
        for (Employee1 emp : employees) {
            emp.display();
            System.out.println();
        }

        System.out.print("Enter employee no to search: ");
        int searchEno = sc.nextInt(); // Example eno to search
        System.out.println("\nSearching for Employee with No: " + searchEno);
        boolean found = false;
        for (Employee1 emp : employees) {
            if (emp.compareByEno(searchEno)) {
                System.out.println("Employee Found!\n");
                emp.display();
                found = true;
                break;
            }
        }
        if (!found) {
            System.out.println("Employee with No " + searchEno + " not found.");
        }
    }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG18EmployeeDemo.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG18EmployeeDemo
Enter number of employees: 2

Enter details for employee 1:

Enter employee no: 101
Enter employee name: Ashbi
Enter employee salary: 50000

Enter details for employee 2:

Enter employee no: 102
Enter employee name: Ayush
Enter employee salary: 60000

Employee Details:
Employee No: 101
Employee Name: Ashbi
Employee Salary: 50000.0

Employee No: 102
Employee Name: Ayush
Employee Salary: 60000.0

Enter employee no to search: 101

Searching for Employee with No: 101
Employee Found!

Employee No: 101
Employee Name: Ashbi
Employee Salary: 50000.0
```

Program No: 19	**Date:** 10-12-2025

Program Title: Program to implement run-time polymorphism in Java(interface).

```
/*
Runtime Polymorphism
@ Ashbi Saju
Roll no: 16
Date: 13-12-2025
*/

interface Animal {
   void sound();
}
class Dog implements Animal {
   public void sound() {
      System.out.println("Woof");
   }
}
class Cat implements Animal {
   public void sound() {
      System.out.println("Meow");
   }
}
public class PG19PolymorphismDemo {
   public static void main(String[] args) {
      Animal myDog = new Dog();
      Animal myCat = new Cat();

      myDog.sound();
      myCat.sound();
   }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG19PolymorphismDemo.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG19PolymorphismDemo
Woof
Meow
```

Program No: 20	**Date:** 10-12-2025

Program Title: Create an interface ishape having two prototypes draw and mov, create two classes circle and rectangle which implements the above interface. In the main function only a reference object of ishape is created, depends on the choice entered by the user, it points to the respective memory location.

```
/*
Shape Interface Program
@ Ashbi Saju
Roll no: 16
Date: 15-12-2025
*/

import java.util.Scanner;
interface ishape {
   void draw();
   void mov();
}
class circle implements ishape {
   public void draw() {
      System.out.println("Drawing Circle");
   }
   public void mov() {
      System.out.println("Moving Circle");
   }
}
class rectangle implements ishape {
   public void draw() {
      System.out.println("Drawing Rectangle");
   }
   public void mov() {
      System.out.println("Moving Rectangle");
   }
}

public class PG20ishapeDemo {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.println("Enter 1 for Circle or 2 for Rectangle:");
      int choice = sc.nextInt();
```

```
        ishape shape;
        if (choice == 1) {
            shape = new circle();
        } else {
            shape = new rectangle();
        }

        shape.draw();
        shape.mov();

        sc.close();
    }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG20ishapeDemo.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG20ishapeDemo
Enter 1 for Circle or 2 for Rectangle:
1
Drawing Circle
Moving Circle

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG20ishapeDemo
Enter 1 for Circle or 2 for Rectangle:
2
Drawing Rectangle
Moving Rectangle
```

Program No: 21	**Date:** 10-12-2025

Program Title: Write a program using multiple catch statements / predefined Exceptions.

```
/*
Multiple Exception Handling
@ Ashbi Saju
Roll no: 16
Date: 15-12-2025
*/

import java.io.*;

public class PG21MultipleCatchDemo {
   public static void main(String[] args) {
      DataInputStream sc = new DataInputStream(System.in);
      try {
         System.out.print("Enter first number:");
         String s = sc.readLine();
         int a = Integer.parseInt(s);
         System.out.print("Enter second number:");
         s = sc.readLine();
         int b = Integer.parseInt(s);
         int result = a / b;
         System.out.println("Result: " + result);
      } catch (IOException e1) {
         System.out.println("IO Exception occurred: " + e1);
      } catch (NumberFormatException e2) {
         System.out.println("Number Format Exception occurred: " + e2);
      } catch (ArithmeticException e3) {
         System.out.println("Arithmetic Exception occurred: " + e3);
      } catch (Exception e4) {
         System.out.println("Other Exception occurred: " + e4);
      }
   }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG21MultipleCatchDemo.java
Note: PG21MultipleCatchDemo.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG21MultipleCatchDemo
Enter first number:10
Enter second number:0
Arithmetic Exception occurred: java.lang.ArithmeticException: / by zero

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG21MultipleCatchDemo
Enter first number:20
Enter second number:2
Result: 10
```

Program No: 22	**Date:** 15-12-2025

Program Title: Write a program to implement a user-defined Exception.

```
/*
User Exception Handling
@ Ashbi Saju
Roll no: 16
Date: 15-12-2025
*/

import java.util.Scanner;
class InvalidAgeException extends Exception {
   public InvalidAgeException(String message) {
      super(message);
   }
}
public class PG22UserDefinedExceptionDemo {
   public static void validateAge(int age) throws InvalidAgeException {
      if (age < 18) {
         throw new InvalidAgeException("Age must be at least 18.");
      } else {
         System.out.println("Valid age: " + age);
      }
   }

   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter your age: ");
      int age = sc.nextInt();
      sc.close();
      try {
         validateAge(age);
      } catch (InvalidAgeException e) {
         System.out.println("Caught Exception: " + e.getMessage());
      }
   }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG22UserDefinedExceptionDemo.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG22UserDefinedExceptionDemo
Enter your age: 21
Valid age: 21

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG22UserDefinedExceptionDemo
Enter your age: 16
Caught Exception: Age must be at least 18.
```

Program No: 23	**Date:** 15-12-2025

Program Title: Write a program to implement re-throw and finally.

```
/*
Exception Flow Control
@ Ashbi Saju
Roll no: 16
Date: 15-12-2025
*/

//Write a program to implement re-throw and finally.

import java.io.*;
public class PG23RethrowFinallyDemo {
   public static void main(String[] args) {
      DataInputStream sc = new DataInputStream(System.in);
      try {
         System.out.print("Enter first number:");
         String s = sc.readLine();
         int a = Integer.parseInt(s);
         System.out.print("Enter second number:");
         s = sc.readLine();
         int b = Integer.parseInt(s);
         int result = a / b;
         System.out.println("Result: " + result);
      } catch (ArithmeticException e) {
         System.out.println("Arithmetic Exception occurred: " + e);
         throw e; // Re-throwing the exception
      } catch (NumberFormatException e) {
         System.out.println("Number Format Exception occurred: " + e);
         throw e; // Re-throwing the exception
      } catch (IOException e) {
         System.out.println("IO Exception occurred: " + e);
         throw new RuntimeException(e); // Wrapping and re-throwing
      } finally {
         System.out.println("Execution of try-catch block is complete.");
      }
   }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG23RethrowFinallyDemo.java
Note: PG23RethrowFinallyDemo.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG23RethrowFinallyDemo
Enter first number:10
Enter second number:0
Arithmetic Exception occurred: java.lang.ArithmeticException: / by zero
Execution of try-catch block is complete.
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at PG23RethrowFinallyDemo.main(PG23RethrowFinallyDemo.java:14)

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG23RethrowFinallyDemo
Enter first number:10
Enter second number:2
Result: 5
Execution of try-catch block is complete.
```

Program No: 24	**Date:** 15-12-2025

Program Title: Create a user-defined exception class for handling invalid password errors. Create another class to read and validate a password based on the following conditions:

1. The password must be at least 8 characters long.
2. It must contain at least one lowercase letter.
3. It must contain at least one uppercase letter.
4. It must contain at least one digit.
5. It must contain at least one special character from the set
   # , * , ! , { , } , (,) , [,] , =
6. The password must not contain consecutive repeating characters.
7. If any condition fails, throw a user-defined exception with an appropriate message; otherwise, display that the password is valid.

```
/*
Password Validation System
@ Ashbi Saju
Roll no: 16
Date: 20-11-2025
*/

import java.util.Scanner;
class InvalidPasswordException extends Exception {
   public InvalidPasswordException(String message) {
      super(message);
   }
}
public class PG24PasswordValidator {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a password to validate: ");
      String password = sc.nextLine();

      try {
         validatePassword(password);
         System.out.println("Password is valid.");
      } catch (InvalidPasswordException e) {
         System.out.println("Invalid Password: " + e.getMessage());
      } finally {
         sc.close();
      }
   }
```

```
  public static void validatePassword(String password) throws InvalidPasswordException {
    if (password.length() < 8) {
      throw new InvalidPasswordException("Password must be at least 8 characters long.");
    }

    if (!password.matches(".*[a-z].*")) {
      throw new InvalidPasswordException("Password must contain at least one lowercase
letter.");
    }

    if (!password.matches(".*[A-Z].*")) {
      throw new InvalidPasswordException("Password must contain at least one uppercase
letter.");
    }

    if (!password.matches(".*[0-9].*")) {
      throw new InvalidPasswordException("Password must contain at least one digit.");
    }

    if (!password.matches(".*[#*!{}()\\[\\]=].*")) {
      throw new InvalidPasswordException("Password must contain at least one special
character (#, *, !, {, }, (, ), [, ], =).");
    }

    for (int i = 1; i < password.length(); i++) {
      if (password.charAt(i) == password.charAt(i - 1)) {
        throw new InvalidPasswordException("Password must not contain consecutive
repeating characters.");
      }
    }
  }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG24PasswordValidator.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG24PasswordValidator
Enter a password to validate: Ab3#xP9!
Password is valid.

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG24PasswordValidator
Enter a password to validate: Ab3#x
Invalid Password: Password must be at least 8 characters long.

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG24PasswordValidator
Enter a password to validate: ab3#xk9!
Invalid Password: Password must contain at least one uppercase letter.

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG24PasswordValidator
Enter a password to validate: Ab39xk92
Invalid Password: Password must contain at least one special character (#, *,
 !, {, }, (, ), [, ], =).

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG24PasswordValidator
Enter a password to validate: Ab33#xP9
Invalid Password: Password must not contain consecutive repeating characters.
```

Program No: 25	**Date:** 17-12-2025

Program Title: Implement threading by inheriting Thread class.

```
/*
Thread Class Implementation
@ Ashbi Saju
Roll no: 16
Date: 20-11-2025
*/

import java.util.Scanner;

class ThreadClass extends Thread {
   int n;

   public ThreadClass(int n) {
      this.n = n;
   }

   public void run() {
      for (int i = 1; i <= n; i++) {
         System.out.println(i);
      }
   }
}

class PG25ThreadClassDemo {
   public static void main(String args[]) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a number for Thread 1: ");
      int num1 = sc.nextInt();
      System.out.print("Enter a number for Thread 2: ");
      int num2 = sc.nextInt();
      ThreadClass t1 = new ThreadClass(num1);
      ThreadClass t2 = new ThreadClass(num2);
      t1.start();
      t2.start();
   }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG25ThreadClassDemo.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG25ThreadClassDemo
Enter a number for Thread 1: 5
Enter a number for Thread 2: 4
1
2
3
4
5
1
2
3
4
```



Program No: 26	**Date:** 17-12-2025

Program Title: Write a Java program to create threads using the Runnable interface.

```
/*
Runnable Interface Threads
@ Ashbi Saju
Roll no: 16
Date: 20-11-2025
*/

import java.util.Scanner;

class ThreadInterface implements Runnable {
   int n;
   public ThreadInterface(int n) {
      this.n = n;
   }
   public void run() {
      for (int i = 1; i <= n; i++) {
         System.out.println(i);
      }
   }
}

class PG26ThreadInterfaceDemo {
   public static void main(String args[]) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a number for Thread 1: ");
      int num1 = sc.nextInt();
      System.out.print("Enter a number for Thread 2: ");
      int num2 = sc.nextInt();
      ThreadInterface t1 = new ThreadInterface(num1);
      ThreadInterface t2 = new ThreadInterface(num2);
      Thread th1 = new Thread(t1);
      Thread th2 = new Thread(t2);
      th1.start();
      th2.start();
   }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG26ThreadInterfaceDemo.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG26ThreadInterfaceDemo
Enter a number for Thread 1: 5
Enter a number for Thread 2: 4
1
2
3
4
5
1
2
3
4
```

Program No: 27	**Date:** 17-12-2025

Program Title: Write a Java program to create two threads, one to display odd numbers and another to display even numbers up to N.

```
/*
Odd Even Threads
@ Ashbi Saju
Roll no: 16
Date: 20-11-2025
*/

import java.util.Scanner;

// Class for Odd
class OddClass extends Thread {
   int n;
   public OddClass(int n) {
      this.n = n;
   }
   public void run() {
      for (int i = 1; i <= n; i++) {
         if (i % 2 != 0)
            System.out.println("\nOdd: " + i);
      }
   }
}

//Class for Even
class EvenClass extends Thread {
   int n;
   public EvenClass(int n) {
      this.n = n;
   }
   public void run() {
      for (int i = 1; i <= n; i++) {
         if (i % 2 == 0)
            System.out.println("\nEven: " + i);
      }
   }
}
```

```
class PG27OddEvenDemo {
   public static void main(String args[]) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a value for n: ");
      int n = sc.nextInt();
      OddClass t1 = new OddClass(n);
      EvenClass t2 = new EvenClass(n);
      t1.start();
      t2.start();
   }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG27OddEvenDemo.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG27OddEvenDemo
Enter a value for n: 6

Odd: 1

Odd: 3

Odd: 5

Even: 2

Even: 4

Even: 6
```

Program No: 28	**Date:** 20-12-2025

Program Title: Search for a given element using n/5 threads.

```
/*
Parallel Search Threads
@ Ashbi Saju
Roll no: 16
Date: 22-11-2025
*/

import java.util.Scanner;

class SearchThread extends Thread {
   int[] arr;
   int start, end, key, threadId;
   int foundIndex = -1;

   SearchThread(int[] arr, int start, int end, int key, int threadId) {
      this.arr = arr;
      this.start = start;
      this.end = end;
      this.key = key;
      this.threadId = threadId;
   }

   public void run() {
      for (int i = start; i < end; i++) {
         if (arr[i] == key) {
            foundIndex = i;
            System.out.println("Element " + key + " found at index " + i +" by Thread-" + threadId);
            break;
         }
      }
   }

   int getFoundIndex() {
      return foundIndex;
   }
}
```

```
public class PG28ParallelSearch {
   public static void main(String[] args) throws InterruptedException {
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter number of elements: ");
      int n = sc.nextInt();

      int[] arr = new int[n];
      System.out.print("Enter elements: ");
      for (int i = 0; i < n; i++) {
         arr[i] = sc.nextInt();
      }

      System.out.print("Enter element to search: ");
      int key = sc.nextInt();

      int threadCount = n / 5;
      int segmentSize = 5;

      SearchThread[] threads = new SearchThread[threadCount];

      for (int i = 0; i < threadCount; i++) {
         int start = i * segmentSize;
         int end = Math.min(start + segmentSize, n);

         threads[i] = new SearchThread(arr, start, end, key, i + 1);
         threads[i].start();
      }

      for (SearchThread t : threads) {
         t.join();
      }

      sc.close();
   }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG28ParallelSearch.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG28ParallelSearch
Enter number of elements: 5
Enter elements: 4 8 2 6 4
Enter element to search: 4
Element 4 found at index 0 by Thread-1

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG28ParallelSearch
Enter number of elements: 20
Enter elements: 5 3 4 2 1 7 6 8 9 5 3 6 4 1 3 5 7 8 3 4
Enter element to search: 6
Element 6 found at index 11 by Thread-3
Element 6 found at index 6 by Thread-2
```



Program No: 29	**Date:** 20-12-2025

Program Title: Develop a program to compute square and cube of a given number using two separate threads.

```
/*
Square Cube Threads
@ Ashbi Saju
Roll no: 16
Date: 22-11-2025
*/

import java.util.Scanner;

class SquareClass extends Thread {
   int n;
   public SquareClass(int n) {
      this.n = n;
   }
   public void run() {
      System.out.println("\nSquare: " + n*n);
   }
}

class CubeClass extends Thread {
   int n;
   public CubeClass(int n) {
      this.n = n;
   }
   public void run() {
      System.out.println("\nCube: " + n*n*n);

   }
}

class PG29SquareCubeDemo {
   public static void main(String args[]) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter the number: ");
      int num = sc.nextInt();
      SquareClass t1 = new SquareClass(num);
      CubeClass t2 = new CubeClass(num);
```

```
        t1.start();
        t2.start();
    }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG29SquareCubeDemo.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG29SquareCubeDemo
Enter the number: 3

Square: 9

Cube: 27
```

Program No: 30	**Date:** 20-12-2025

Program Title: Implement Printer Queue Simulation without synchronization but using multiple threads.

```
/*
Printer Queue Simulation
@ Ashbi Saju
Roll no: 16
Date: 22-11-2025
*/

import java.util.Scanner;

class PrinterClass {
   public void printMessage(String message) {
      System.out.println("Printing message: [");
      System.out.println(message);
      System.out.println("]");
   }
}

class PrinterThread extends Thread {
   PrinterClass printer;
   String message;
   public PrinterThread(PrinterClass printer, String message) {
      this.printer = printer;
      this.message = message;
   }
   public void run() {
      printer.printMessage(message);
   }
}

class PG30PrinterDemo {
   public static void main(String args[]) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter the message for Thread 1: ");
      String message1 = sc.nextLine();
      System.out.print("Enter the message for Thread 2: ");
      String message2 = sc.nextLine();
      System.out.print("Enter the message for Thread 3: ");
```

```
        String message3 = sc.nextLine();

        PrinterClass printer = new PrinterClass();
        PrinterThread t1 = new PrinterThread(printer, message1);
        PrinterThread t2 = new PrinterThread(printer, message2);
        PrinterThread t3 = new PrinterThread(printer, message3);
        t1.start();
        t2.start();
        t3.start();
    }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG30PrinterDemo.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG30PrinterDemo
Enter the message for Thread 1: Hello
Enter the message for Thread 2: Welcome
Enter the message for Thread 3: Home
Printing message: [
Hello
]
Printing message: [
Welcome
]
Printing message: [
Home
]
```

Program No: 31	**Date:** 22-12-2025

Program Title: Implement Printer Queue simulation using multithreading and synchronization.

```
/*
Synchronized Printer Queue
@ Ashbi Saju
Roll no: 16
Date: 22-11-2025
*/

import java.util.Scanner;

class PrinterSyncClass {
   public synchronized void printMessage(String message) {

      try {
         System.out.println("Printing message: [");
         Thread.sleep(1000);
         System.out.println(message);
         System.out.println("]");
      } catch (InterruptedException e) {
         System.out.println(e);
      }

   }
}

class PG31PrinterSyncThread extends Thread {
   PrinterSyncClass printer;
   String message;
   public PG31PrinterSyncThread(PrinterSyncClass printer, String message) {
      this.printer = printer;
      this.message = message;
   }
   public void run() {
      printer.printMessage(message);
   }
}
```

```
class PrinterSyncDemo {
   public static void main(String args[]) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter the message for Thread 1: ");
      String message1 = sc.nextLine();
      System.out.print("Enter the message for Thread 2: ");
      String message2 = sc.nextLine();
      System.out.print("Enter the message for Thread 3: ");
      String message3 = sc.nextLine();

      PrinterClass printer = new PrinterClass();
      PrinterThread t1 = new PrinterThread(printer, message1);
      PrinterThread t2 = new PrinterThread(printer, message2);
      PrinterThread t3 = new PrinterThread(printer, message3);
      t1.start();
      t2.start();
      t3.start();
   }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG31PrinterSyncDemo.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG31PrinterSyncDemo
Enter the message for Thread 1: Hello
Enter the message for Thread 2: Welcome
Enter the message for Thread 3: Home
Printing message: [
Hello
]
Printing message: [
Welcome
]
Printing message: [
Home
]
```

Program No: 32	**Date:** 22-12-2025

Program Title: Write a Java program to demonstrate inter-thread communication using wait(), notify(), and notifyAll() by implementing

a. Producer consumer problem

b. Readers writer's problem

```
/*
Inter Thread Communication
@ Ashbi Saju
Roll no: 16
Date: 05-12-2026
*/
```

a) Producer consumer problem

```
class SharedMemory {
   int data;
   boolean available = false;

   public synchronized int consume() throws InterruptedException {
      while (!available) {
         wait();
      }
      available = false;
      notify();
      return data;
   }

   public synchronized void produce(int value) throws InterruptedException {
      while (available) {
         wait();
      }
      data = value;
      available = true;
      notify();
   }
}
class Producer extends Thread {
   private SharedMemory s;

   public Producer(SharedMemory s) {
      this.s = s;
   }
```

```
    public void run() {
        for (int i = 1; i <= 10; i++) {
            System.out.println("\nProducing: " + i);
            try {
                s.produce(i);
                Thread.sleep(1000);
            } catch (InterruptedException e) {
            }

        }
    }
}
class Consumer extends Thread {
    private SharedMemory s;

    public Consumer(SharedMemory s) {
        this.s = s;
    }

    public void run() {
        for (int i = 1; i <= 10; i++) {
            int value;
            try {
                value = s.consume();
                System.out.println("Consuming: " + value);
                Thread.sleep(150);
            } catch (InterruptedException e) {
            }
        }
    }
}
public class PG32ProducerConsumerMain {
    public static void main(String[] args) {
        SharedMemory s = new SharedMemory();
        Producer producer = new Producer(s);
        Consumer consumer = new Consumer(s);

        producer.start();
        consumer.start();
    }
}
```

b) Readers writer's problem

```
//Implement readers writers problem using Threads in java

class SharedResource {
   int readers = 0;
   boolean writing = false;

   public synchronized void startReading() throws InterruptedException {
      while (writing) {
         wait();
      }
      readers++;
   }

   public synchronized void stopReading() {
      readers--;
      if (readers == 0) {
         notify();
      }
   }

   public synchronized void startWriting() throws InterruptedException {
      while (readers > 0 || writing) {
         wait();
      }
      writing = true;
   }

   public synchronized void stopWriting() {
      writing = false;
      notify();
   }
}
class Reader extends Thread {
   private SharedResource resource;

   public Reader(SharedResource resource) {
      this.resource = resource;
   }
```

```
    public void run() {
        try {
            resource.startReading();
            System.out.println("Reader " + Thread.currentThread().getId() + " is reading.");
            Thread.sleep(1000);
            resource.stopReading();
            System.out.println("Reader " + Thread.currentThread().getId() + " has finished reading.");
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
class Writer extends Thread {
    private SharedResource resource;

    public Writer(SharedResource resource) {
        this.resource = resource;
    }

    public void run() {
        try {
            resource.startWriting();
            System.out.println("Writer " + Thread.currentThread().getId() + " is writing.");
            Thread.sleep(1000);
            resource.stopWriting();
            System.out.println("Writer " + Thread.currentThread().getId() + " has finished writing.");
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
public class PG32ReadersWritersProblem {
    public static void main(String[] args) {
        SharedResource resource = new SharedResource();

        for (int i = 0; i < 5; i++) {
            new Reader(resource).start();
            new Writer(resource).start();
        }
    }
}
```

Output

a) Producer consumer problem

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG32ProducerConsumerMain

Producing: 1
Consuming: 1

Producing: 2
Consuming: 2

Producing: 3
Consuming: 3

Producing: 4
Consuming: 4

Producing: 5
Consuming: 5

Producing: 6
Consuming: 6

Producing: 7
Consuming: 7

Producing: 8
Consuming: 8

Producing: 9
Consuming: 9

Producing: 10
Consuming: 10
```

b) Readers writer's problem

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG32ReadersWritersProblem
Reader 37 is reading.
Reader 41 is reading.
Reader 35 is reading.
Reader 43 is reading.
Reader 39 is reading.
Reader 41 has finished reading.
Reader 43 has finished reading.
Reader 39 has finished reading.
Reader 35 has finished reading.
Reader 37 has finished reading.
Writer 36 is writing.
Writer 38 is writing.
Writer 36 has finished writing.
Writer 38 has finished writing.
Writer 40 is writing.
Writer 40 has finished writing.
Writer 42 is writing.
Writer 42 has finished writing.
Writer 44 is writing.
Writer 44 has finished writing.
```

Program No: 33	**Date:** 22-12-2025

Program Title: Write a Java program to simulate bank deposit and withdrawal using threads for a single account without synchronization.

```
/*
Unsynchronized Bank Account
@ Ashbi Saju
Roll no: 16
Date: 05-12-2026
*/

import java.util.Scanner;

class Bank {
   int balance;
   Bank(int balance) {
      this.balance = balance;
   }

   void deposit(int amount) {
   int temp = balance;
   try { Thread.sleep(100); } catch (Exception e) {}
   balance = temp + amount;
      System.out.println("Deposited: " + amount +
                  " | Balance: " + balance);
   }

   void withdraw(int amount) {
   int temp = balance;
   try { Thread.sleep(100); } catch (Exception e) {}
   balance = temp - amount;
      System.out.println("Withdrawn: " + amount +
                  " | Balance: " + balance);
   }
}

class Deposit extends Thread {
   Bank bank;
   int amount;

   Deposit(Bank bank, int amount) {
```

```
        this.bank = bank;
        this.amount = amount;
    }

    public void run() {
        bank.deposit(amount);
    }
}

class Withdraw extends Thread {
    Bank bank;
    int amount;

    Withdraw(Bank bank, int amount) {
        this.bank = bank;
        this.amount = amount;
    }

    public void run() {
        bank.withdraw(amount);
    }
}

public class PG33BankDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int initialBalance = 10000;
        System.out.println("Initial Balance: " + initialBalance);
        Bank bank = new Bank(initialBalance);

        System.out.print("Enter the amount to deposit: ");
        int depositAmount = sc.nextInt();
        System.out.print("Enter the amount to withdraw: ");
        int withdrawAmount = sc.nextInt();
        Deposit d = new Deposit(bank, depositAmount);
        Withdraw w = new Withdraw(bank, withdrawAmount);

        d.start();
        w.start();
    }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG33BankDemo.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG33BankDemo
Initial Balance: 10000
Enter the amount to deposit: 10000
Enter the amount to withdraw: 5000
Withdrawn: 5000 | Balance: 20000
Deposited: 10000 | Balance: 20000
```

Program No: 34	**Date:** 24-12-2025

Program Title: Implement thread priority.

```
/*
Thread Priority
@ Ashbi Saju
Roll no: 16
Date: 05-12-2026
*/

// Thread class
class PriorityThread extends Thread {

   PriorityThread(String name) {
      super(name);
   }

   public void run() {
      for (int i = 1; i <= 5; i++) {
         System.out.println(getName() +
               " | Priority: " + getPriority() +
               " | Count: " + i);
      }
   }
}

// Main class
class PG34ThreadPriorityDemo {
   public static void main(String[] args) {

      PriorityThread t1 = new PriorityThread("Low Priority Thread");
      PriorityThread t2 = new PriorityThread("Normal Priority Thread");
      PriorityThread t3 = new PriorityThread("High Priority Thread");

      // Setting priorities
      t1.setPriority(Thread.MIN_PRIORITY);   // 1
      t2.setPriority(Thread.NORM_PRIORITY);  // 5
      t3.setPriority(Thread.MAX_PRIORITY);   // 10
```

```
        // Start threads
        t1.start();
        t2.start();
        t3.start();
    }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG34ThreadPriorityDemo.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG34ThreadPriorityDemo
High Priority Thread | Priority: 10 | Count: 1
Normal Priority Thread | Priority: 5 | Count: 1
Low Priority Thread | Priority: 1 | Count: 1
High Priority Thread | Priority: 10 | Count: 2
Normal Priority Thread | Priority: 5 | Count: 2
Low Priority Thread | Priority: 1 | Count: 2
High Priority Thread | Priority: 10 | Count: 3
Normal Priority Thread | Priority: 5 | Count: 3
Low Priority Thread | Priority: 1 | Count: 3
Low Priority Thread | Priority: 1 | Count: 4
High Priority Thread | Priority: 10 | Count: 4
Normal Priority Thread | Priority: 5 | Count: 4
Low Priority Thread | Priority: 1 | Count: 5
High Priority Thread | Priority: 10 | Count: 5
Normal Priority Thread | Priority: 5 | Count: 5
```

Program No: 35	**Date:** 24-12-2025

Program Title: Develop a Java program to simulate ATM operations with multiple users accessing the same account.
[Develop a program to prevent race condition in banking transactions using synchronized methods.]

```
/*
Synchronized ATM System
@ Ashbi Saju
Roll no: 16
Date: 05-12-2026
*/

import java.util.Scanner;

// Bank account class
class BankAccount {
   private int balance;

   BankAccount(int balance) {
      this.balance = balance;
   }

   // synchronized withdraw method
   synchronized void withdraw(String user, int amount) {
      System.out.println(user + " trying to withdraw " + amount);

      if (balance >= amount) {
         try {
            Thread.sleep(500); // simulate ATM delay
         } catch (InterruptedException e) {
            System.out.println(e);
         }
         balance -= amount;
         System.out.println(user + " withdrawal successful. Balance = " + balance);
      } else {
         System.out.println(user + " withdrawal failed. Insufficient balance.");
      }
      System.out.println();
   }
```

```
    // synchronized deposit method
    synchronized void deposit(String user, int amount) {

        System.out.println(user + " depositing " + amount);

        try {
            Thread.sleep(500);
        } catch (InterruptedException e) {
            System.out.println(e);
        }
        balance += amount;
        System.out.println(user + " deposit successful. Balance = " + balance);
        System.out.println();
    }
}

// ATM user thread
class ATMUser extends Thread {
    BankAccount account;
    String user;
    int amount;
    int choice;

    ATMUser(BankAccount account, String user, int choice, int amount) {
        this.account = account;
        this.user = user;
        this.choice = choice;
        this.amount = amount;
    }

    public void run() {
        if (choice == 1) {
            account.deposit(user, amount);
        } else if (choice == 2) {
            account.withdraw(user, amount);
        }
    }
}
```

```
// Main class
class PG35ATMSimulationUserInput {
   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      System.out.print("Enter initial account balance: ");
      int balance = sc.nextInt();

      BankAccount account = new BankAccount(balance);

      System.out.print("Enter number of users: ");
      int n = sc.nextInt();

      ATMUser[] users = new ATMUser[n];

      for (int i = 0; i < n; i++) {
         System.out.println("\nUser " + (i + 1));
         System.out.println("1. Deposit");
         System.out.println("2. Withdraw");
         System.out.print("Enter choice: ");
         int choice = sc.nextInt();

         System.out.print("Enter amount: ");
         int amount = sc.nextInt();

         users[i] = new ATMUser(account, "User-" + (i + 1), choice, amount);
      }

      // Start all user threads
      for (int i = 0; i < n; i++) {
         users[i].start();
      }

      sc.close();
   }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG35ATMSimulationUserInput.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG35ATMSimulationUserInput
Enter initial account balance: 10000
Enter number of users: 2

User 1
1. Deposit
2. Withdraw
Enter choice: 1
Enter amount: 10000

User 2
1. Deposit
2. Withdraw
Enter choice: 2
Enter amount: 5000
User-1 depositing 10000
User-1 deposit successful. Balance = 20000

User-2 trying to withdraw 5000
User-2 withdrawal successful. Balance = 15000
```

Program No: 36	**Date:** 05-01-2026

Program Title: Write a program to implement a Generic method.

```
/*
Generic Method
@ Ashbi Saju
Roll no: 16
Date: 05-01-2026
*/

public class PG36GenericMethods {

   // Generic method to print any type of array
   public static <T> void printArray(T[] array) {
      System.out.print("Array elements: ");
      for (T element : array) {
         System.out.print(element + " ");
      }
      System.out.println();
   }


   // Generic method to find maximum of three values
   public static <T extends Comparable<T>> T findMax(T a, T b, T c) {
      T max = a;
      if (b.compareTo(max) > 0) max = b;
      if (c.compareTo(max) > 0) max = c;
      return max;
   }


   // Generic method to swap two elements of an array
   public static <T> void swap(T[] array, int i, int j) {
      T temp = array[i];
      array[i] = array[j];
      array[j] = temp;
   }

   // Generic method to check if an element exists in an array
   public static <T> boolean contains(T[] array, T key) {
```

```
        for (T element : array) {
            if (element.equals(key)) return true;
        }
        return false;
    }


    public static void main(String[] args) {

        // Integer array
        Integer[] intArray = {10, 20, 30, 40, 50};
        printArray(intArray);

        // String array
        String[] strArray = {"Java", "Python", "C++", "HTML"};
        printArray(strArray);

        // Double array
        Double[] dblArray = {1.1, 2.2, 3.3, 4.4};
        printArray(dblArray);

        // Find Maximum
        System.out.println("\nMax of (10, 55, 30)     : " + findMax(10, 55, 30));
        System.out.println("Max of (Apple, Mango, Fig): " + findMax("Apple", "Mango", "Fig"));
        System.out.println("Max of (1.5, 9.9, 3.3)  : " + findMax(1.5, 9.9, 3.3));

        // Swap
        System.out.println("\nBefore swap: ");
        printArray(intArray);
        swap(intArray, 0, 4);
        System.out.println("After swap (index 0 & 4): ");
        printArray(intArray);

        // Contains
        System.out.println("\n\"Python\" in strArray : " + contains(strArray, "Python"));
        System.out.println("\"Ruby\" in strArray   : " + contains(strArray, "Ruby"));
    }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG36GenericMethods.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG36GenericMethods
Array elements: 10 20 30 40 50
Array elements: Java Python C++ HTML
Array elements: 1.1 2.2 3.3 4.4

Max of (10, 55, 30)     : 55
Max of (Apple, Mango, Fig): Mango
Max of (1.5, 9.9, 3.3)  : 9.9

Before swap:
Array elements: 10 20 30 40 50
After swap (index 0 & 4):
Array elements: 50 20 30 40 10

"Python" in strArray : true
"Ruby" in strArray   : false
```

Program No: 37	**Date:** 05-01-2026

Program Title: Write a program to implement a Generic class.

```
/*
Generic Class
@ Ashbi Saju
Roll no: 16
Date: 05-01-2026
*/

public class PG37GenericClassDemo {
   // Generic class with type parameter <T>
   static class Box<T> {
      private T value;

      public Box(T value) {
         this.value = value;
      }

      public T getValue() {
         return value;
      }

      public void setValue(T value) {
         this.value = value;
      }

      public void displayType() {
         System.out.println("Type : " + value.getClass().getName());
         System.out.println("Value: " + value);
      }
   }

   // Generic class with TWO type parameters <K, V> — like a Key-Value Pair
   static class Pair<K, V> {
      private K key;
      private V value;

      public Pair(K key, V value) {
         this.key   = key;
```

```
        this.value = value;
    }

    public K getKey()   { return key;   }
    public V getValue() { return value; }

    public void display() {
        System.out.println("Key: " + key + "   |  Value: " + value);
    }
}

// Generic Stack class using an array
static class Stack<T> {
    private Object[] data;
    private int top;
    private int capacity;

    public Stack(int capacity) {
        this.capacity = capacity;
        data = new Object[capacity];
        top  = -1;
    }

    public void push(T item) {
        if (top == capacity - 1) {
            System.out.println("Stack Overflow!");
            return;
        }
        data[++top] = item;
        System.out.println(item + " pushed to stack");
    }

    @SuppressWarnings("unchecked")
    public T pop() {
        if (top == -1) {
            System.out.println("Stack Underflow!");
            return null;
        }
        return (T) data[top--];
    }
    @SuppressWarnings("unchecked")
```

```
    public T peek() {
        if (top == -1) return null;
        return (T) data[top];
    }

    public boolean isEmpty() { return top == -1; }
    public void display() {
        System.out.print("Stack (top→bottom): ");
        for (int i = top; i >= 0; i--)
            System.out.print(data[i] + " ");
        System.out.println();
    }
}

public static void main(String[] args) {
    // ---- Box<T> demo ----------------------------------------------
    System.out.println("===== Box<T> =====");
    Box<Integer> intBox = new Box<>(42);
    intBox.displayType();
    Box<String> strBox = new Box<>("Hello Java");
    strBox.displayType();
    Box<Double> dblBox = new Box<>(3.14);
    dblBox.displayType();

    // ---- Pair<K,V> demo -------------------------------------------
    System.out.println("\n===== Pair<K,V> =====");
    Pair<Integer, String> student = new Pair<>(101, "Alice");
    student.display();
    Pair<String, Double> score = new Pair<>("Math", 98.5);
    score.display();
    Pair<String, String> country = new Pair<>("Capital", "New Delhi");
    country.display();

    // ---- Stack<T> demo --------------------------------------------
    System.out.println("\n===== Stack<T> (Integer) =====");
    Stack<Integer> intStack = new Stack<>(5);
    intStack.push(10);
    intStack.push(20);
    intStack.push(30);
    intStack.display();
    System.out.println("Peek : " + intStack.peek());
```

```
        System.out.println("Pop  : " + intStack.pop());
        intStack.display();
        System.out.println("\n===== Stack<T> (String) =====");
        Stack<String> strStack = new Stack<>(5);
        strStack.push("Apple");
        strStack.push("Banana");
        strStack.push("Cherry");
        strStack.display();
        System.out.println("Pop  : " + strStack.pop());
        strStack.display();
    }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG37GenericClassDemo.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG37GenericClassDemo
===== Box<T> =====
Type : java.lang.Integer
Value: 42
Type : java.lang.String
Value: Hello Java
Type : java.lang.Double
Value: 3.14

===== Pair<K,V> =====
Key: 101  |  Value: Alice
Key: Math  |  Value: 98.5
Key: Capital  |  Value: New Delhi

===== Stack<T> (Integer) =====
10 pushed to stack
20 pushed to stack
30 pushed to stack
Stack (top?bottom): 30 20 10
Peek : 30
Pop  : 30
Stack (top?bottom): 20 10

===== Stack<T> (String) =====
Apple pushed to stack
Banana pushed to stack
Cherry pushed to stack
Stack (top?bottom): Cherry Banana Apple
Pop  : Cherry
Stack (top?bottom): Banana Apple
```

Program No: 38	**Date:** 05-01-2026

Program Title: Write a program to create a menu driven program for performing the following operations.

a. Length of a given string
b. Compare for equality
c. Extract a substring from a string.
d. Convert to uppercase and lowercase
e. Reverse a string

```
/*
String Operations Menu
@ Ashbi Saju
Roll no: 16
Date: 05-01-2026
*/

import java.util.Scanner;

public class PG38StringOperations {

   // a. Length of a string
   static void findLength(String str) {
      System.out.println("String : \"" + str + "\"");
      System.out.println("Length : " + str.length());
   }

   // b. Compare for equality
   static void compareStrings(String str1, String str2) {
      System.out.println("String 1 : \"" + str1 + "\"");
      System.out.println("String 2 : \"" + str2 + "\"");

      if (str1.equals(str2))
         System.out.println("Result   : Strings are EQUAL (case-sensitive)");
      else
         System.out.println("Result   : Strings are NOT EQUAL (case-sensitive)");

      if (str1.equalsIgnoreCase(str2))
         System.out.println("Result   : Strings are EQUAL (case-insensitive)");
      else
         System.out.println("Result   : Strings are NOT EQUAL (case-insensitive)");

      int cmp = str1.compareTo(str2);
```

```
        if      (cmp < 0) System.out.println("Order    : \"" + str1 + "\" comes BEFORE \"" + str2 + "\"
alphabetically");
        else if (cmp > 0) System.out.println("Order    : \"" + str1 + "\" comes AFTER  \"" + str2 + "\"
alphabetically");
        else              System.out.println("Order    : Both strings are identical");
    }

    // c. Extract a substring
    static void extractSubstring(String str, Scanner sc) {
        System.out.println("String : \"" + str + "\"");
        System.out.println("Length : " + str.length());
        System.out.print("Enter start index      : ");
        int start = sc.nextInt();
        System.out.print("Enter end index        : ");
        int end = sc.nextInt();
        sc.nextLine();

        if (start < 0 || end > str.length() || start > end) {
            System.out.println("Invalid indices! Please check the range.");
        } else {
            System.out.println("Substring [" + start + ", " + end + ") : \"" + str.substring(start, end) +
"\"");
            System.out.println("Substring from " + start + " to end   : \"" + str.substring(start) + "\"");
        }
    }

    // d. Convert to uppercase and lowercase
    static void convertCase(String str) {
        System.out.println("Original   : \"" + str + "\"");
        System.out.println("Uppercase  : \"" + str.toUpperCase() + "\"");
        System.out.println("Lowercase  : \"" + str.toLowerCase() + "\"");
    }

    // e. Reverse a string
    static void reverseString(String str) {
        String reversed = new StringBuilder(str).reverse().toString();
        System.out.println("Original : \"" + str + "\"");
        System.out.println("Reversed : \"" + reversed + "\"");

        if (str.equalsIgnoreCase(reversed))
            System.out.println("Note     : This string is a PALINDROME!");
```

```
    else
        System.out.println("Note     : This string is NOT a palindrome.");
}

// — main ————————————————————————————————————————————————
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int choice;

    do {
        System.out.println("\n--- STRING OPERATIONS MENU ---");
        System.out.println("1. Length of a String");
        System.out.println("2. Compare Two Strings");
        System.out.println("3. Extract a Substring");
        System.out.println("4. Convert to Upper/Lowercase");
        System.out.println("5. Reverse a String");
        System.out.println("6. Exit");
        System.out.println("-----------------------------");
        System.out.print("Enter your choice (1-6): ");
        choice = sc.nextInt();
        sc.nextLine();

        System.out.println("-------------------------------------");

        switch (choice) {

            case 1:
                System.out.print("Enter a string: ");
                String s1 = sc.nextLine();
                findLength(s1);
                break;

            case 2:
                System.out.print("Enter String 1: ");
                String cmp1 = sc.nextLine();
                System.out.print("Enter String 2: ");
                String cmp2 = sc.nextLine();
                compareStrings(cmp1, cmp2);
                break;

            case 3:
```

```
            System.out.print("Enter a string: ");
            String s3 = sc.nextLine();
            extractSubstring(s3, sc);
            break;

        case 4:
            System.out.print("Enter a string: ");
            String s4 = sc.nextLine();
            convertCase(s4);
            break;

        case 5:
            System.out.print("Enter a string: ");
            String s5 = sc.nextLine();
            reverseString(s5);
            break;

        case 6:
            System.out.println("Exiting... Goodbye!");
            break;

        default:
            System.out.println("Invalid choice! Please enter between 1 and 6.");
      }

      System.out.println("-----------------------------------");

    } while (choice != 6);

    sc.close();
  }
}
```

Output

```
--- STRING OPERATIONS MENU ---
1. Length of a String
2. Compare Two Strings
3. Extract a Substring
4. Convert to Upper/Lowercase
5. Reverse a String
6. Exit
----------------------------
Enter your choice (1-6): 1
----------------------------------------
Enter a string: Hello World
String : "Hello World"
Length : 11
----------------------------------------

--- STRING OPERATIONS MENU ---
1. Length of a String
2. Compare Two Strings
3. Extract a Substring
4. Convert to Upper/Lowercase
5. Reverse a String
6. Exit
----------------------------
Enter your choice (1-6): 2
----------------------------------------
Enter String 1: Java
Enter String 2: java
String 1 : "Java"
String 2 : "java"
Result   : Strings are NOT EQUAL (case-sensitive)
Result   : Strings are EQUAL (case-insensitive)
Order    : "Java" comes BEFORE "java" alphabetically
----------------------------------------

--- STRING OPERATIONS MENU ---
1. Length of a String
2. Compare Two Strings
3. Extract a Substring
4. Convert to Upper/Lowercase
5. Reverse a String
6. Exit
----------------------------
Enter your choice (1-6): 3
----------------------------------------
```

```
----------------------------------------
Enter a string: Programming
String : "Programming"
Length : 11
Enter start index       : 0
Enter end index         : 7
Substring [0, 7) : "Program"
Substring from 0 to end   : "Programming"
----------------------------------------

--- STRING OPERATIONS MENU ---
1. Length of a String
2. Compare Two Strings
3. Extract a Substring
4. Convert to Upper/Lowercase
5. Reverse a String
6. Exit
-----------------------------
Enter your choice (1-6): 4
----------------------------------------
Enter a string: Hello Java
Original   : "Hello Java"
Uppercase  : "HELLO JAVA"
Lowercase  : "hello java"
----------------------------------------

--- STRING OPERATIONS MENU ---
1. Length of a String
2. Compare Two Strings
3. Extract a Substring
4. Convert to Upper/Lowercase
5. Reverse a String
6. Exit
-----------------------------
Enter your choice (1-6): 5
----------------------------------------
Enter a string: malayalam
Original : "malayalam"
Reversed : "malayalam"
Note     : This string is a PALINDROME!
----------------------------------------
```

Program No: 39	**Date:** 12-01-2026

Program Title: Program to implement StringBuffer class.

```
/*
StringBuffer Demo
@ Ashbi Saju
Roll no: 16
Date: 12-01-2026
*/

public class PG39StringBufferDemo {

   // a. Append to StringBuffer
   static void appendDemo() {
      StringBuffer sb = new StringBuffer("Hello");
      System.out.println("Original       : " + sb);
      sb.append(" World");
      System.out.println("After append() : " + sb);
      sb.append(123);
      System.out.println("After append(123): " + sb);
   }

   // b. Insert into StringBuffer
   static void insertDemo() {
      StringBuffer sb = new StringBuffer("Hello World");
      System.out.println("Original      : " + sb);
      sb.insert(5, " Beautiful");
      System.out.println("After insert() : " + sb);
   }

   // c. Delete from StringBuffer
   static void deleteDemo() {
      StringBuffer sb = new StringBuffer("Hello Beautiful World");
      System.out.println("Original       : " + sb);
      sb.delete(5, 15);
      System.out.println("After delete()  : " + sb);
      sb.deleteCharAt(0);
      System.out.println("After deleteCharAt(0): " + sb);
   }
```

```
// d. Reverse a StringBuffer
static void reverseDemo() {
   StringBuffer sb = new StringBuffer("Java Programming");
   System.out.println("Original       : " + sb);
   sb.reverse();
   System.out.println("After reverse() : " + sb);

   StringBuffer sb2 = new StringBuffer("madam");
   System.out.println("\nOriginal       : " + sb2);
   StringBuffer sb2copy = new StringBuffer(sb2).reverse();
   if (sb2.toString().equals(sb2copy.toString()))
      System.out.println("\"" + sb2 + "\" is a PALINDROME!");
   else
      System.out.println("\"" + sb2 + "\" is NOT a palindrome.");
}

// e. Replace in StringBuffer
static void replaceDemo() {
   StringBuffer sb = new StringBuffer("Hello World");
   System.out.println("Original       : " + sb);
   sb.replace(6, 11, "Java");
   System.out.println("After replace() : " + sb);
}

// f. Capacity and Length
static void capacityLengthDemo() {
   StringBuffer sb = new StringBuffer();
   System.out.println("Default Capacity : " + sb.capacity());
   System.out.println("Default Length   : " + sb.length());

   sb.append("Hello Java");
   System.out.println("\nAfter appending \"Hello Java\":");
   System.out.println("Length          : " + sb.length());
   System.out.println("Capacity        : " + sb.capacity());

   StringBuffer sb2 = new StringBuffer(50);
   System.out.println("\nCustom Capacity  : " + sb2.capacity());
}

// g. charAt and setCharAt
static void charAtDemo() {
```

```
    StringBuffer sb = new StringBuffer("Hello Java");
    System.out.println("Original          : " + sb);
    System.out.println("charAt(0)         : " + sb.charAt(0));
    System.out.println("charAt(6)         : " + sb.charAt(6));
    sb.setCharAt(0, 'h');
    System.out.println("After setCharAt(0,'h'): " + sb);
}

// h. indexOf and lastIndexOf
static void indexOfDemo() {
    StringBuffer sb = new StringBuffer("Java is great. Java is fun.");
    System.out.println("String            : " + sb);
    System.out.println("indexOf(\"Java\")    : " + sb.indexOf("Java"));
    System.out.println("lastIndexOf(\"Java\") : " + sb.lastIndexOf("Java"));
    System.out.println("indexOf(\"is\")      : " + sb.indexOf("is"));
}

// i. substring from StringBuffer
static void substringDemo() {
    StringBuffer sb = new StringBuffer("Hello Java World");
    System.out.println("Original          : " + sb);
    System.out.println("substring(6)      : " + sb.substring(6));
    System.out.println("substring(6, 10)  : " + sb.substring(6, 10));
}

// j. Convert StringBuffer to String
static void toStringDemo() {
    StringBuffer sb = new StringBuffer("StringBuffer to String");
    System.out.println("StringBuffer : " + sb);
    String str = sb.toString();
    System.out.println("String     : " + str);
    System.out.println("Type       : " + str.getClass().getName());
}

// ── main ──────────────────────────────────────────────────────────
public static void main(String[] args) {

    System.out.println("StringBuffer CLASS DEMO");
    System.out.println("\na. append()");
    appendDemo();
```

```
        System.out.println("\nb. insert()");
        insertDemo();
        System.out.println("\nc. delete()");
        deleteDemo();
        System.out.println("\nd. reverse()");
        reverseDemo();
        System.out.println("\ne. replace()");
        replaceDemo();
        System.out.println("\nf. capacity() & length()");
        capacityLengthDemo();
        System.out.println("\ng. charAt() & setCharAt()");
        charAtDemo();
        System.out.println("\nh. indexOf() & lastIndexOf()");
        indexOfDemo();
        System.out.println("\ni. substring()");
        substringDemo();
        System.out.println("\nj. toString()");
        toStringDemo();
    }
}
```

Output

```
StringBuffer CLASS DEMO

a. append()
Original        : Hello
After append()  : Hello World
After append(123): Hello World123

b. insert()
Original        : Hello World
After insert()  : Hello Beautiful World

c. delete()
Original         : Hello Beautiful World
After delete()   : Hello World
After deleteCharAt(0): ello World
```

```
d. reverse()
Original        : Java Programming
After reverse() : gnimmargorP avaJ

Original        : madam
"madam" is a PALINDROME!

e. replace()
Original        : Hello World
After replace() : Hello Java

f. capacity() & length()
Default Capacity : 16
Default Length   : 0

After appending "Hello Java":
Length           : 10
Capacity         : 16

Custom Capacity  : 50

g. charAt() & setCharAt()
Original            : Hello Java
charAt(0)           : H
charAt(6)           : J
After setCharAt(0,'h'): hello Java

h. indexOf() & lastIndexOf()
String              : Java is great. Java is fun.
indexOf("Java")     : 0
lastIndexOf("Java") : 15
indexOf("is")       : 5

i. substring()
Original             : Hello Java World
substring(6)         : Java World
substring(6, 10)     : Java

j. toString()
StringBuffer : StringBuffer to String
String       : StringBuffer to String
Type         : java.lang.String
```

Program No: 40	**Date:** 12-01-2026

Program Title: Program to implement server client communication.

```
/*
Client Server Communication
@ Ashbi Saju
Roll no: 16
Date: 12-01-2026
*/
```

Server.java

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class PG40Server {
   public static void main(String[] args) {
      int port = 5000;

      try {
         ServerSocket serverSocket = new ServerSocket(port);
         System.out.println("Server started. Listening on port " + port + "...");
         System.out.println("Waiting for client to connect...");

         Socket socket = serverSocket.accept();
         System.out.println("Client connected: " + socket.getInetAddress());

         BufferedReader input  = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
         PrintWriter    output = new PrintWriter(socket.getOutputStream(), true);
         Scanner        sc     = new Scanner(System.in);

         String clientMessage, serverMessage;

         while (true) {
            clientMessage = input.readLine();

            if (clientMessage == null || clientMessage.equalsIgnoreCase("bye")) {
               System.out.println("Client disconnected.");
```

```
            output.println("bye");
            break;
          }

          System.out.println("Client : " + clientMessage);

          System.out.print("Server : ");
          serverMessage = sc.nextLine();
          output.println(serverMessage);

          if (serverMessage.equalsIgnoreCase("bye")) {
            System.out.println("Server disconnected.");
            break;
          }
        }

        input.close();
        output.close();
        socket.close();
        serverSocket.close();
        sc.close();
        System.out.println("Connection closed.");

      } catch (IOException e) {
        System.out.println("Server error: " + e.getMessage());
      }
    }
}
```

Client.java

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class PG40Client {
   public static void main(String[] args) {
      String host = "localhost";
      int    port = 5000;
```

```
    try {
        Socket socket = new Socket(host, port);
        System.out.println("Connected to Server at " + host + ":" + port);
        System.out.println("Type your message (type 'bye' to exit):");

        BufferedReader input  = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
        PrintWriter    output = new PrintWriter(socket.getOutputStream(), true);
        Scanner        sc     = new Scanner(System.in);

        String clientMessage, serverMessage;

        while (true) {
            System.out.print("Client : ");
            clientMessage = sc.nextLine();
            output.println(clientMessage);

            if (clientMessage.equalsIgnoreCase("bye")) {
                System.out.println("Client disconnected.");
                break;
            }

            serverMessage = input.readLine();

            if (serverMessage == null || serverMessage.equalsIgnoreCase("bye")) {
                System.out.println("Server disconnected.");
                break;
            }

            System.out.println("Server : " + serverMessage);
        }

        input.close();
        output.close();
        socket.close();
        sc.close();
        System.out.println("Connection closed.");

    } catch (IOException e) {
        System.out.println("Client error: " + e.getMessage());
    }
```

```
    }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG40Server.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG40Server
Server started. Listening on port 5000...
Waiting for client to connect...
Client connected: /127.0.0.1
Client : Hii...
Server : Hello...
Client : Nice to meet you.
Server : Thanks.
Client disconnected.
Connection closed.
```

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG40Client.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG40Client
Connected to Server at localhost:5000
Type your message (type 'bye' to exit):
Client : Hii...
Server : Hello...
Client : Nice to meet you.
Server : Thanks.
Client : bye
Client disconnected.
Connection closed.
```

Program No: 41	**Date:** 12-01-2026

Program Title: Program to implement one-one chatting.

```
/*
One-to-One Chat
@ Ashbi Saju
Roll no: 16
Date: 12-01-2026
*/
```

Server.java

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class PG41ChatServer {
   public static void main(String[] args) {
      int port = 5000;

      try {
         ServerSocket serverSocket = new ServerSocket(port);
         System.out.println("Server started on port " + port);
         System.out.println("Waiting for client...");

         Socket socket = serverSocket.accept();
         System.out.println("Client connected!");
         System.out.println("You can start chatting. Type 'bye' to exit.\n");

         BufferedReader input  = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
         PrintWriter    output = new PrintWriter(socket.getOutputStream(), true);
         Scanner        sc     = new Scanner(System.in);

         // Thread to continuously receive messages from client
         Thread receiveThread = new Thread(() -> {
            try {
               String message;
               while ((message = input.readLine()) != null) {
                  if (message.equalsIgnoreCase("bye")) {
```

```
                    System.out.println("Client has left the chat.");
                    System.exit(0);
                }
                System.out.println("Client : " + message);
            }
        } catch (IOException e) {
            System.out.println("Connection lost.");
        }
    });

    // Thread to continuously send messages to client
    Thread sendThread = new Thread(() -> {
        String message;
        while (true) {
            System.out.print("Server : ");
            message = sc.nextLine();
            output.println(message);
            if (message.equalsIgnoreCase("bye")) {
                System.out.println("You left the chat.");
                System.exit(0);
            }
        }
    });

    receiveThread.start();
    sendThread.start();

    receiveThread.join();
    sendThread.join();

    socket.close();
    serverSocket.close();
    sc.close();

  } catch (IOException | InterruptedException e) {
    System.out.println("Server error: " + e.getMessage());
  }
 }
}
```

Client.java

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class PG41ChatClient {
   public static void main(String[] args) {
      String host = "localhost";
      int    port = 5000;

      try {
         Socket socket = new Socket(host, port);
         System.out.println("Connected to Server!");
         System.out.println("You can start chatting. Type 'bye' to exit.\n");

         BufferedReader input  = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
         PrintWriter    output = new PrintWriter(socket.getOutputStream(), true);
         Scanner        sc     = new Scanner(System.in);

         // Thread to continuously receive messages from server
         Thread receiveThread = new Thread(() -> {
            try {
               String message;
               while ((message = input.readLine()) != null) {
                  if (message.equalsIgnoreCase("bye")) {
                     System.out.println("Server has left the chat.");
                     System.exit(0);
                  }
                  System.out.println("Server : " + message);
               }
            } catch (IOException e) {
               System.out.println("Connection lost.");
            }
         });

         // Thread to continuously send messages to server
         Thread sendThread = new Thread(() -> {
            String message;
            while (true) {
```

```
            System.out.print("Client : ");
            message = sc.nextLine();
            output.println(message);
            if (message.equalsIgnoreCase("bye")) {
               System.out.println("You left the chat.");
               System.exit(0);
            }
         }
      });

      receiveThread.start();
      sendThread.start();

      receiveThread.join();
      sendThread.join();

      socket.close();
      sc.close();

   } catch (IOException | InterruptedException e) {
      System.out.println("Client error: " + e.getMessage());
   }
  }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG41ChatServer.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG41ChatServer
Server started on port 5000
Waiting for client...
Client connected!
Type your message. Type 'bye' to exit.

Server : Hi! Who is this?
Client : This is Ayush. How are you?
Server : I am good! What do you need?
Client : Can you help me with Java?
Server : Sure! Ask your question.
Client : bye
Client has left the chat.
Connection closed.
```

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG41ChatClient.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG41ChatClient
Connected to Server!
Wait for server message, then reply. Type 'bye' to exit.

Server : Hi! Who is this?
Client : This is Ayush. How are you?
Server : I am good! What do you need?
Client : Can you help me with Java?
Server : Sure! Ask your question.
Client : bye
Client has left the chat.
Connection closed.
```

Program No: 42	**Date:** 17-01-2026

Program Title: A file name is given in the client, download the content from the server.

```
/*
File Download Client
@ Ashbi Saju
Roll no: 16
Date: 17-01-2026
*/
```

Server.java

```
import java.io.*;
import java.net.*;

public class PG42FileServer {
   public static void main(String[] args) {
      int port = 5000;

      try {
         ServerSocket serverSocket = new ServerSocket(port);
         System.out.println("File Server started on port " + port);
         System.out.println("Waiting for client...");

         Socket socket = serverSocket.accept();
         System.out.println("Client connected: " + socket.getInetAddress());

         BufferedReader input  = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
         PrintWriter    output = new PrintWriter(socket.getOutputStream(), true);

         // Receive filename from client
         String fileName = input.readLine();
         System.out.println("Client requested file: " + fileName);

         // Check if file exists
         File file = new File(fileName);

         if (!file.exists()) {
            output.println("ERROR: File not found on server.");
            System.out.println("File not found: " + fileName);
```

```
        } else {
            // Send file content line by line
            output.println("SUCCESS");
            BufferedReader fileReader = new BufferedReader(new FileReader(file));
            String line;
            System.out.println("Sending file content...");
            while ((line = fileReader.readLine()) != null) {
                output.println(line);
            }
            output.println("EOF");   // End of file marker
            fileReader.close();
            System.out.println("File sent successfully.");
        }

        input.close();
        output.close();
        socket.close();
        serverSocket.close();
        System.out.println("Connection closed.");

    } catch (IOException e) {
        System.out.println("Server error: " + e.getMessage());
    }
  }
}
```

Client.java

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class PG42FileClient {
  public static void main(String[] args) {
    String host = "localhost";
    int    port = 5000;

    try {
      Socket socket = new Socket(host, port);
      System.out.println("Connected to File Server.");
```

```
        BufferedReader input  = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
        PrintWriter    output = new PrintWriter(socket.getOutputStream(), true);
        Scanner        sc     = new Scanner(System.in);

        // Send filename to server
        System.out.print("Enter filename to download: ");
        String fileName = sc.nextLine();
        output.println(fileName);

        // Read server response
        String response = input.readLine();

        if (response.startsWith("ERROR")) {
           System.out.println(response);
        } else {
           System.out.println("Downloading file: " + fileName);
           System.out.println("--------- File Content ---------");

           // Receive and display file content
           String line;
           StringBuilder fileContent = new StringBuilder();
           while ((line = input.readLine()) != null) {
              if (line.equals("EOF")) break;
              System.out.println(line);
              fileContent.append(line).append("\n");
           }

           System.out.println("--------------------------------");

           // Save downloaded content to a new file
           String downloadedFile = "downloaded_" + fileName;
           BufferedWriter writer = new BufferedWriter(new FileWriter(downloadedFile));
           writer.write(fileContent.toString());
           writer.close();

           System.out.println("File saved as: " + downloadedFile);
        }

        input.close();
```

```
        output.close();
        socket.close();
        sc.close();
        System.out.println("Connection closed.");

      } catch (IOException e) {
        System.out.println("Client error: " + e.getMessage());
      }
   }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG42FileServer.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG42FileServer
File Server started on port 5000
Waiting for client...
Client connected: /127.0.0.1
Client requested file: Sample.txt
Sending file content...
File sent successfully.
Connection closed.
```

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG42FileClient.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG42FileClient
Connected to File Server.
Enter filename to download: Sample.txt
Downloading file: Sample.txt
--------- File Content ---------
--------------------------------
File saved as: downloaded_Sample.txt
Connection closed.
```

Program No: 43	**Date:** 17-01-2026

Program Title: Upload the content from client to server.

```
/*
File Upload Client
@ Ashbi Saju
Roll no: 16
Date: 17-01-2026
*/
```

Server.java

```
import java.io.*;
import java.net.*;

public class PG43UploadServer {
   public static void main(String[] args) {
      int port = 5000;

      try {
         ServerSocket serverSocket = new ServerSocket(port);
         System.out.println("Upload Server started on port " + port);
         System.out.println("Waiting for client...");

         Socket socket = serverSocket.accept();
         System.out.println("Client connected: " + socket.getInetAddress());

         BufferedReader input  = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
         PrintWriter    output = new PrintWriter(socket.getOutputStream(), true);

         // Receive filename from client
         String fileName = input.readLine();
         System.out.println("Client is uploading file: " + fileName);

         // Save uploaded content to a new file on server
         String savedFile = "uploaded_" + fileName;
         BufferedWriter fileWriter = new BufferedWriter(new FileWriter(savedFile));

         String line;
```

```
        int lineCount = 0;

        // Receive file content line by line until EOF
        while ((line = input.readLine()) != null) {
           if (line.equals("EOF")) break;
           fileWriter.write(line);
           fileWriter.newLine();
           lineCount++;
        }

        fileWriter.close();
        System.out.println("File received successfully.");
        System.out.println("Total lines received : " + lineCount);
        System.out.println("File saved as       : " + savedFile);

        // Send acknowledgement to client
        output.println("SUCCESS: File uploaded. Saved as " + savedFile);

        input.close();
        output.close();
        socket.close();
        serverSocket.close();
        System.out.println("Connection closed.");

     } catch (IOException e) {
        System.out.println("Server error: " + e.getMessage());
     }
  }
}
```

Client.java

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class PG43UploadClient {
   public static void main(String[] args) {
      String host = "localhost";
      int    port = 5000;
```

```
    try {
        Socket socket = new Socket(host, port);
        System.out.println("Connected to Upload Server.");

        BufferedReader input  = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
        PrintWriter    output = new PrintWriter(socket.getOutputStream(), true);
        Scanner        sc     = new Scanner(System.in);

        // Ask for filename to upload
        System.out.print("Enter filename to upload: ");
        String fileName = sc.nextLine();

        // Check if file exists on client side
        File file = new File(fileName);
        if (!file.exists()) {
            System.out.println("ERROR: File not found on client.");
            socket.close();
            sc.close();
            return;
        }

        // Send filename to server
        output.println(fileName);

        // Read and send file content line by line
        BufferedReader fileReader = new BufferedReader(new FileReader(file));
        String line;
        int lineCount = 0;

        System.out.println("Uploading file: " + fileName);
        System.out.println("--------- File Content ---------");

        while ((line = fileReader.readLine()) != null) {
            System.out.println(line);
            output.println(line);
            lineCount++;
        }
        output.println("EOF");   // End of file marker
        fileReader.close();
```

```
        System.out.println("--------------------------------");
        System.out.println("Total lines sent : " + lineCount);
        System.out.println("Upload complete.");

        // Receive acknowledgement from server
        String ack = input.readLine();
        System.out.println("Server response  : " + ack);

        input.close();
        output.close();
        socket.close();
        sc.close();
        System.out.println("Connection closed.");

    } catch (IOException e) {
        System.out.println("Client error: " + e.getMessage());
    }
  }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG43UploadServer.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG43UploadServer
Upload Server started on port 5000
Waiting for client...
Client connected: /127.0.0.1
Client is uploading file: Sample.txt
File received successfully.
Total lines received : 0
File saved as        : uploaded_Sample.txt
Connection closed.
```

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG43UploadClient.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG43UploadClient
Connected to Upload Server.
Enter filename to upload: Sample.txt
Uploading file: Sample.txt
--------- File Content ---------
--------------------------------
Total lines sent : 0
Upload complete.
Server response  : SUCCESS: File uploaded. Saved as uploaded_Sample.txt
Connection closed.
```

Program No: 44	**Date:** 17-01-2026

Program Title: Program to implement public chatting.

```
/*
Public Chat System
@ Ashbi Saju
Roll no: 16
Date: 17-01-2026
*/
```

Server.java

```
import java.io.*;
import java.net.*;
import java.util.*;

public class PG44PublicChatServer {

   // List to keep track of all connected clients
   static List<ClientHandler> clients = new ArrayList<>();

   public static void main(String[] args) {
      int port = 5000;

      try {
         ServerSocket serverSocket = new ServerSocket(port);
         System.out.println("Public Chat Server started on port " + port);
         System.out.println("Waiting for clients...\n");

         // Keep accepting new clients
         while (true) {
            Socket socket = serverSocket.accept();
            System.out.println("New client connected: " + socket.getInetAddress());

            ClientHandler client = new ClientHandler(socket);
            clients.add(client);
            client.start();
         }

      } catch (IOException e) {
```

```
            System.out.println("Server error: " + e.getMessage());
        }
    }

    // Broadcast message to all connected clients except the sender
    static void broadcast(String message, ClientHandler sender) {
        for (ClientHandler client : clients) {
            if (client != sender) {
                client.sendMessage(message);
            }
        }
    }

    // Remove client from list when disconnected
    static void removeClient(ClientHandler client) {
        clients.remove(client);
        System.out.println(client.name + " disconnected. Active clients: " + clients.size());
    }
}

// Handles each client in a separate thread
class ClientHandler extends Thread {

    Socket        socket;
    BufferedReader input;
    PrintWriter    output;
    String        name;

    public ClientHandler(Socket socket) {
        this.socket = socket;
    }

    public void run() {
        try {
            input  = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            output = new PrintWriter(socket.getOutputStream(), true);

            // First message from client is their name
            output.println("Enter your name: ");
            name = input.readLine();
            System.out.println(name + " has joined the chat.");
```

```
        // Notify all other clients
        PG44PublicChatServer.broadcast(name + " has joined the chat!", this);
        output.println("Welcome " + name + "! You are now connected.");
        output.println("Type 'bye' to leave the chat.\n");

        String message;

        // Keep receiving and broadcasting messages
        while ((message = input.readLine()) != null) {
            if (message.equalsIgnoreCase("bye")) {
                PG44PublicChatServer.broadcast(name + " has left the chat.", this);
                System.out.println(name + " has left the chat.");
                break;
            }
            System.out.println("[" + name + "] : " + message);
            PG44PublicChatServer.broadcast("[" + name + "] : " + message, this);
        }

    } catch (IOException e) {
        System.out.println(name + " disconnected unexpectedly.");
    } finally {
        PG44PublicChatServer.removeClient(this);
        try { socket.close(); } catch (IOException e) {}
    }
  }

  // Send a message to this client
  public void sendMessage(String message) {
    output.println(message);
  }
}
```

Client.java

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class PG44PublicChatClient {
```

```
   public static void main(String[] args) {
      String host = “localhost”;
      int    port = 5000;

      try {
         Socket socket = new Socket(host, port);
         System.out.println(“Connected to Public Chat Server.”);

         BufferedReader input  = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
         PrintWriter    output = new PrintWriter(socket.getOutputStream(), true);
         Scanner        sc     = new Scanner(System.in);

         // Thread to receive messages from server
         Thread receiveThread = new Thread(() -> {
            try {
               String message;
               while ((message = input.readLine()) != null) {
                  System.out.println(message);
               }
            } catch (IOException e) {
               System.out.println(“Disconnected from server.”);
            }
         });

         // Thread to send messages to server
         Thread sendThread = new Thread(() -> {
            try {
               String message;
               while ((message = sc.nextLine()) != null) {
                  output.println(message);
                  if (message.equalsIgnoreCase(“bye”)) {
                     System.out.println(“You left the chat.”);
                     break;
                  }
               }
            } catch (Exception e) {
               System.out.println(“Error sending message.”);
            }
         });
```

```
        receiveThread.start();
        sendThread.start();

        receiveThread.join();
        sendThread.join();

        socket.close();
        sc.close();

      } catch (IOException | InterruptedException e) {
        System.out.println("Client error: " + e.getMessage());
      }
   }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG44PublicChatServer.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG44PublicChatServer
Public Chat Server started on port 5000
Waiting for clients...

New client connected: /127.0.0.1
Ashbi has joined the chat.
New client connected: /127.0.0.1
Jees has joined the chat.
New client connected: /127.0.0.1
Kishore has joined the chat.
[Ashbi] : Hii
[Jees] : Hiii
[Kishore] : Hii, how ae you doing?
[Ashbi] : Doing good
[Jees] : Good
Ashbi has left the chat.
Ashbi disconnected. Active clients: 2
Jees has left the chat.
Jees disconnected. Active clients: 1
Kishore has left the chat.
Kishore disconnected. Active clients: 0
^C
```

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG44PublicChatClient.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG44PublicChatClient
Connected to Public Chat Server.
Enter your name:
Ashbi
Welcome Ashbi! You are now connected.
Type 'bye' to leave the chat.

Jees has joined the chat!
Kishore has joined the chat!
Hii
[Jees] : Hiii
[Kishore] : Hii, how ae you doing?
Doing good
[Jees] : Good
bye
You left the chat.
```

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG44PublicChatClient
Connected to Public Chat Server.
Enter your name:
Jees
Welcome Jees! You are now connected.
Type 'bye' to leave the chat.

Kishore has joined the chat!
[Ashbi] : Hii
Hiii
[Kishore] : Hii, how ae you doing?
[Ashbi] : Doing good
Good
Ashbi has left the chat.
bye
You left the chat.
```

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG44PublicChatClient
Connected to Public Chat Server.
Enter your name:
Kishore
Welcome Kishore! You are now connected.
Type 'bye' to leave the chat.

[Ashbi] : Hii
[Jees] : Hiii
Hii, how ae you doing?
[Ashbi] : Doing good
[Jees] : Good
Ashbi has left the chat.
Jees has left the chat.
bye
You left the chat.
```

Program No: 45	**Date:** 19-01-2026

Program Title: Program to implement connection less communication.

```
/*
Connectionless Communication
@ Ashbi Saju
Roll no: 16
Date: 19-01-2026
*/
```

Server.java

```
import java.net.*;

public class PG45UDPServer {
   public static void main(String[] args) {
      int port = 5000;

      try {
         // Create UDP socket
         DatagramSocket socket = new DatagramSocket(port);
         System.out.println("UDP Server started on port " + port);
         System.out.println("Waiting for messages...\n");

         byte[] buffer = new byte[1024];

         while (true) {
            // Receive packet from client
            DatagramPacket receivePacket = new DatagramPacket(buffer, buffer.length);
            socket.receive(receivePacket);

            String message = new String(receivePacket.getData(), 0, receivePacket.getLength());
            InetAddress clientAddress = receivePacket.getAddress();
            int         clientPort    = receivePacket.getPort();

            System.out.println("Client [" + clientAddress + ":" + clientPort + "] : " + message);

            if (message.equalsIgnoreCase("bye")) {
               System.out.println("Client has disconnected.");
               // Send bye back to client
```

```
            byte[] replyData = "bye".getBytes();
            DatagramPacket replyPacket = new DatagramPacket(
               replyData, replyData.length, clientAddress, clientPort);
            socket.send(replyPacket);
            break;
         }

         // Send reply to client
         String reply     = "Server received: " + message;
         byte[] replyData  = reply.getBytes();
         DatagramPacket replyPacket = new DatagramPacket(
            replyData, replyData.length, clientAddress, clientPort);
         socket.send(replyPacket);
         System.out.println("Server replied  : " + reply);
      }

      socket.close();
      System.out.println("Server socket closed.");

   } catch (Exception e) {
      System.out.println("Server error: " + e.getMessage());
   }
  }
}
```

Client.java

```
import java.net.*;
import java.util.Scanner;

public class PG45UDPClient {
  public static void main(String[] args) {
    String host = "localhost";
    int    port = 5000;

    try {
      // Create UDP socket (no connection needed)
      DatagramSocket socket   = new DatagramSocket();
      InetAddress    address  = InetAddress.getByName(host);
      Scanner        sc       = new Scanner(System.in);
```

```
        System.out.println("UDP Client started.");
        System.out.println("Connected to Server at " + host + ":" + port);
        System.out.println("Type your message (type 'bye' to exit):\n");

        byte[] buffer = new byte[1024];

        while (true) {
            // Send message to server
            System.out.print("Client : ");
            String message  = sc.nextLine();
            byte[] sendData = message.getBytes();

            DatagramPacket sendPacket = new DatagramPacket(
                sendData, sendData.length, address, port);
            socket.send(sendPacket);

            // Receive reply from server
            DatagramPacket receivePacket = new DatagramPacket(buffer, buffer.length);
            socket.receive(receivePacket);

            String reply = new String(receivePacket.getData(), 0, receivePacket.getLength());

            if (reply.equalsIgnoreCase("bye") || message.equalsIgnoreCase("bye")) {
                System.out.println("Disconnected from server.");
                break;
            }

            System.out.println("Server : " + reply);
        }

        socket.close();
        sc.close();
        System.out.println("Client socket closed.");

    } catch (Exception e) {
        System.out.println("Client error: " + e.getMessage());
    }
  }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG45UDPServer.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG45UDPServer
UDP Server started on port 5000
Waiting for messages...

Client [/127.0.0.1:59227] : Hello Server!
Server replied  : Server received: Hello Server!
Client [/127.0.0.1:59227] : How are you?
Server replied  : Server received: How are you?
Client [/127.0.0.1:59227] : So this is UDP...
Server replied  : Server received: So this is UDP...
Client [/127.0.0.1:59227] : bye
Client has disconnected.
Server socket closed.
```

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG45UDPClient.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG45UDPClient
UDP Client started.
Connected to Server at localhost:5000
Type your message (type 'bye' to exit):

Client : Hello Server!
Server : Server received: Hello Server!
Client : How are you?
Server : Server received: How are you?
Client : So this is UDP...
Server : Server received: So this is UDP...
Client : bye
Disconnected from server.
Client socket closed.
```

Program No: 46	**Date:** 19-01-2026

Program Title: Program to get protocol, file name, host, path and port of a given URL.

```
/*
URL Components Extractor
@ Ashbi Saju
Roll no: 16
Date: 19-01-2026
*/

import java.net.*;

public class PG46URLInfo {
   public static void main(String[] args) {
      try {
         // Test with multiple URLs
         String[] urls = {
            "https://www.google.com/search?q=java",
            "http://www.example.com:8080/docs/index.html",
            "https://github.com/user/repository",
            "ftp://ftp.example.com/files/sample.txt"
         };

         for (String urlString : urls) {
            System.out.println("==========================================");
            System.out.println("URL     : " + urlString);
            System.out.println("------------------------------------------");

            URL url = new URL(urlString);

            System.out.println("Protocol : " + url.getProtocol());
            System.out.println("Host     : " + url.getHost());
            System.out.println("Port     : " + (url.getPort() == -1 ? "Default (" + url.getDefaultPort() +
")" : url.getPort()));
            System.out.println("Path     : " + (url.getPath().isEmpty() ? "/" : url.getPath()));
            System.out.println("File     : " + (url.getFile().isEmpty() ? "None" : url.getFile()));
            System.out.println("Query    : " + (url.getQuery() == null ? "None" : url.getQuery()));
            System.out.println("Ref      : " + (url.getRef()   == null ? "None" : url.getRef()));
         }
```

```
            System.out.println("========================================");
        } catch (MalformedURLException e) {
            System.out.println("Invalid URL: " + e.getMessage());
        }
    }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG46URLInfo.java
Note: PG46URLInfo.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG46URLInfo
========================================
URL      : https://www.google.com/search?q=java
----------------------------------------
Protocol : https
Host     : www.google.com
Port     : Default (443)
Path     : /search
File     : /search?q=java
Query    : q=java
Ref      : None
========================================
URL      : http://www.example.com:8080/docs/index.html
----------------------------------------
Protocol : http
Host     : www.example.com
Port     : 8080
Path     : /docs/index.html
File     : /docs/index.html
Query    : None
Ref      : None
========================================
URL      : ftp://ftp.example.com/files/sample.txt
----------------------------------------
Protocol : ftp
Host     : ftp.example.com
Port     : Default (21)
Path     : /files/sample.txt
File     : /files/sample.txt
Query    : None
Ref      : None
========================================
```

Program No: 47	**Date:** 19-01-2026

Program Title: Program to download a file from given URL.

```
/*
URL File Downloader
@ Ashbi Saju
Roll no: 16
Date: 19-01-2026
*/

import java.io.*;
import java.net.*;

public class PG47URLDownload {
   // Method to download file from URL
   static void downloadFile(String urlString) {
      try {
         URL url = new URL(urlString);

         // Extract filename from URL
         String fileName = urlString.substring(urlString.lastIndexOf("/") + 1);
         if (fileName.isEmpty() || !fileName.contains(".")) {
            fileName = "downloaded_file.html";
         }

         System.out.println("URL      : " + urlString);
         System.out.println("Protocol : " + url.getProtocol());
         System.out.println("Host     : " + url.getHost());
         System.out.println("File     : " + fileName);
         System.out.println("----------------------------------------");
         System.out.println("Starting download...");

         // Open connection to URL
         HttpURLConnection connection = (HttpURLConnection) url.openConnection();
         connection.setRequestMethod("GET");
         connection.setConnectTimeout(10000);
         connection.setReadTimeout(10000);

         int responseCode = connection.getResponseCode();
         System.out.println("Response Code : " + responseCode);
```

```
        if (responseCode == HttpURLConnection.HTTP_OK) {

            // Get file size
            long fileSize = connection.getContentLengthLong();
            System.out.println("File Size     : " +
               (fileSize == -1 ? "Unknown" : formatSize(fileSize)));
            System.out.println("Content Type  : " + connection.getContentType());
            System.out.println("----------------------------------------");

            // Input stream from URL
            InputStream inputStream  = connection.getInputStream();

            // Output stream to save file
            FileOutputStream  outputStream = new FileOutputStream(fileName);

            byte[] buffer     = new byte[4096];
            int    bytesRead;
            long   totalBytes = 0;

            // Read and write in chunks
            while ((bytesRead = inputStream.read(buffer)) != -1) {
               outputStream.write(buffer, 0, bytesRead);
               totalBytes += bytesRead;
               System.out.print("\rDownloaded: " + formatSize(totalBytes));
            }
            outputStream.close();
            inputStream.close();
            connection.disconnect();
            System.out.println("\nDownload complete!");
            System.out.println("Saved as      : " + fileName);
            System.out.println("Total Size    : " + formatSize(totalBytes));
        } else {
            System.out.println("Download failed. Server returned: " + responseCode);
        }
    } catch (MalformedURLException e) {
        System.out.println("Invalid URL   : " + e.getMessage());
    } catch (IOException e) {
        System.out.println("Download error: " + e.getMessage());
    }
}
```

```
    // Helper method to format file size
    static String formatSize(long bytes) {
        if (bytes < 1024)
            return bytes + " B";
        else if (bytes < 1024 * 1024)
            return String.format("%.2f KB", bytes / 1024.0);
        else if (bytes < 1024 * 1024 * 1024)
            return String.format("%.2f MB", bytes / (1024.0 * 1024));
        else
            return String.format("%.2f GB", bytes / (1024.0 * 1024 * 1024));
    }
    public static void main(String[] args) {
        java.util.Scanner sc = new java.util.Scanner(System.in);
        System.out.println("--- URL File Downloader ---\n");
        System.out.print("Enter URL to download: ");
        String urlString = sc.nextLine().trim();
        System.out.println("----------------------------------------");
        downloadFile(urlString);
        System.out.println("----------------------------------------");
        sc.close();
    }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG47URLDownload.java
Note: PG47URLDownload.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG47URLDownload
--- URL File Downloader ---

Enter URL to download: https://www.iitk.ac.in/esc101/share/downloads/javanotes5.pdf
----------------------------------------
URL      : https://www.iitk.ac.in/esc101/share/downloads/javanotes5.pdf
Protocol : https
Host     : www.iitk.ac.in
File     : javanotes5.pdf
----------------------------------------
Starting download...
Response Code : 200
File Size     : 5.01 MB
Content Type  : application/pdf
----------------------------------------
Downloaded: 5.01 MB KB
Download complete!
Saved as      : javanotes5.pdf
Total Size    : 5.01 MB
----------------------------------------
```

Program No: 48	**Date:** 31-01-2026

Program Title: Program to display the content of a webpage.

```
/*
Webpage Content Viewer
@ Ashbi Saju
Roll no: 16
Date: 31-01-2026
*/

import java.io.*;
import java.net.*;

public class PG48WebPageContent {

   // Method to display webpage content
   static void displayContent(String urlString) {
      try {
         URL url = new URL(urlString);

         // Open HTTP connection
         HttpURLConnection connection = (HttpURLConnection) url.openConnection();
         connection.setRequestMethod("GET");
         connection.setConnectTimeout(10000);
         connection.setReadTimeout(10000);

         // Set browser-like header to avoid request rejection
         connection.setRequestProperty("User-Agent",
            "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36");

         int responseCode = connection.getResponseCode();

         System.out.println("URL          : " + urlString);
         System.out.println("Protocol     : " + url.getProtocol());
         System.out.println("Host         : " + url.getHost());
         System.out.println("Response Code : " + responseCode);
         System.out.println("Content Type  : " + connection.getContentType());
         System.out.println("----------------------------------------");

         if (responseCode == HttpURLConnection.HTTP_OK) {
```

```
        // Read webpage content
        BufferedReader reader = new BufferedReader(
            new InputStreamReader(connection.getInputStream()));

        String line;
        int    lineCount = 0;
        StringBuilder content = new StringBuilder();

        System.out.println("---------- Webpage Content ----------\n");

        while ((line = reader.readLine()) != null) {
            System.out.println(line);
            content.append(line).append("\n");
            lineCount++;
        }

        reader.close();
        connection.disconnect();

        System.out.println("\n-----------------------------------------");
        System.out.println("Total Lines   : " + lineCount);
        System.out.println("Total Chars   : " + content.length());

        // Save content to a file
        String fileName = "webpage_content.html";
        BufferedWriter writer = new BufferedWriter(new FileWriter(fileName));
        writer.write(content.toString());
        writer.close();

        System.out.println("Content saved : " + fileName);

    } else if (responseCode == HttpURLConnection.HTTP_MOVED_TEMP ||
               responseCode == HttpURLConnection.HTTP_MOVED_PERM) {
        System.out.println("Page redirected to: " + connection.getHeaderField("Location"));

    } else {
        System.out.println("Failed to fetch page. Response: " + responseCode);
    }

} catch (MalformedURLException e) {
```

```
            System.out.println("Invalid URL   : " + e.getMessage());
        } catch (IOException e) {
            System.out.println("Connection error: " + e.getMessage());
        }
    }

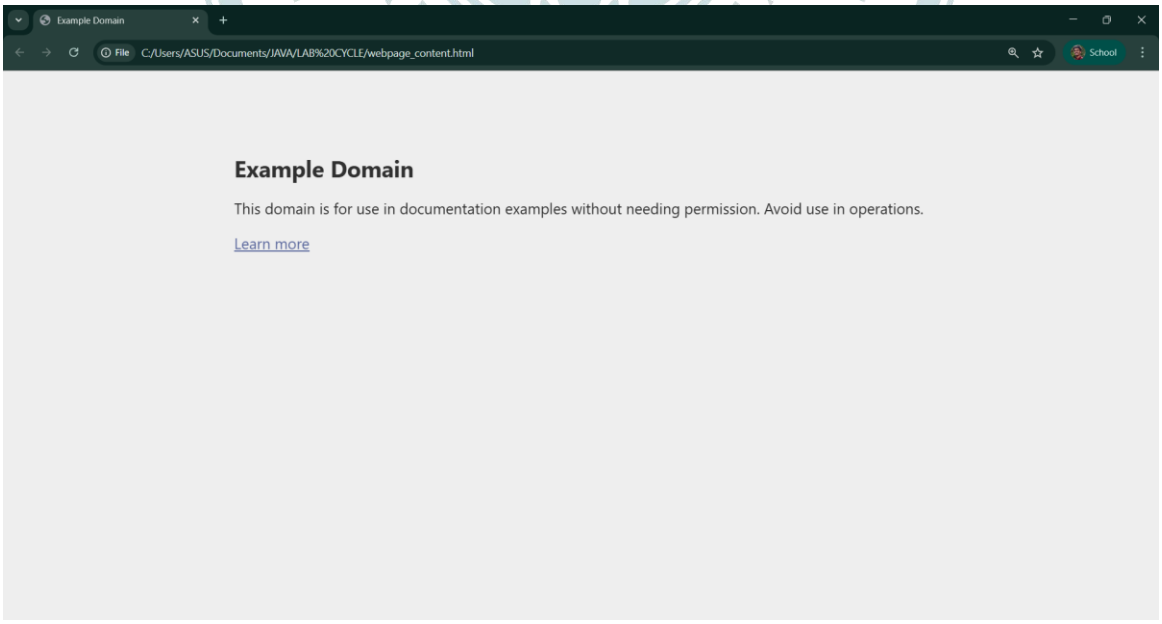
    public static void main(String[] args) {
        java.util.Scanner sc = new java.util.Scanner(System.in);

        System.out.println("--- Webpage Content Viewer ---\n");
        System.out.print("Enter URL: ");
        String urlString = sc.nextLine().trim();

        System.out.println("----------------------------------------");
        displayContent(urlString);
        System.out.println("----------------------------------------");

        sc.close();
    }
}
```

Output



```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG48WebPageContent
--- Webpage Content Viewer ---

Enter URL: http://example.com
----------------------------------------
URL           : http://example.com
Protocol      : http
Host          : example.com
Response Code : 200
Content Type  : text/html
----------------------------------------
---------- Webpage Content ----------

<!doctype html>
<html lang="en">
    <head>
        <title>
            Example Domain
        </title>
        <meta name="viewport" content="width=device-width, initial-scale=1">
        <style>
            body{background:#eee;width:60vw;margin:15vh auto;font-family:system-ui,sans-ser
if}h1{font-size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}
        </style>
    </head>
    <body>
        <div>
            <h1>
                Example Domain
            </h1>
            <p>
                This domain is for use in documentation examples without needing permission
. Avoid use in operations.
            </p>
            <p>
                <a href="https://iana.org/domains/example">
                    Learn more
                </a>
            </p>
        </div>
    </body>
</html>

------------------------------------
Content saved : webpage_content.html
----------------------------------------
```

Program No: 49	**Date:** 31-01-2026

Program Title: Program to insert values into a table.

```
/*
Database Insert Operation
@ Ashbi Saju
Roll no: 16
Date: 31-01-2026
*/

import java.sql.*;
import java.util.Scanner;

public class PG49InsertValues {

   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);
      String no, name, sal, sql;

      try {
         // Step 1: Load Oracle JDBC Driver
         Class.forName("oracle.jdbc.OracleDriver");
         System.out.println("Driver loaded successfully.");

         // Step 2: Establish connection
         Connection con = DriverManager.getConnection(
            "jdbc:oracle:thin:@localhost:1521:orcl", "mca", "mca");
         System.out.println("Database connected successfully.");

         // Step 3: Create Statement
         Statement st = con.createStatement();

         // Step 4: Get input and insert
         char choice;
         int  count = 0;

         do {
            System.out.println("\nEnter Employee Details:");
            System.out.print("Enter the Emp No   : ");
```

```
        no = sc.nextLine();

        System.out.print("Enter the Emp Name : ");
        name = sc.nextLine();

        System.out.print("Enter the Emp Salary: ");
        sal = sc.nextLine();

        sql = "INSERT INTO Emp16 VALUES(" + no + ",'" + name + "'," + sal + ")";

        st.executeUpdate(sql);
        count++;
        System.out.println("Record inserted successfully!");

        System.out.print("\nInsert another record? (y/n): ");
        choice = sc.nextLine().charAt(0);

    } while (choice == 'y' || choice == 'Y');

    System.out.println("\nTotal records inserted: " + count);

    // Step 5: Display all records
    System.out.println("\n---------- Emp16 Table ----------");
    System.out.printf("%-10s %-20s %-10s%n", "Emp No", "Emp Name", "Salary");
    System.out.println("-------------------------------------");

    ResultSet rs = st.executeQuery("SELECT * FROM Emp16");
    while (rs.next()) {
        System.out.printf("%-10s %-20s %-10s%n",
            rs.getString(1),
            rs.getString(2),
            rs.getString(3));
    }
    System.out.println("-----------------------------------------");

    // Step 6: Close resources
    rs.close();
    st.close();
    con.close();
    sc.close();
    System.out.println("Connection closed.");
```

```
    } catch (ClassNotFoundException e) {
        System.out.println("Driver not found : " + e.getMessage());
    } catch (SQLException e) {
        System.out.println("SQL error       : " + e.getMessage());
    }
  }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac -cp .;ojdbc17.jar PG49InsertValues.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java -cp .;ojdbc17.jar PG49InsertValues
Driver loaded successfully.
Database connected successfully.

Enter Employee Details:
Enter the Emp No   : 101
Enter the Emp Name : Ashbi
Enter the Emp Salary: 50000
Record inserted successfully!

Insert another record? (y/n): y

Enter Employee Details:
Enter the Emp No   : 102
Enter the Emp Name : Jees
Enter the Emp Salary: 40000
Record inserted successfully!

Insert another record? (y/n): y

Enter Employee Details:
Enter the Emp No   : 103
Enter the Emp Name : Kishor
Enter the Emp Salary: 45000
Record inserted successfully!

Insert another record? (y/n): n

Total records inserted: 3

---------- Emp16 Table ----------
Emp No     Emp Name             Salary
---------------------------------------
101        Ashbi                50000
102        Jees                 40000
103        Kishor               45000
---------------------------------------
Connection closed.
```

Program No: 50	**Date:** 31-01-2026

Program Title: Assume that login is a table which has Uname, Upass and some records. Check whether the record is present or not.

```
/*
Login Authentication Check
@ Ashbi Saju
Roll no: 16
Date: 31-01-2026
*/

import java.sql.*;
import java.util.Scanner;

public class PG50LoginCheck {

   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);
      String uname, upass, sql;

      try {
         // Step 1: Load Oracle JDBC Driver
         Class.forName("oracle.jdbc.OracleDriver");

         // Step 2: Establish connection
         Connection con = DriverManager.getConnection(
            "jdbc:oracle:thin:@localhost:1521:orcl", "mca", "mca");
         System.out.println("Database connected successfully.");

         // Step 3: Create Statement
         Statement st = con.createStatement();

         // Step 4: Get login credentials from user
         System.out.println("\n---------- Login ----------");
         System.out.print("Enter Username : ");
         uname = sc.nextLine();

         System.out.print("Enter Password : ");
         upass = sc.nextLine();
```

```
        // Step 5: Query to check record
        sql = "SELECT * FROM login WHERE Uname='" + uname + "' AND Upass='" + upass + "'";

        ResultSet rs = st.executeQuery(sql);

        // Step 6: Check if record exists
        if (rs.next()) {
            System.out.println("\nLogin Successful! Welcome, " + uname + "!");
        } else {
            System.out.println("\nLogin Failed! Invalid Username or Password.");
        }

        // Step 7: Close resources
        rs.close();
        st.close();
        con.close();
        sc.close();

    } catch (ClassNotFoundException e) {
        System.out.println("Driver not found : " + e.getMessage());
    } catch (SQLException e) {
        System.out.println("SQL error      : " + e.getMessage());
    }
  }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac -cp .;ojdbc17.jar PG50LoginCheck.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java -cp .;ojdbc17.jar PG50LoginCheck
Database connected successfully.

---------- Login ----------
Enter Username : Ashbi
Enter Password : ashbi123

Login Successful! Welcome, Ashbi!

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java -cp .;ojdbc17.jar PG50LoginCheck
Database connected successfully.

---------- Login ----------
Enter Username : Ashbi
Enter Password : ashbi

Login Failed! Invalid Username or Password.
```

Program No: 51	**Date:** 02-02-2026

Program Title: Department is table having dno, dname, dloc. Emp is table having eno, ename, esal and dno. Write a program for displaying the information in the following order:

eno	ename	esal	dname	dloc
1	abc	10,000	MCA	MB
2	bcd	20,000	MSW	OB

```
/*
Employee Department Join
@ Ashbi Saju
Roll no: 16
Date: 02-02-2026
*/

import java.sql.*;

public class PG51JoinDisplay {

   public static void main(String[] args) {

      try {
         // Step 1: Load Oracle JDBC Driver
         Class.forName("oracle.jdbc.OracleDriver");

         // Step 2: Establish connection
         Connection con = DriverManager.getConnection(
            "jdbc:oracle:thin:@localhost:1521:orcl", "mca", "mca");
         System.out.println("Database connected successfully.");

         // Step 3: Create Statement
         Statement st = con.createStatement();

         // Step 4: JOIN query to fetch data from both tables
         String sql = "SELECT e.eno, e.ename, e.esal, d.dname, d.dloc " +
                  "FROM Emp e, Department d " +
                  "WHERE e.dno = d.dno";

         ResultSet rs = st.executeQuery(sql);

         // Step 5: Display results
         System.out.println("\n---------- Employee - Department Details ----------");
         System.out.printf("%-5s %-15s %-12s %-15s %-15s%n",
```

```
                    "ENO", "ENAME", "ESAL", "DNAME", "DLOC");
        System.out.println("-------------------------------------------------");

        int count = 0;
        while (rs.next()) {
           System.out.printf("%-5s %-15s %-12s %-15s %-15s%n",
              rs.getString("eno"),
              rs.getString("ename"),
              rs.getString("esal"),
              rs.getString("dname"),
              rs.getString("dloc"));
           count++;
        }
        System.out.println("-------------------------------------------------");
        System.out.println("Total Records : " + count);

        // Step 6: Close resources
        rs.close();
        st.close();
        con.close();
        System.out.println("Connection closed.");

     } catch (ClassNotFoundException e) {
        System.out.println("Driver not found : " + e.getMessage());
     } catch (SQLException e) {
        System.out.println("SQL error       : " + e.getMessage());
     }
   }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac -cp .;ojdbc17.jar PG51JoinDisplay.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java -cp .;ojdbc17.jar PG51JoinDisplay
Database connected successfully.

---------- Employee - Department Details ----------
ENO   ENAME           ESAL         DNAME           DLOC
-------------------------------------------------
1     abc             10000        MCA             MB
2     bcd             20000        MSW             OB
-------------------------------------------------
Total Records : 2
Connection closed.
```

Program No: 52	**Date:** 02-02-2026

Program Title: Write a database program including
a. callable statement
b. prepared statement

```
/*
JDBC Statements Demo
@ Ashbi Saju
Roll no: 16
Date: 02-02-2026
*/

/*
CREATE OR REPLACE PROCEDURE get_salary(p_eno IN NUMBER, p_sal OUT NUMBER)
AS BEGIN
  SELECT sal INTO p_sal FROM Emp16 WHERE eno = p_eno;
END;
/

CREATE OR REPLACE PROCEDURE get_count(p_count OUT NUMBER)
AS BEGIN
  SELECT COUNT(*) INTO p_count FROM Emp16;
END;
/

CREATE OR REPLACE PROCEDURE raise_salary(p_eno IN NUMBER, p_percent IN NUMBER,
p_newsal OUT NUMBER)
AS BEGIN
  UPDATE Emp16 SET sal = sal + (sal * p_percent / 100) WHERE eno = p_eno;
  SELECT sal INTO p_newsal FROM Emp16 WHERE eno = p_eno;
END;
/
*/

import java.sql.*;
import java.util.Scanner;

public class PG52CallablePrepared {

   // ── a. PREPARED STATEMENT ──────────────────────────────────────────
   static void preparedStatementDemo(Connection con) throws SQLException {
      Scanner sc = new Scanner(System.in);
      String  sql;
```

```
System.out.println("\n========== PREPARED STATEMENT ==========");
System.out.println("1. Insert Employee");
System.out.println("2. Update Salary");
System.out.println("3. Delete Employee");
System.out.println("4. Search Employee");
System.out.print("Enter choice: ");
int choice = Integer.parseInt(sc.nextLine());

switch (choice) {

  // INSERT
  case 1:
    sql = "INSERT INTO Emp16 VALUES(?, ?, ?)";
    PreparedStatement pInsert = con.prepareStatement(sql);

    System.out.print("Enter Emp No   : ");
    pInsert.setInt(1, Integer.parseInt(sc.nextLine()));
    System.out.print("Enter Emp Name : ");
    pInsert.setString(2, sc.nextLine());
    System.out.print("Enter Emp Sal  : ");
    pInsert.setFloat(3, Float.parseFloat(sc.nextLine()));

    int ins = pInsert.executeUpdate();
    if (ins > 0)
      System.out.println("Record inserted successfully!");
    pInsert.close();
    break;

  // UPDATE
  case 2:
    sql = "UPDATE Emp16 SET esal = ? WHERE eno = ?";
    PreparedStatement pUpdate = con.prepareStatement(sql);

    System.out.print("Enter Emp No     : ");
    int updateNo = Integer.parseInt(sc.nextLine());
    System.out.print("Enter New Salary  : ");
    float newSal = Float.parseFloat(sc.nextLine());

    pUpdate.setFloat(1, newSal);
    pUpdate.setInt  (2, updateNo);
```

```
            int upd = pUpdate.executeUpdate();
            if (upd > 0)
                System.out.println("Salary updated successfully!");
            else
                System.out.println("Employee not found.");
            pUpdate.close();
            break;

        // DELETE
        case 3:
            sql = "DELETE FROM Emp16 WHERE eno = ?";
            PreparedStatement pDelete = con.prepareStatement(sql);

            System.out.print("Enter Emp No to delete: ");
            pDelete.setInt(1, Integer.parseInt(sc.nextLine()));

            int del = pDelete.executeUpdate();
            if (del > 0)
                System.out.println("Record deleted successfully!");
            else
                System.out.println("Employee not found.");
            pDelete.close();
            break;

        // SEARCH
        case 4:
            sql = "SELECT * FROM Emp16 WHERE eno = ?";
            PreparedStatement pSearch = con.prepareStatement(sql);

            System.out.print("Enter Emp No to search: ");
            pSearch.setInt(1, Integer.parseInt(sc.nextLine()));

            ResultSet rs = pSearch.executeQuery();
            if (rs.next()) {
                System.out.println("\nEmployee Found:");
                System.out.println("Emp No   : " + rs.getInt   ("eno"));
                System.out.println("Emp Name : " + rs.getString ("ename"));
                System.out.println("Emp Sal  : " + rs.getFloat  ("esal"));
            } else {
                System.out.println("Employee not found.");
            }
```

```
            rs.close();
            pSearch.close();
            break;

        default:
            System.out.println("Invalid choice.");
    }
}

// ── b. CALLABLE STATEMENT ──────────────────────────────────────────────
static void callableStatementDemo(Connection con) throws SQLException {
    Scanner sc = new Scanner(System.in);

    System.out.println("\n========== CALLABLE STATEMENT ==========");
    System.out.println("1. Get Employee Salary  (IN parameter)");
    System.out.println("2. Get Employee Count   (OUT parameter)");
    System.out.println("3. Raise Salary       (IN + OUT parameter)");
    System.out.print("Enter choice: ");
    int choice = Integer.parseInt(sc.nextLine());

    switch (choice) {

        // IN parameter — pass empno, get salary
        case 1:
            CallableStatement csSal = con.prepareCall("{call get_salary(?, ?)}");

            System.out.print("Enter Emp No: ");
            int eno = Integer.parseInt(sc.nextLine());

            csSal.setInt(1, eno);                    // IN  parameter
            csSal.registerOutParameter(2, Types.FLOAT);    // OUT parameter

            csSal.execute();

            float salary = csSal.getFloat(2);
            System.out.println("Salary of Emp " + eno + " : " + salary);
            csSal.close();
            break;

        // OUT parameter — get total employee count
        case 2:
```

```
            CallableStatement cCount = con.prepareCall("{call get_count(?)}");

            cCount.registerOutParameter(1, Types.INTEGER);  // OUT parameter

            cCount.execute();

            int empCount = cCount.getInt(1);
            System.out.println("Total Employees: " + empCount);
            cCount.close();
            break;

        // IN + OUT parameter — raise salary by percentage
        case 3:
            CallableStatement cRaise = con.prepareCall("{call raise_salary(?, ?, ?)}");

            System.out.print("Enter Emp No     : ");
            int raiseEno = Integer.parseInt(sc.nextLine());
            System.out.print("Enter Raise %     : ");
            float percent = Float.parseFloat(sc.nextLine());

            cRaise.setInt  (1, raiseEno);              // IN  parameter
            cRaise.setFloat(2, percent);               // IN  parameter
            cRaise.registerOutParameter(3, Types.FLOAT);   // OUT parameter

            cRaise.execute();

            float newSalary = cRaise.getFloat(3);
            System.out.println("New Salary of Emp " + raiseEno + " : " + newSalary);
            cRaise.close();
            break;

        default:
            System.out.println("Invalid choice.");
    }
}

// — main ————————————————————————————————————————————
public static void main(String[] args) {

    try {
        // Step 1: Load Oracle JDBC Driver
```

```
        Class.forName("oracle.jdbc.OracleDriver");

        // Step 2: Establish connection
        Connection con = DriverManager.getConnection(
           "jdbc:oracle:thin:@localhost:1521:orcl", "mca", "mca");
        System.out.println("Database connected successfully.");

        Scanner sc = new Scanner(System.in);
        int choice;

        do {
           System.out.println("\n--- MAIN MENU ---");
           System.out.println("1. Prepared Statement");
           System.out.println("2. Callable Statement");
           System.out.println("3. Exit");
           System.out.print("Enter choice: ");
           choice = Integer.parseInt(sc.nextLine());

           switch (choice) {
              case 1: preparedStatementDemo(con); break;
              case 2: callableStatementDemo(con); break;
              case 3: System.out.println("Exiting..."); break;
              default: System.out.println("Invalid choice.");
           }

        } while (choice != 3);

        con.close();
        sc.close();
        System.out.println("Connection closed.");

     } catch (ClassNotFoundException e) {
        System.out.println("Driver not found : " + e.getMessage());
     } catch (SQLException e) {
        System.out.println("SQL error       : " + e.getMessage());
     }
  }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac -cp .;ojdbc17.jar PG52CallablePrepared.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java -cp .;ojdbc17.jar PG52CallablePrepared
Database connected successfully.

--- MAIN MENU ---
1. Prepared Statement
2. Callable Statement
3. Exit
Enter choice: 1

========== PREPARED STATEMENT ==========
1. Insert Employee
2. Update Salary
3. Delete Employee
4. Search Employee
Enter choice: 1
Enter Emp No   : 105
Enter Emp Name : Rena
Enter Emp Sal  : 50000
Record inserted successfully!

--- MAIN MENU ---
1. Prepared Statement
2. Callable Statement
3. Exit
Enter choice: 2

========== CALLABLE STATEMENT ==========
1. Get Employee Salary  (IN parameter)
2. Get Employee Count   (OUT parameter)
3. Raise Salary         (IN + OUT parameter)
Enter choice: 1
Enter Emp No: 101
Salary of Emp 101 : 50000.0

--- MAIN MENU ---
1. Prepared Statement
2. Callable Statement
3. Exit
Enter choice: 2

========== CALLABLE STATEMENT ==========
1. Get Employee Salary  (IN parameter)
2. Get Employee Count   (OUT parameter)
3. Raise Salary         (IN + OUT parameter)
Enter choice: 2
Total Employees: 6

--- MAIN MENU ---
1. Prepared Statement
2. Callable Statement
3. Exit
Enter choice: 2

========== CALLABLE STATEMENT ==========
1. Get Employee Salary  (IN parameter)
2. Get Employee Count   (OUT parameter)
3. Raise Salary         (IN + OUT parameter)
Enter choice: 3
Enter Emp No       : 102
Enter Raise %      : 10
New Salary of Emp 102 : 44000.0

--- MAIN MENU ---
1. Prepared Statement
2. Callable Statement
3. Exit
Enter choice: 3
Exiting...
Connection closed.

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>
```

Program No: 53	**Date:** 02-02-2026

Program Title: Program to implement
a. Transaction.
b. Batch processing

```
/*
Transaction Batch Processing
@ Ashbi Saju
Roll no: 16
Date: 02-02-2026
*/

import java.sql.*;
import java.util.Scanner;

public class PG53TransactionBatch {

   // ── a. TRANSACTION ──────────────────────────────────────────────
   static void transactionDemo(Connection con) throws SQLException {
      Scanner sc = new Scanner(System.in);

      System.out.println("\n========== TRANSACTION DEMO ==========");

      try {
         // Disable auto commit
         con.setAutoCommit(false);
         System.out.println("Auto commit disabled.");

         Statement st = con.createStatement();

         System.out.print("Enter number of records to insert: ");
         int n = Integer.parseInt(sc.nextLine());

         for (int i = 1; i <= n; i++) {
            System.out.println("\nEnter details for record " + i + ":");
            System.out.print("Enter Emp No   : ");
            String no = sc.nextLine();

            System.out.print("Enter Emp Name : ");
            String name = sc.nextLine();

            System.out.print("Enter Emp Sal  : ");
```

```
            String sal = sc.nextLine();

            String sql = "INSERT INTO Emp16 VALUES(" + no + ",'" + name + "'," + sal + ")";
            st.executeUpdate(sql);
            System.out.println("Record " + i + " inserted.");
        }

        // Ask user to commit or rollback
        System.out.print("\nDo you want to COMMIT? (y/n): ");
        char choice = sc.nextLine().charAt(0);

        if (choice == 'y' || choice == 'Y') {
            con.commit();
            System.out.println("Transaction COMMITTED successfully!");
        } else {
            con.rollback();
            System.out.println("Transaction ROLLED BACK. No changes saved.");
        }

        // Display records after transaction
        System.out.println("\n---------- Emp16 Table After Transaction ----------");
        System.out.printf("%-10s %-20s %-10s%n", "Emp No", "Emp Name", "Salary");
        System.out.println("-----------------------------------------------");

        ResultSet rs = st.executeQuery("SELECT * FROM Emp16");
        while (rs.next()) {
            System.out.printf("%-10s %-20s %-10s%n",
                rs.getString(1),
                rs.getString(2),
                rs.getString(3));
        }
        System.out.println("-----------------------------------------------");

        rs.close();
        st.close();

    } catch (SQLException e) {
        // Rollback on error
        con.rollback();
        System.out.println("Error occurred. Transaction ROLLED BACK.");
        System.out.println("SQL error : " + e.getMessage());
```

```
    } finally {
        // Re-enable auto commit
        con.setAutoCommit(true);
        System.out.println("Auto commit re-enabled.");
    }
}

// — b. BATCH PROCESSING ————————————————————————————————
static void batchDemo(Connection con) throws SQLException {
    Scanner sc = new Scanner(System.in);

    System.out.println("\n========== BATCH PROCESSING DEMO ==========");

    try {
        con.setAutoCommit(false);
        System.out.println("Auto commit disabled.");

        Statement st = con.createStatement();

        System.out.print("Enter number of records to batch insert: ");
        int n = Integer.parseInt(sc.nextLine());

        for (int i = 1; i <= n; i++) {
            System.out.println("\nEnter details for record " + i + ":");
            System.out.print("Enter Emp No   : ");
            String no = sc.nextLine();

            System.out.print("Enter Emp Name : ");
            String name = sc.nextLine();

            System.out.print("Enter Emp Sal  : ");
            String sal = sc.nextLine();

            String sql = "INSERT INTO Emp16 VALUES(" + no + ",'" + name + "'," + sal + ")";

            // Add to batch instead of executing immediately
            st.addBatch(sql);
            System.out.println("Record " + i + " added to batch.");
        }

        System.out.println("\nExecuting batch...");
```

```
    // Execute all batched statements at once
    int[] results = st.executeBatch();

    // Commit after batch
    con.commit();

    System.out.println("Batch executed successfully!");
    System.out.println("Total records inserted: " + results.length);

    for (int i = 0; i < results.length; i++) {
       System.out.println("Record " + (i + 1) + " : " +
          (results[i] > 0 ? "Inserted" : "Failed"));
    }

    // Display records after batch
    System.out.println("\n---------- Emp16 Table After Batch ----------");
    System.out.printf("%-10s %-20s %-10s%n", "Emp No", "Emp Name", "Salary");
    System.out.println("-----------------------------------------------");

    ResultSet rs = st.executeQuery("SELECT * FROM Emp16");
    while (rs.next()) {
       System.out.printf("%-10s %-20s %-10s%n",
          rs.getString(1),
          rs.getString(2),
          rs.getString(3));
    }
    System.out.println("-----------------------------------------------");

    rs.close();
    st.close();

  } catch (BatchUpdateException e) {
    con.rollback();
    System.out.println("Batch error. Transaction ROLLED BACK.");
    System.out.println("Batch error : " + e.getMessage());
  } catch (SQLException e) {
    con.rollback();
    System.out.println("Error occurred. Transaction ROLLED BACK.");
    System.out.println("SQL error   : " + e.getMessage());
  } finally {
```

```
        con.setAutoCommit(true);
        System.out.println("Auto commit re-enabled.");
    }
}

// ── main ──────────────────────────────────────────────────────────────
public static void main(String[] args) {

    try {
        // Step 1: Load Oracle JDBC Driver
        Class.forName("oracle.jdbc.OracleDriver");

        // Step 2: Establish connection
        Connection con = DriverManager.getConnection(
            "jdbc:oracle:thin:@localhost:1521:orcl", "mca", "mca");
        System.out.println("Database connected successfully.");

        Scanner sc = new Scanner(System.in);
        int choice;

        do {
            System.out.println("\n--- MAIN MENU ---");
            System.out.println("1. Transaction");
            System.out.println("2. Batch Processing");
            System.out.println("3. Exit");
            System.out.print("Enter choice: ");
            choice = Integer.parseInt(sc.nextLine());

            switch (choice) {
                case 1: transactionDemo(con); break;
                case 2: batchDemo(con);       break;
                case 3: System.out.println("Exiting..."); break;
                default: System.out.println("Invalid choice.");
            }

        } while (choice != 3);

        con.close();
        sc.close();
        System.out.println("Connection closed.");
```

```
        } catch (ClassNotFoundException e) {
            System.out.println("Driver not found : " + e.getMessage());
        } catch (SQLException e) {
            System.out.println("SQL error       : " + e.getMessage());
        }
    }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac -cp .;ojdbc17.jar PG53TransactionBatch.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java -cp .;ojdbc17.jar PG53TransactionBatch
Database connected successfully.

--- MAIN MENU ---
1. Transaction
2. Batch Processing
3. Exit
Enter choice: 1

========== TRANSACTION DEMO ==========
Auto commit disabled.
Enter number of records to insert: 2

Enter details for record 1:
Enter Emp No   : 103
Enter Emp Name : Kishore
Enter Emp Sal  : 45000
Record 1 inserted.

Enter details for record 2:
Enter Emp No   : 104
Enter Emp Name : Jasin
Enter Emp Sal  : 50000
Record 2 inserted.

Do you want to COMMIT? (y/n): y
Transaction COMMITTED successfully!

---------- Emp16 Table After Transaction ----------
Emp No     Emp Name             Salary
-------------------------------------------------
101        Ashbi                50000
102        Jees                 44000
103        Kishore              45000
104        Jasin                50000
-------------------------------------------------
Auto commit re-enabled.
```

```
--- MAIN MENU ---
1. Transaction
2. Batch Processing
3. Exit
Enter choice: 2

========== BATCH PROCESSING DEMO ==========
Auto commit disabled.
Enter number of records to batch insert: 3

Enter details for record 1:
Enter Emp No   : 105
Enter Emp Name : Rena
Enter Emp Sal  : 40000
Record 1 added to batch.
```

```
Enter details for record 2:
Enter Emp No   : 106
Enter Emp Name : Diya
Enter Emp Sal  : 60000
Record 2 added to batch.

Enter details for record 3:
Enter Emp No   : 107
Enter Emp Name : Merlin
Enter Emp Sal  : 55000
Record 3 added to batch.

Executing batch...
Batch executed successfully!
Total records inserted: 3
Record 1 : Inserted
Record 2 : Inserted
Record 3 : Inserted

---------- Emp16 Table After Batch ----------
Emp No     Emp Name             Salary
-------------------------------------------------
101        Ashbi                50000
102        Jees                 44000
106        Diya                 60000
107        Merlin               55000
103        Kishore              45000
104        Jasin                50000
105        Rena                 40000
-------------------------------------------------
Auto commit re-enabled.

--- MAIN MENU ---
1. Transaction
2. Batch Processing
3. Exit
Enter choice: 3
Exiting...
Connection closed.
```

Program No: 54	**Date:** 07-02-2026

Program Title: Program to implement Serialization and De-Serialization.

```
/*
Object Serialization Demo
@ Ashbi Saju
Roll no: 16
Date: 07-02-2026
*/
```

Serialization.java

```
import java.io.*;
import java.util.Scanner;

class Employee implements Serializable {
   private static final long serialVersionUID = 1L;
   int    eno;
   String ename;
   float  esal;
   String edept;

   public Employee(int eno, String ename, float esal, String edept) {
      this.eno   = eno;
      this.ename = ename;
      this.esal  = esal;
      this.edept = edept;
   }
}

public class PG54Serialization {

   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.println("Enter Employee Details:");

      System.out.print("Emp No   : ");
      int eno = Integer.parseInt(sc.nextLine());

      System.out.print("Emp Name : ");
      String ename = sc.nextLine();

      System.out.print("Emp Sal  : ");
      float esal = Float.parseFloat(sc.nextLine());
```

```
        System.out.print("Emp Dept : ");
        String edept = sc.nextLine();
        Employee emp = new Employee(eno, ename, esal, edept);

        try {
            FileOutputStream   fos = new FileOutputStream("employee.ser");
            ObjectOutputStream oos = new ObjectOutputStream(fos);
            oos.writeObject(emp);
            oos.close();
            fos.close();
            sc.close();

            System.out.println("\nObject serialized successfully!");
            System.out.println("----------------------------------------");
            System.out.println("Emp No   : " + emp.eno);
            System.out.println("Emp Name : " + emp.ename);
            System.out.println("Emp Sal  : " + emp.esal);
            System.out.println("Emp Dept : " + emp.edept);
            System.out.println("----------------------------------------");
            System.out.println("Saved to : employee.ser");

        } catch (IOException e) {
            System.out.println("Serialization error: " + e.getMessage());
        }
    }
}
```

Deserialization.java

```
import java.io.*;

class Employee implements Serializable {
    private static final long serialVersionUID = 1L;
    int    eno;
    String ename;
    float  esal;
    String edept;
}

public class PG54Deserialization {
    public static void main(String[] args) {
        try {
            FileInputStream   fis = new FileInputStream("employee.ser");
            ObjectInputStream ois = new ObjectInputStream(fis);

            Employee emp = (Employee) ois.readObject();
```

```
        ois.close();
        fis.close();

        System.out.println("Object de-serialized successfully!");
        System.out.println("----------------------------------------");
        System.out.println("Emp No   : " + emp.eno);
        System.out.println("Emp Name : " + emp.ename);
        System.out.println("Emp Sal  : " + emp.esal);
        System.out.println("Emp Dept : " + emp.edept);
        System.out.println("----------------------------------------");

      } catch (FileNotFoundException e) {
        System.out.println("File not found. Run Serialization first.");
      } catch (IOException | ClassNotFoundException e) {
        System.out.println("De-serialization error: " + e.getMessage());
      }
    }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG54Serialization.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG54Serialization
Enter Employee Details:
Emp No   : 101
Emp Name : Ashbi
Emp Sal  : 50000
Emp Dept : MCA

Object serialized successfully!
----------------------------------------
Emp No   : 101
Emp Name : Ashbi
Emp Sal  : 50000.0
Emp Dept : MCA
----------------------------------------
Saved to : employee.ser

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG54Deserialization.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG54Deserialization
Object de-serialized successfully!
----------------------------------------
Emp No   : 101
Emp Name : Ashbi
Emp Sal  : 50000.0
Emp Dept : MCA
----------------------------------------
```

Program No: 55	**Date:** 07-02-2026

Program Title: Program to implement IS A Serialization and De-Serialization.

```
/*
IS-A Serialization
@ Ashbi Saju
Roll no: 16
Date: 07-02-2026
*/
```

Serialization.java

```
import java.io.*;

class Person implements Serializable {
   int    id;
   String name;

   Person(int id, String name) {
      this.id   = id;
      this.name = name;
   }
}

class Student extends Person {
   String course;
   float  marks;

   public Student(int id, String name, String course, float marks) {
      super(id, name);
      this.course = course;
      this.marks  = marks;
   }
}

class Teacher extends Person {
   String subject;
   float  salary;

   public Teacher(int id, String name, String subject, float salary) {
      super(id, name);
      this.subject = subject;
      this.salary  = salary;
   }
}
```

```
public class PG55ISASerialization {
   public static void main(String args[]) {
      try {
         // Create Student object
         Student s = new Student(101, "Ashbi", "MCA", 88.5f);

         // Serialize Student
         FileOutputStream   fout1 = new FileOutputStream("student.txt");
         ObjectOutputStream out1  = new ObjectOutputStream(fout1);
         out1.writeObject(s);
         out1.flush();
         out1.close();
         System.out.println("Student serialized successfully.");
         System.out.println(s.id + " " + s.name + " " + s.course + " " + s.marks);

         // Create Teacher object
         Teacher t = new Teacher(501, "Jees", "Java", 75000f);

         // Serialize Teacher
         FileOutputStream   fout2 = new FileOutputStream("teacher.txt");
         ObjectOutputStream out2  = new ObjectOutputStream(fout2);
         out2.writeObject(t);
         out2.flush();
         out2.close();
         System.out.println("\nTeacher serialized successfully.");
         System.out.println(t.id + " " + t.name + " " + t.subject + " " + t.salary);

      } catch (Exception e) {
         System.out.println(e);
      }
   }
}
```

Dederialization.java

```
import java.io.*;

class Person implements Serializable {
   int    id;
   String name;
   Person(int id, String name) {
      this.id   = id;
      this.name = name;
   }
}
```

```
class Student extends Person {
   String course;
   float  marks;

   public Student(int id, String name, String course, float marks) {
      super(id, name);
      this.course = course;
      this.marks  = marks;
   }
}

class Teacher extends Person {
   String subject;
   float  salary;

   public Teacher(int id, String name, String subject, float salary) {
      super(id, name);
      this.subject = subject;
      this.salary  = salary;
   }
}

public class PG55ISADeserialization {
   public static void main(String args[]) {
      try {
         // Deserialize Student
         ObjectInputStream in1 = new ObjectInputStream(
            new FileInputStream("student.txt"));
         Student s = (Student) in1.readObject();
         in1.close();
         System.out.println("Student de-serialized successfully.");
         System.out.println(s.id + " " + s.name + " " + s.course + " " + s.marks);

         // Deserialize Teacher
         ObjectInputStream in2 = new ObjectInputStream(
            new FileInputStream("teacher.txt"));
         Teacher t = (Teacher) in2.readObject();
         in2.close();
         System.out.println("\nTeacher de-serialized successfully.");
         System.out.println(t.id + " " + t.name + " " + t.subject + " " + t.salary);

      } catch (Exception e) {
         System.out.println(e);
      }
   }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG55ISASerialization.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG55ISASerialization
Student serialized successfully.
101 Alice MCA 88.5

Teacher serialized successfully.
501 John Java 75000.0

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG55ISASerialization.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG55ISASerialization
Student serialized successfully.
101 Ashbi MCA 88.5

Teacher serialized successfully.
501 Jees Java 75000.0

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG55ISADeserialization.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG55ISADeserialization
Student de-serialized successfully.
101 Ashbi MCA 88.5

Teacher de-serialized successfully.
501 Jees Java 75000.0
```

Program No: 56	**Date:** 07-02-2026

Program Title: Program to implement HAS A Serialization and De-Serialization.

```
/*
HAS-A Serialization
@ Ashbi Saju
Roll no: 16
Date: 07-02-2026
*/
```

Serialization.java

```
import java.io.*;

class Address implements Serializable {
   String street;
   String city;
   String state;
   int    pincode;

   public Address(String street, String city, String state, int pincode) {
      this.street  = street;
      this.city    = city;
      this.state   = state;
      this.pincode = pincode;
   }
}

class Department implements Serializable {
   int    dno;
   String dname;
   String dloc;

   public Department(int dno, String dname, String dloc) {
      this.dno   = dno;
      this.dname = dname;
      this.dloc  = dloc;
   }
}

class Employee implements Serializable {
   int       eno;
   String    ename;
   float     esal;
   Address   address;    // HAS-A
```

```
   Department department;  // HAS-A

   public Employee(int eno, String ename, float esal,
               String street, String city, String state, int pincode,
               int dno, String dname, String dloc) {
      this.eno       = eno;
      this.ename     = ename;
      this.esal      = esal;
      address        = new Address(street, city, state, pincode);
      department     = new Department(dno, dname, dloc);
   }
}

public class PG56HASASerialization {
   public static void main(String args[]) {
      try {
         // Create Employee object
         Employee emp = new Employee(
            101, "Ashbi", 50000f,
            "Karukutty", "Angamaly", "Kerala", 682001,
            10, "MCA", "Block B"
         );

         // Serialize Employee
         FileOutputStream   fout = new FileOutputStream("employee.txt");
         ObjectOutputStream out  = new ObjectOutputStream(fout);
         out.writeObject(emp);
         out.flush();
         out.close();

         System.out.println("success");
         System.out.println(emp.eno + " " + emp.ename + " " + emp.esal);
         System.out.println(emp.address.street + " " + emp.address.city + " " +
                     emp.address.state  + " " + emp.address.pincode);
         System.out.println(emp.department.dno + " " + emp.department.dname + " " +
                     emp.department.dloc);

      } catch (Exception e) {
         System.out.println(e);
      }
   }
}
```

Deserialization.java

```
import java.io.*;

class Address implements Serializable {
   String street;
   String city;
   String state;
   int    pincode;

   public Address(String street, String city, String state, int pincode) {
      this.street  = street;
      this.city    = city;
      this.state   = state;
      this.pincode = pincode;
   }
}

class Department implements Serializable {
   int    dno;
   String dname;
   String dloc;

   public Department(int dno, String dname, String dloc) {
      this.dno   = dno;
      this.dname = dname;
      this.dloc  = dloc;
   }
}

class Employee implements Serializable {
   int       eno;
   String    ename;
   float     esal;
   Address   address;    // HAS-A
   Department department;  // HAS-A

   public Employee(int eno, String ename, float esal,
                   String street, String city, String state, int pincode,
                   int dno, String dname, String dloc) {
      this.eno       = eno;
```

```
        this.ename     = ename;
        this.esal      = esal;
        address        = new Address(street, city, state, pincode);
        department     = new Department(dno, dname, dloc);
    }
}

public class PG56HASADeserialization {
    public static void main(String args[]) {
        try {
            // Deserialize Employee
            ObjectInputStream in = new ObjectInputStream(
                new FileInputStream("employee.txt"));
            Employee emp = (Employee) in.readObject();
            in.close();
            System.out.println("success");
            System.out.println(emp.eno + " " + emp.ename + " " + emp.esal);
            System.out.println(emp.address.street + " " + emp.address.city + " " +
                               emp.address.state  + " " + emp.address.pincode);
            System.out.println(emp.department.dno + " " + emp.department.dname + " " +
                               emp.department.dloc);

        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG56HASASerialization.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG56HASASerialization
success
101 Ashbi 50000.0
Karukutty Angamaly Kerala 682001
10 MCA Block B

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG56HASADeserialization.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG56HASADeserialization
success
101 Ashbi 50000.0
Karukutty Angamaly Kerala 682001
10 MCA Block B
```

Program No: 57	**Date:** 09-02-2026

Program Title: Program to De-Serialize selected attributes of an object using Transient keyword.

```
/*
Transient Deserialization
@ Ashbi Saju
Roll no: 16
Date: 09-02-2026
*/
```

Serialization.java

```
import java.io.*;

class Student implements Serializable {
   int    id;
   String name;
   transient int age;  // will NOT be serialized

   public Student(int id, String name, int age) {
      this.id   = id;
      this.name = name;
      this.age  = age;
   }
}

public class PG57TransientSerialization {
   public static void main(String args[]) {
      try {
         Student s = new Student(101, "Ashbi", 21);

         FileOutputStream   fout = new FileOutputStream("student.txt");
         ObjectOutputStream out  = new ObjectOutputStream(fout);
         out.writeObject(s);
         out.flush();
         out.close();

         System.out.println("success");
         System.out.println("Before Serialization:");
         System.out.println("ID   : " + s.id);
         System.out.println("Name : " + s.name);
         System.out.println("Age  : " + s.age);

      } catch (Exception e) {
         System.out.println(e);
```

```
    }
  }
}
```

Deserialization.java

```
import java.io.*;

class Student implements Serializable {
   int    id;
   String name;
   transient int age;  // will NOT be serialized

   public Student(int id, String name, int age) {
      this.id   = id;
      this.name = name;
      this.age  = age;
   }
}

public class PG57TransientDeserialization {
   public static void main(String args[]) {
      try {
         ObjectInputStream in = new ObjectInputStream(
            new FileInputStream("student.txt"));
         Student s = (Student) in.readObject();
         in.close();

         System.out.println("success");
         System.out.println("After Deserialization:");
         System.out.println("ID   : " + s.id);
         System.out.println("Name : " + s.name);
         System.out.println("Age  : " + s.age + " (transient - not serialized)");

      } catch (Exception e) {
         System.out.println(e);
      }
   }
}
```

Output

```
C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG57TransientSerialization.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG57TransientSerialization
success
Before Serialization:
ID   : 101
Name : Ashbi
Age  : 21

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>javac PG57TransientDeserialization.java

C:\Users\ASUS\Documents\JAVA\LAB CYCLE>java PG57TransientDeserialization
success
After Deserialization:
ID   : 101
Name : Ashbi
Age  : 0 (transient - not serialized)
```



Program No: 58	**Date:** 09-02-2026

Program Title: Find the maximum of three numbers using AWT.

```
/*
AWT Maximum Finder
@ Ashbi Saju
Roll no: 16
Date: 09-02-2026
*/

import java.awt.*;
import java.awt.event.*;

public class PG58MaxOfThree extends Frame implements ActionListener {

   // Labels
   Label l1, l2, l3, lResult;

   // TextFields
   TextField tf1, tf2, tf3, tfResult;

   // Buttons
   Button btnFind, btnClear, btnExit;

   public PG58MaxOfThree() {

      // ── Frame settings ──────────────────────────────────────
      setTitle("Maximum of Three Numbers");
      setSize(400, 300);
      setLayout(null);
      setVisible(true);

      // ── Labels ──────────────────────────────────────────────
      l1 = new Label("Enter Number 1 :");
      l1.setBounds(50, 50, 120, 25);
      add(l1);

      l2 = new Label("Enter Number 2 :");
      l2.setBounds(50, 90, 120, 25);
      add(l2);
```

```
l3 = new Label("Enter Number 3 :");
l3.setBounds(50, 130, 120, 25);
add(l3);

lResult = new Label("Maximum      :");
lResult.setBounds(50, 190, 120, 25);
add(lResult);

// ── TextFields ──────────────────────────────────────────────
tf1 = new TextField();
tf1.setBounds(180, 50, 150, 25);
add(tf1);

tf2 = new TextField();
tf2.setBounds(180, 90, 150, 25);
add(tf2);

tf3 = new TextField();
tf3.setBounds(180, 130, 150, 25);
add(tf3);

tfResult = new TextField();
tfResult.setBounds(180, 190, 150, 25);
tfResult.setEditable(false);
tfResult.setBackground(Color.lightGray);
add(tfResult);

// ── Buttons ─────────────────────────────────────────────────
btnFind = new Button("Find Maximum");
btnFind.setBounds(50, 240, 110, 30);
add(btnFind);

btnClear = new Button("Clear");
btnClear.setBounds(170, 240, 80, 30);
add(btnClear);

btnExit = new Button("Exit");
btnExit.setBounds(260, 240, 80, 30);
add(btnExit);
```

```
    // ── Action Listeners ──────────────────────────────────────────
    btnFind.addActionListener(this);
    btnClear.addActionListener(this);
    btnExit.addActionListener(this);

    // ── Window Close ──────────────────────────────────────────────
    addWindowListener(new WindowAdapter() {
        public void windowClosing(WindowEvent e) {
            System.exit(0);
        }
    });
}

// ── Action Performed ──────────────────────────────────────────────
public void actionPerformed(ActionEvent e) {

    if (e.getSource() == btnFind) {
        try {
            int n1 = Integer.parseInt(tf1.getText());
            int n2 = Integer.parseInt(tf2.getText());
            int n3 = Integer.parseInt(tf3.getText());

            int max = n1;
            if (n2 > max) max = n2;
            if (n3 > max) max = n3;

            tfResult.setText(String.valueOf(max));

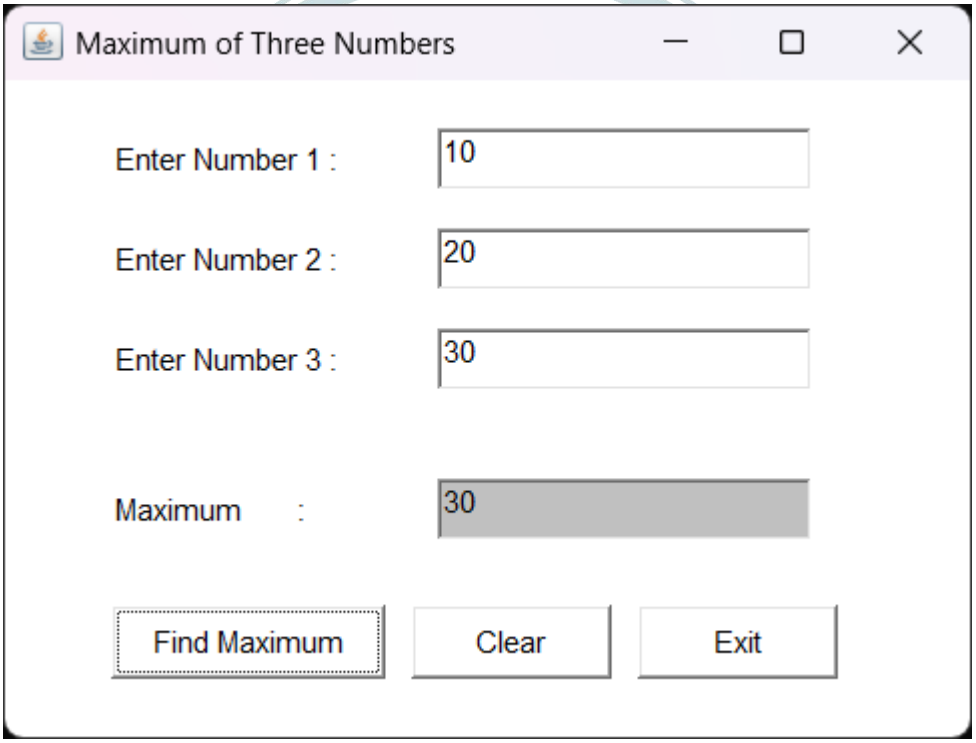
        } catch (NumberFormatException ex) {
            tfResult.setText("Invalid input!");
        }
    }

    if (e.getSource() == btnClear) {
        tf1.setText("");
        tf2.setText("");
        tf3.setText("");
        tfResult.setText("");
        tf1.requestFocus();
    }
```

```
        if (e.getSource() == btnExit) {
            System.exit(0);
        }
    }

    // — main ————————————————————————————————————————————————————
    public static void main(String[] args) {
        new PG58MaxOfThree();
    }
}
```

Output



Program No: 59	**Date:** 09-02-2026

Program Title: Implement all the functionality of LIST control.

```
/*
AWT List Control
@ Ashbi Saju
Roll no: 16
Date: 09-02-2026
*/

import java.awt.*;
import java.awt.event.*;

public class PG59ListDemo extends Frame implements ActionListener, ItemListener {

   // Lists
   List listLeft, listRight;

   // Labels
   Label lLeft, lRight, lSelected, lStatus;

   // TextFields
   TextField tfItem, tfSelected;

   // Buttons
   Button btnAdd, btnDelete, btnMoveRight, btnMoveLeft;
   Button btnMoveAll, btnClear, btnExit;
   Button btnSelectAll, btnDeselect;

   public PG59ListDemo() {

      setTitle("AWT List Control - All Functionalities");
      setSize(620, 500);
      setLayout(null);
      setVisible(true);

      // ── Labels ──────────────────────────────────────────────
      lLeft = new Label("Available Items:");
      lLeft.setBounds(30, 20, 120, 25);
      add(lLeft);
```

```
lRight = new Label("Selected Items:");
lRight.setBounds(420, 20, 120, 25);
add(lRight);

lSelected = new Label("Item selected:");
lSelected.setBounds(30, 370, 100, 25);
add(lSelected);

lStatus = new Label("Status: Ready");
lStatus.setBounds(30, 440, 550, 25);
lStatus.setForeground(Color.blue);
add(lStatus);

// — Left List (multi-select) ————————————————————————
listLeft = new List(10, true);
listLeft.setBounds(30, 50, 180, 200);
listLeft.add("Alice");
listLeft.add("Bob");
listLeft.add("Charlie");
listLeft.add("David");
listLeft.add("Eve");
listLeft.add("Frank");
listLeft.add("Grace");
listLeft.add("Henry");
add(listLeft);

// — Right List (multi-select) ———————————————————————
listRight = new List(10, true);
listRight.setBounds(420, 50, 180, 200);
add(listRight);

// — TextField for adding new item ——————————————————————
Label lAdd = new Label("New Item:");
lAdd.setBounds(30, 270, 70, 25);
add(lAdd);

tfItem = new TextField();
tfItem.setBounds(110, 270, 100, 25);
add(tfItem);
```

```
// ── TextField to show selected item ──────────────────────────
tfSelected = new TextField();
tfSelected.setBounds(140, 370, 430, 25);
tfSelected.setEditable(false);
tfSelected.setBackground(Color.lightGray);
add(tfSelected);

// ── Middle Buttons (transfer) ────────────────────────────────
btnMoveRight = new Button("  >>  ");
btnMoveRight.setBounds(240, 80, 70, 30);
add(btnMoveRight);

btnMoveLeft = new Button("  <<  ");
btnMoveLeft.setBounds(240, 120, 70, 30);
add(btnMoveLeft);

btnMoveAll = new Button(" >> All");
btnMoveAll.setBounds(240, 160, 70, 30);
add(btnMoveAll);

Button btnMoveAllBack = new Button("<< All ");
btnMoveAllBack.setBounds(240, 200, 70, 30);
add(btnMoveAllBack);

// ── Bottom Buttons ──────────────────────────────────────────
btnAdd = new Button("Add");
btnAdd.setBounds(30, 310, 70, 30);
add(btnAdd);

btnDelete = new Button("Delete");
btnDelete.setBounds(110, 310, 70, 30);
add(btnDelete);

btnSelectAll = new Button("Select All");
btnSelectAll.setBounds(190, 310, 90, 30);
add(btnSelectAll);

btnDeselect = new Button("Deselect");
btnDeselect.setBounds(290, 310, 80, 30);
add(btnDeselect);
```

```
    btnClear = new Button("Clear All");
    btnClear.setBounds(380, 310, 80, 30);
    add(btnClear);

    btnExit = new Button("Exit");
    btnExit.setBounds(470, 310, 70, 30);
    add(btnExit);

    // ── Action Listeners ──────────────────────────────────────
    btnAdd.addActionListener(this);
    btnDelete.addActionListener(this);
    btnMoveRight.addActionListener(this);
    btnMoveLeft.addActionListener(this);
    btnMoveAll.addActionListener(this);
    btnMoveAllBack.addActionListener(this);
    btnSelectAll.addActionListener(this);
    btnDeselect.addActionListener(this);
    btnClear.addActionListener(this);
    btnExit.addActionListener(this);

    // ── Item Listeners ────────────────────────────────────────
    listLeft.addItemListener(this);
    listRight.addItemListener(this);

    // ── Window Close ──────────────────────────────────────────
    addWindowListener(new WindowAdapter() {
        public void windowClosing(WindowEvent e) {
            System.exit(0);
        }
    });
}

// ── Action Performed ──────────────────────────────────────────
public void actionPerformed(ActionEvent e) {

    // Add new item to left list
    if (e.getSource() == btnAdd) {
        String item = tfItem.getText().trim();
        if (!item.isEmpty()) {
            listLeft.add(item);
            tfItem.setText("");
```

```
            lStatus.setText("Status: Added \"" + item + "\" to list.");
        } else {
            lStatus.setText("Status: Please enter an item to add.");
        }
    }

    // Delete selected item from left list
    if (e.getSource() == btnDelete) {
        String[] selected = listLeft.getSelectedItems();
        if (selected.length > 0) {
            for (String item : selected) {
                listLeft.remove(item);
            }
            lStatus.setText("Status: Deleted " + selected.length + " item(s).");
        } else {
            lStatus.setText("Status: Please select item(s) to delete.");
        }
    }

    // Move selected items from left to right
    if (e.getSource() == btnMoveRight) {
        String[] selected = listLeft.getSelectedItems();
        if (selected.length > 0) {
            for (String item : selected) {
                listRight.add(item);
                listLeft.remove(item);
            }
            lStatus.setText("Status: Moved " + selected.length + " item(s) to right.");
        } else {
            lStatus.setText("Status: Please select item(s) to move.");
        }
    }

    // Move selected items from right to left
    if (e.getSource() == btnMoveLeft) {
        String[] selected = listRight.getSelectedItems();
        if (selected.length > 0) {
            for (String item : selected) {
                listLeft.add(item);
                listRight.remove(item);
            }
```

```
            lStatus.setText("Status: Moved " + selected.length + " item(s) to left.");
        } else {
            lStatus.setText("Status: Please select item(s) to move back.");
        }
    }

    // Move all items from left to right
    if (e.getSource() == btnMoveAll) {
        String[] items = listLeft.getItems();
        if (items.length > 0) {
            for (String item : items) {
                listRight.add(item);
            }
            listLeft.removeAll();
            lStatus.setText("Status: Moved all " + items.length + " item(s) to right.");
        } else {
            lStatus.setText("Status: Left list is already empty.");
        }
    }

    // Move all items from right to left
    if (e.getActionCommand().equals("<< All ")) {
        String[] items = listRight.getItems();
        if (items.length > 0) {
            for (String item : items) {
                listLeft.add(item);
            }
            listRight.removeAll();
            lStatus.setText("Status: Moved all " + items.length + " item(s) to left.");
        } else {
            lStatus.setText("Status: Right list is already empty.");
        }
    }

    // Select all items in left list
    if (e.getSource() == btnSelectAll) {
        for (int i = 0; i < listLeft.getItemCount(); i++) {
            listLeft.select(i);
        }
        lStatus.setText("Status: All items selected.");
    }
```

```
        // Deselect all items in left list
        if (e.getSource() == btnDeselect) {
            for (int i = 0; i < listLeft.getItemCount(); i++) {
                listLeft.deselect(i);
            }
            tfSelected.setText("");
            lStatus.setText("Status: All items deselected.");
        }
        // Exit
        if (e.getSource() == btnExit) {
            System.exit(0);
        }
    }

    // — Item State Changed ————————————————————————————————————
    public void itemStateChanged(ItemEvent e) {
        String[] leftSelected  = listLeft.getSelectedItems();
        String[] rightSelected = listRight.getSelectedItems();

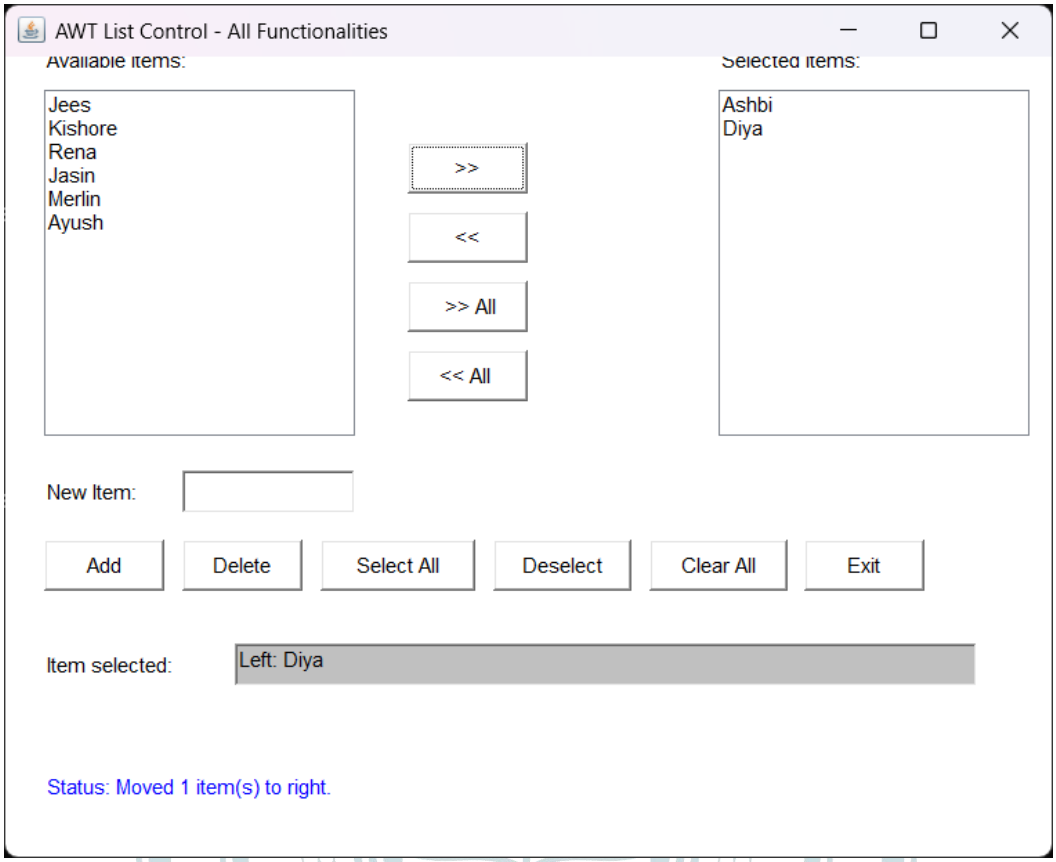
        StringBuilder sb = new StringBuilder();

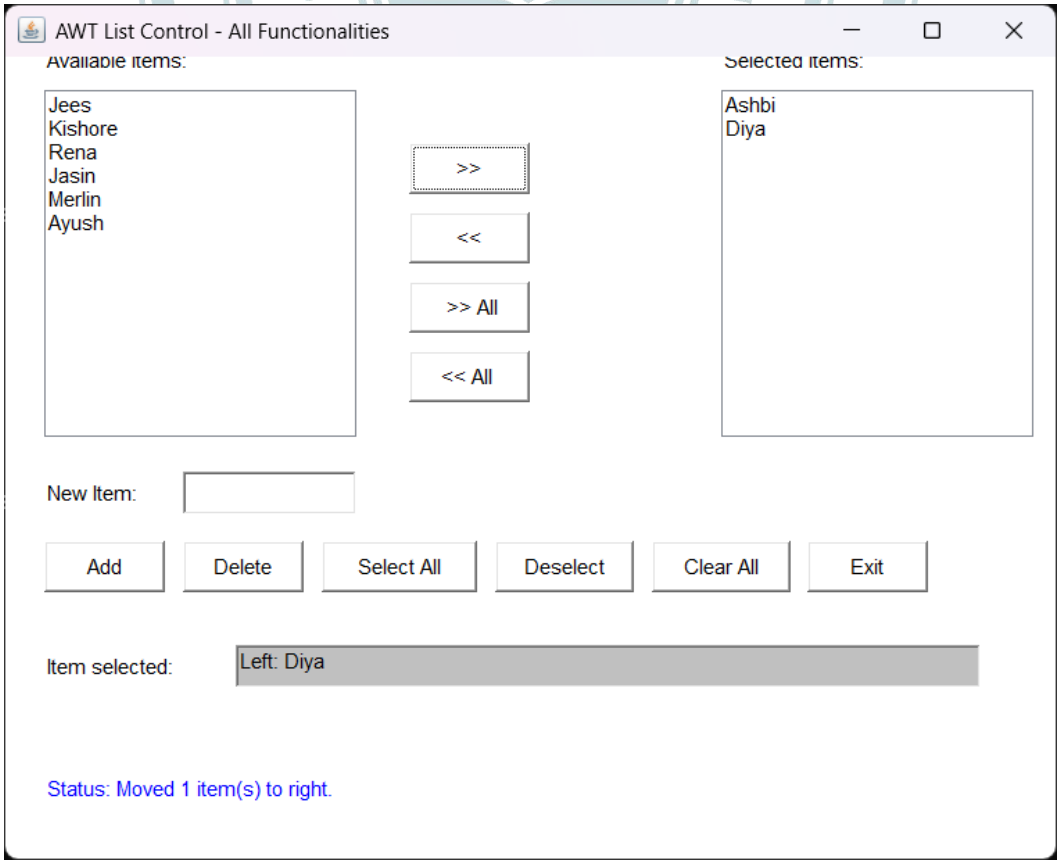
        if (leftSelected.length > 0) {
            sb.append("Left: ");
            for (String s : leftSelected) sb.append(s).append("  ");
        }
        if (rightSelected.length > 0) {
            sb.append("Right: ");
            for (String s : rightSelected) sb.append(s).append("  ");
        }

        tfSelected.setText(sb.toString());
        lStatus.setText("Status: " + (leftSelected.length + rightSelected.length)
            + " item(s) selected.");
    }

    // — main ——————————————————————————————————————————————————
    public static void main(String[] args) {
        new PG59ListDemo();
    }
}
```

Output


AWT List Control - All Functionalities
Available Items:
Selected Items:
Jees
Kishore
Rena
Jasin
Merlin
Ayush
Ashbi
Diya
>>
<<
>> All
<< All
New Item:
Add
Delete
Select All
Deselect
Clear All
Exit
Item selected:
Left: Diya
Status: Moved 1 item(s) to right.


AWT List Control - All Functionalities
Available Items:
Selected Items:
Jees
Kishore
Rena
Jasin
Merlin
Ayush
Ashbi
Diya
>>
<<
>> All
<< All
New Item:
Add
Delete
Select All
Deselect
Clear All
Exit
Item selected:
Left: Diya
Status: Moved 1 item(s) to right.

Program No: 60	**Date:** 16-02-2026

Program Title: Implement Check box and radio button in AWT.

```
/*
AWT Checkbox Radio
@ Ashbi Saju
Roll no: 16
Date: 16-02-2026
*/

import java.awt.*;
import java.awt.event.*;

public class PG60CheckboxRadioDemo extends Frame implements ItemListener, ActionListener {

   // ── Checkboxes ──────────────────────────────────────────────
   Checkbox chkJava, chkPython, chkCpp, chkHTML, chkSQL;

   // ── Radio Buttons (CheckboxGroup) ──────────────────────────
   CheckboxGroup genderGroup, courseGroup;
   Checkbox rbMale, rbFemale, rbOther;
   Checkbox rbUG, rbPG, rbPhD;

   // ── Labels ──────────────────────────────────────────────────
   Label lSkills, lGender, lCourse, lResult;

   // ── TextFields ──────────────────────────────────────────────
   TextField tfSkills, tfGender, tfCourse, tfResult;

   // ── Buttons ─────────────────────────────────────────────────
   Button btnSubmit, btnClear, btnExit;

   public PG60CheckboxRadioDemo() {

      setTitle("Checkbox and Radio Button Demo");
      setSize(520, 550);
      setLayout(null);
      setVisible(true);

      // ── Section 1: Checkboxes ──────────────────────────────
```

```
Label lTitle1 = new Label("Select Skills (Checkbox - Multiple Selection):");
lTitle1.setBounds(30, 20, 350, 25);
lTitle1.setForeground(Color.blue);
add(lTitle1);

chkJava = new Checkbox("Java");
chkJava.setBounds(30, 55, 80, 25);
add(chkJava);

chkPython = new Checkbox("Python");
chkPython.setBounds(120, 55, 80, 25);
add(chkPython);

chkCpp = new Checkbox("C++");
chkCpp.setBounds(210, 55, 80, 25);
add(chkCpp);

chkHTML = new Checkbox("HTML");
chkHTML.setBounds(300, 55, 80, 25);
add(chkHTML);

chkSQL = new Checkbox("SQL");
chkSQL.setBounds(390, 55, 80, 25);
add(chkSQL);

lSkills = new Label("Selected Skills:");
lSkills.setBounds(30, 95, 110, 25);
add(lSkills);

tfSkills = new TextField();
tfSkills.setBounds(150, 95, 330, 25);
tfSkills.setEditable(false);
tfSkills.setBackground(Color.lightGray);
add(tfSkills);

// ── Divider ──────────────────────────────────────────────────────
Label lLine1 = new Label("------------------------------------------------");
lLine1.setBounds(30, 130, 450, 20);
add(lLine1);

// ── Section 2: Radio Buttons — Gender ────────────────────────────
```

```
Label lTitle2 = new Label("Select Gender (Radio Button - Single Selection):");
lTitle2.setBounds(30, 155, 350, 25);
lTitle2.setForeground(Color.blue);
add(lTitle2);

genderGroup = new CheckboxGroup();

rbMale = new Checkbox("Male", genderGroup, false);
rbMale.setBounds(30, 190, 80, 25);
add(rbMale);

rbFemale = new Checkbox("Female", genderGroup, false);
rbFemale.setBounds(120, 190, 80, 25);
add(rbFemale);

rbOther = new Checkbox("Other", genderGroup, false);
rbOther.setBounds(210, 190, 80, 25);
add(rbOther);

lGender = new Label("Selected Gender:");
lGender.setBounds(30, 225, 110, 25);
add(lGender);

tfGender = new TextField();
tfGender.setBounds(150, 225, 330, 25);
tfGender.setEditable(false);
tfGender.setBackground(Color.lightGray);
add(tfGender);

// ── Section 3: Radio Buttons — Course ──────────────────────────
Label lTitle3 = new Label("Select Course (Radio Button - Single Selection):");
lTitle3.setBounds(30, 285, 350, 25);
lTitle3.setForeground(Color.blue);
add(lTitle3);

courseGroup = new CheckboxGroup();

rbUG = new Checkbox("Under Graduate", courseGroup, false);
rbUG.setBounds(30, 320, 130, 25);
add(rbUG);
```

```
rbPG = new Checkbox("Post Graduate", courseGroup, false);
rbPG.setBounds(170, 320, 120, 25);
add(rbPG);

rbPhD = new Checkbox("PhD", courseGroup, false);
rbPhD.setBounds(300, 320, 80, 25);
add(rbPhD);

lCourse = new Label("Selected Course:");
lCourse.setBounds(30, 355, 110, 25);
add(lCourse);

tfCourse = new TextField();
tfCourse.setBounds(150, 355, 330, 25);
tfCourse.setEditable(false);
tfCourse.setBackground(Color.lightGray);
add(tfCourse);

// ── Result Field ──────────────────────────────────────────
lResult = new Label("Full Summary:");
lResult.setBounds(30, 415, 110, 25);
add(lResult);

tfResult = new TextField();
tfResult.setBounds(150, 415, 330, 25);
tfResult.setEditable(false);
tfResult.setBackground(Color.lightGray);
add(tfResult);

// ── Buttons ───────────────────────────────────────────────
btnSubmit = new Button("Submit");
btnSubmit.setBounds(80, 460, 90, 30);
add(btnSubmit);

btnClear = new Button("Clear");
btnClear.setBounds(200, 460, 90, 30);
add(btnClear);

btnExit = new Button("Exit");
btnExit.setBounds(320, 460, 90, 30);
add(btnExit);
```

```
    // — Item Listeners ————————————————————————————————
    chkJava.addItemListener(this);
    chkPython.addItemListener(this);
    chkCpp.addItemListener(this);
    chkHTML.addItemListener(this);
    chkSQL.addItemListener(this);

    rbMale.addItemListener(this);
    rbFemale.addItemListener(this);
    rbOther.addItemListener(this);

    rbUG.addItemListener(this);
    rbPG.addItemListener(this);
    rbPhD.addItemListener(this);

    // — Action Listeners ——————————————————————————————
    btnSubmit.addActionListener(this);
    btnClear.addActionListener(this);
    btnExit.addActionListener(this);

    // — Window Close ——————————————————————————————————
    addWindowListener(new WindowAdapter() {
        public void windowClosing(WindowEvent e) {
            System.exit(0);
        }
    });
}

// — Item State Changed ———————————————————————————————
public void itemStateChanged(ItemEvent e) {

    // Update skills text field
    StringBuilder skills = new StringBuilder();
    if (chkJava.getState())   skills.append("Java  ");
    if (chkPython.getState()) skills.append("Python  ");
    if (chkCpp.getState())    skills.append("C++  ");
    if (chkHTML.getState())   skills.append("HTML  ");
    if (chkSQL.getState())    skills.append("SQL  ");
    tfSkills.setText(skills.toString().trim());
```

```
    // Update gender text field
    Checkbox selectedGender = genderGroup.getSelectedCheckbox();
    tfGender.setText(selectedGender != null ? selectedGender.getLabel() : "");

    // Update course text field
    Checkbox selectedCourse = courseGroup.getSelectedCheckbox();
    tfCourse.setText(selectedCourse != null ? selectedCourse.getLabel() : "");
}

// ── Action Performed ──────────────────────────────────────────────
public void actionPerformed(ActionEvent e) {

    // Submit — show full summary
    if (e.getSource() == btnSubmit) {
        String skills = tfSkills.getText();
        String gender = tfGender.getText();
        String course = tfCourse.getText();

        if (skills.isEmpty() || gender.isEmpty() || course.isEmpty()) {
            tfResult.setText("Please fill all selections!");
            tfResult.setForeground(Color.red);
        } else {
            tfResult.setText("Skills: " + skills +
                             " | Gender: " + gender +
                             " | Course: " + course);
            tfResult.setForeground(Color.darkGray);
        }
    }

    // Clear — reset everything
    if (e.getSource() == btnClear) {
        chkJava.setState(false);
        chkPython.setState(false);
        chkCpp.setState(false);
        chkHTML.setState(false);
        chkSQL.setState(false);

        genderGroup.setSelectedCheckbox(null);
        courseGroup.setSelectedCheckbox(null);

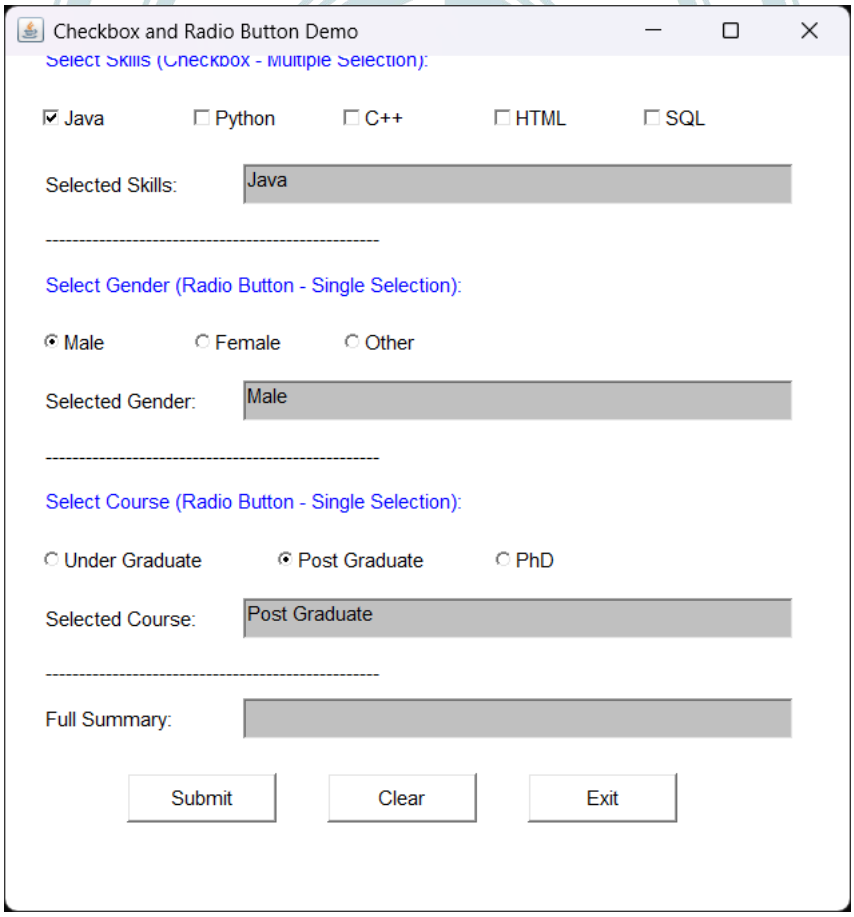
        tfSkills.setText("");
```

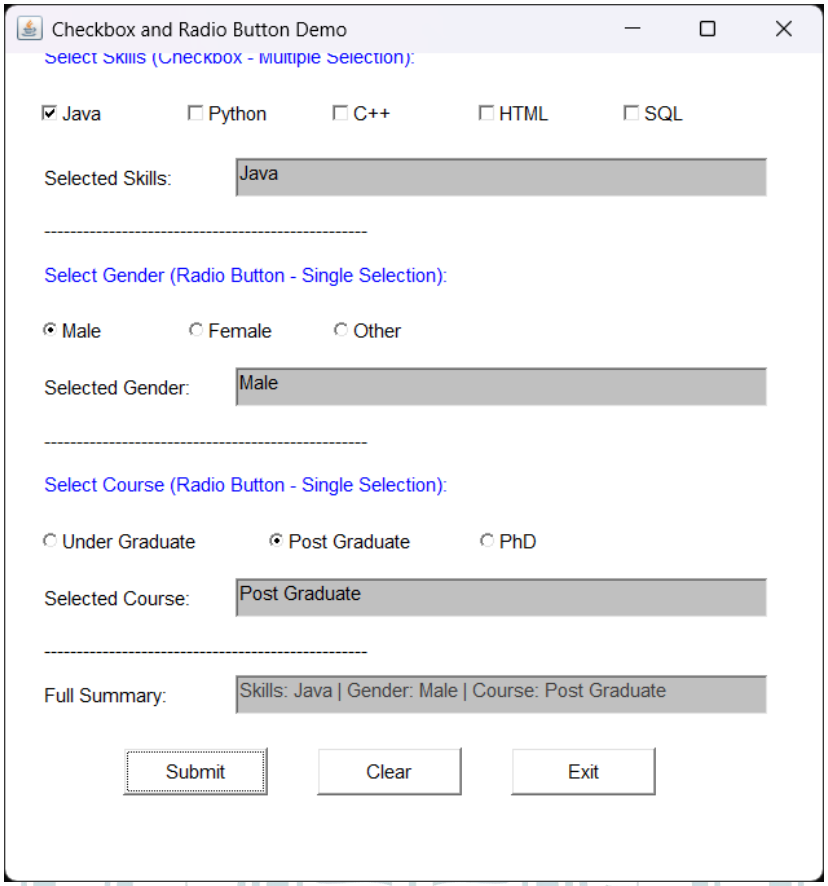
```
        tfGender.setText("");
        tfCourse.setText("");
        tfResult.setText("");
    }

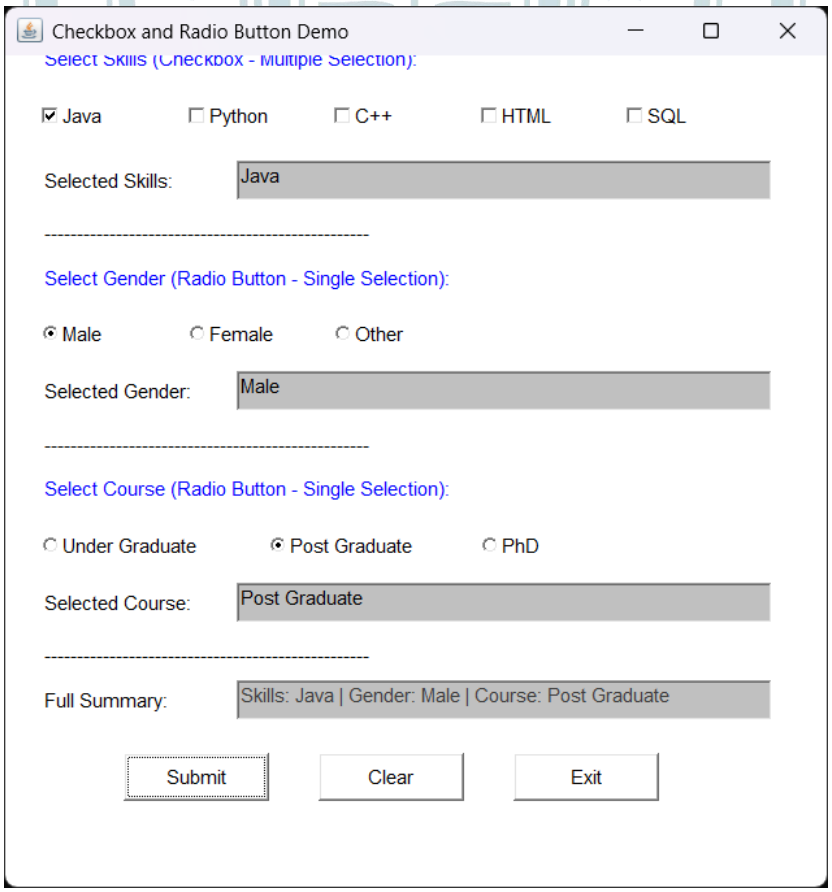
    // Exit
    if (e.getSource() == btnExit) {
        System.exit(0);
    }
  }

  // — main ————————————————————————————————————————————
  public static void main(String[] args) {
    new PG60CheckboxRadioDemo();
  }
}
```

Output




Checkbox and Radio Button Demo
Select Skills (Checkbox - Multiple Selection):
Java
Python
C++
HTML
SQL
Selected Skills:
Java
Select Gender (Radio Button - Single Selection):
Male
Female
Other
Selected Gender:
Male
Select Course (Radio Button - Single Selection):
Under Graduate
Post Graduate
PhD
Selected Course:
Post Graduate
Full Summary:
Skills: Java | Gender: Male | Course: Post Graduate
Submit
Clear
Exit


Checkbox and Radio Button Demo
Select Skills (Checkbox - Multiple Selection):
Java
Python
C++
HTML
SQL
Selected Skills:
Java
Select Gender (Radio Button - Single Selection):
Male
Female
Other
Selected Gender:
Male
Select Course (Radio Button - Single Selection):
Under Graduate
Post Graduate
PhD
Selected Course:
Post Graduate
Full Summary:
Skills: Java | Gender: Male | Course: Post Graduate
Submit
Clear
Exit

Program No: 61	**Date:** 16-02-2026

Program Title: Demo of menus using AWT.

```
/*
AWT Menu Demo
@ Ashbi Saju
Roll no: 16
Date: 16-02-2026
*/

import java.awt.*;
import java.awt.event.*;

public class PG61MenuDemo extends Frame implements ActionListener {

   // TextArea to display output
   TextArea ta;

   // Status label
   Label lStatus;

   // ── Menu Bar ──────────────────────────────────────────────
   MenuBar menuBar;

   // ── Menus ─────────────────────────────────────────────────
   Menu menuFile, menuEdit, menuFormat, menuView, menuHelp;

   // ── File Menu Items ───────────────────────────────────────
   MenuItem miNew, miOpen, miSave, miSaveAs, miPrint, miExit;

   // ── Edit Menu Items ───────────────────────────────────────
   MenuItem miCut, miCopy, miPaste, miDelete, miSelectAll, miFind;

   // ── Format Menu Items ─────────────────────────────────────
   MenuItem miBold, miItalic, miUnderline;
   CheckboxMenuItem miWordWrap;
   Menu menuFont, menuFontSize;
   MenuItem miFontArial, miFontCourier, miFontTimesNew;
   MenuItem miSize10, miSize14, miSize18, miSize24;
```

```
// ── View Menu Items ──────────────────────────────────────────────
CheckboxMenuItem miToolbar, miStatusBar, miFullScreen;

// ── Help Menu Items ──────────────────────────────────────────────
MenuItem miAbout, miHelp;

public PG61MenuDemo() {

   setTitle("AWT Menu Demo");
   setSize(600, 500);
   setLayout(null);

   // ── TextArea ──────────────────────────────────────────────────
   ta = new TextArea("Welcome to AWT Menu Demo!\n" +
                     "Use the menu bar above to try all menu options.\n");
   ta.setBounds(10, 10, 570, 400);
   ta.setFont(new Font("Arial", Font.PLAIN, 14));
   add(ta);

   // ── Status Label ──────────────────────────────────────────────
   lStatus = new Label("Status: Ready");
   lStatus.setBounds(10, 420, 570, 25);
   lStatus.setForeground(Color.blue);
   add(lStatus);

   // ── Menu Bar ──────────────────────────────────────────────────
   menuBar = new MenuBar();

   // ── File Menu ─────────────────────────────────────────────────
   menuFile = new Menu("File");

   miNew    = new MenuItem("New",       new MenuShortcut('N'));
   miOpen   = new MenuItem("Open",      new MenuShortcut('O'));
   miSave   = new MenuItem("Save",      new MenuShortcut('S'));
   miSaveAs = new MenuItem("Save As");
   miPrint  = new MenuItem("Print",     new MenuShortcut('P'));
   miExit   = new MenuItem("Exit");

   menuFile.add(miNew);
   menuFile.add(miOpen);
   menuFile.add(miSave);
```

```
menuFile.add(miSaveAs);
menuFile.addSeparator();
menuFile.add(miPrint);
menuFile.addSeparator();
menuFile.add(miExit);

// — Edit Menu ————————————————————————————————————————
menuEdit = new Menu("Edit");

miCut      = new MenuItem("Cut",       new MenuShortcut('X'));
miCopy     = new MenuItem("Copy",      new MenuShortcut('C'));
miPaste    = new MenuItem("Paste",     new MenuShortcut('V'));
miDelete   = new MenuItem("Delete");
miSelectAll = new MenuItem("Select All", new MenuShortcut('A'));
miFind     = new MenuItem("Find",      new MenuShortcut('F'));

menuEdit.add(miCut);
menuEdit.add(miCopy);
menuEdit.add(miPaste);
menuEdit.add(miDelete);
menuEdit.addSeparator();
menuEdit.add(miSelectAll);
menuEdit.addSeparator();
menuEdit.add(miFind);

// — Format Menu ——————————————————————————————————————
menuFormat = new Menu("Format");

miBold      = new MenuItem("Bold");
miItalic    = new MenuItem("Italic");
miUnderline  = new MenuItem("Underline");
miWordWrap   = new CheckboxMenuItem("Word Wrap", true);

// Font submenu
menuFont = new Menu("Font");
miFontArial    = new MenuItem("Arial");
miFontCourier  = new MenuItem("Courier");
miFontTimesNew  = new MenuItem("Times New Roman");
menuFont.add(miFontArial);
menuFont.add(miFontCourier);
menuFont.add(miFontTimesNew);
```

```
// Font size submenu
menuFontSize = new Menu("Font Size");
miSize10 = new MenuItem("10");
miSize14 = new MenuItem("14");
miSize18 = new MenuItem("18");
miSize24 = new MenuItem("24");
menuFontSize.add(miSize10);
menuFontSize.add(miSize14);
menuFontSize.add(miSize18);
menuFontSize.add(miSize24);

menuFormat.add(miBold);
menuFormat.add(miItalic);
menuFormat.add(miUnderline);
menuFormat.addSeparator();
menuFormat.add(miWordWrap);
menuFormat.addSeparator();
menuFormat.add(menuFont);
menuFormat.add(menuFontSize);

// ── View Menu ──────────────────────────────────────────────────────
menuView = new Menu("View");

miToolbar   = new CheckboxMenuItem("Toolbar",    true);
miStatusBar = new CheckboxMenuItem("Status Bar", true);
miFullScreen = new CheckboxMenuItem("Full Screen", false);

menuView.add(miToolbar);
menuView.add(miStatusBar);
menuView.addSeparator();
menuView.add(miFullScreen);

// ── Help Menu ──────────────────────────────────────────────────────
menuHelp = new Menu("Help");

miHelp  = new MenuItem("Help Topics");
miAbout = new MenuItem("About");

menuHelp.add(miHelp);
menuHelp.addSeparator();
```

```
        menuHelp.add(miAbout);

        // ── Add menus to menu bar ──────────────────────────────────────
        menuBar.add(menuFile);
        menuBar.add(menuEdit);
        menuBar.add(menuFormat);
        menuBar.add(menuView);
        menuBar.setHelpMenu(menuHelp);

        // ── Action Listeners — File ────────────────────────────────────
        miNew.addActionListener(this);
        miOpen.addActionListener(this);
        miSave.addActionListener(this);
        miSaveAs.addActionListener(this);
        miPrint.addActionListener(this);
        miExit.addActionListener(this);

        // ── Action Listeners — Edit ────────────────────────────────────
        miCut.addActionListener(this);
        miCopy.addActionListener(this);
        miPaste.addActionListener(this);
        miDelete.addActionListener(this);
        miSelectAll.addActionListener(this);
        miFind.addActionListener(this);

        // ── Action Listeners — Format ──────────────────────────────────
        miBold.addActionListener(this);
        miItalic.addActionListener(this);
        miUnderline.addActionListener(this);
        miWordWrap.addActionListener(this);
        miFontArial.addActionListener(this);
        miFontCourier.addActionListener(this);
        miFontTimesNew.addActionListener(this);
        miSize10.addActionListener(this);
        miSize14.addActionListener(this);
        miSize18.addActionListener(this);
        miSize24.addActionListener(this);

        // ── Action Listeners — View ────────────────────────────────────
        miToolbar.addActionListener(this);
        miStatusBar.addActionListener(this);
```

```
    miFullScreen.addActionListener(this);

    // ── Action Listeners — Help ──────────────────────────────────
    miHelp.addActionListener(this);
    miAbout.addActionListener(this);
    setVisible(true);
}

// ── Action Performed ──────────────────────────────────────────────
public void actionPerformed(ActionEvent e) {

    String cmd = e.getActionCommand();

    switch (cmd) {

        // File menu
        case "New":
            ta.setText("");
            lStatus.setText("Status: New file created.");
            break;

        case "Open":
            ta.append(">> Open file dialog would appear here.\n");
            lStatus.setText("Status: Open selected.");
            break;

        case "Save":
            ta.append(">> File saved.\n");
            lStatus.setText("Status: File saved.");
            break;

        case "Save As":
            ta.append(">> Save As dialog would appear here.\n");
            lStatus.setText("Status: Save As selected.");
            break;

        case "Print":
            ta.append(">> Print dialog would appear here.\n");
            lStatus.setText("Status: Print selected.");
            break;
```

```
        case "Exit":
            System.exit(0);
            break;

        // Edit menu
        case "Cut":
            ta.append(">> Cut selected text.\n");
            lStatus.setText("Status: Cut.");
            break;

        case "Copy":
            ta.append(">> Copied selected text.\n");
            lStatus.setText("Status: Copy.");
            break;

        case "Paste":
            ta.append(">> Pasted text.\n");
            lStatus.setText("Status: Paste.");
            break;

        case "Delete":
            ta.append(">> Deleted selected text.\n");
            lStatus.setText("Status: Delete.");
            break;

        case "Select All":
            ta.selectAll();
            lStatus.setText("Status: All text selected.");
            break;

        case "Find":
            ta.append(">> Find dialog would appear here.\n");
            lStatus.setText("Status: Find selected.");
            break;

        // Format menu
        case "Bold":
            ta.setFont(new Font(ta.getFont().getName(), Font.BOLD,
                ta.getFont().getSize()));
            lStatus.setText("Status: Bold applied.");
            break;
```

```
case "Italic":
   ta.setFont(new Font(ta.getFont().getName(), Font.ITALIC,
      ta.getFont().getSize()));
   lStatus.setText("Status: Italic applied.");
   break;

case "Underline":
   ta.append(">> Underline applied.\n");
   lStatus.setText("Status: Underline selected.");
   break;

case "Word Wrap":
   lStatus.setText("Status: Word Wrap = " + miWordWrap.getState());
   break;

// Font submenu
case "Arial":
   ta.setFont(new Font("Arial", ta.getFont().getStyle(),
      ta.getFont().getSize()));
   lStatus.setText("Status: Font changed to Arial.");
   break;

case "Courier":
   ta.setFont(new Font("Courier", ta.getFont().getStyle(),
      ta.getFont().getSize()));
   lStatus.setText("Status: Font changed to Courier.");
   break;

case "Times New Roman":
   ta.setFont(new Font("Times New Roman", ta.getFont().getStyle(),
      ta.getFont().getSize()));
   lStatus.setText("Status: Font changed to Times New Roman.");
   break;

// Font size submenu
case "10":
   ta.setFont(new Font(ta.getFont().getName(),
      ta.getFont().getStyle(), 10));
   lStatus.setText("Status: Font size set to 10.");
   break;
```

```
case "14":
   ta.setFont(new Font(ta.getFont().getName(),
      ta.getFont().getStyle(), 14));
   lStatus.setText("Status: Font size set to 14.");
   break;

case "18":
   ta.setFont(new Font(ta.getFont().getName(),
      ta.getFont().getStyle(), 18));
   lStatus.setText("Status: Font size set to 18.");
   break;

case "24":
   ta.setFont(new Font(ta.getFont().getName(),
      ta.getFont().getStyle(), 24));
   lStatus.setText("Status: Font size set to 24.");
   break;

// View menu
case "Toolbar":
   lStatus.setText("Status: Toolbar = " + miToolbar.getState());
   break;

case "Status Bar":
   lStatus.setText("Status: Status Bar = " + miStatusBar.getState());
   break;

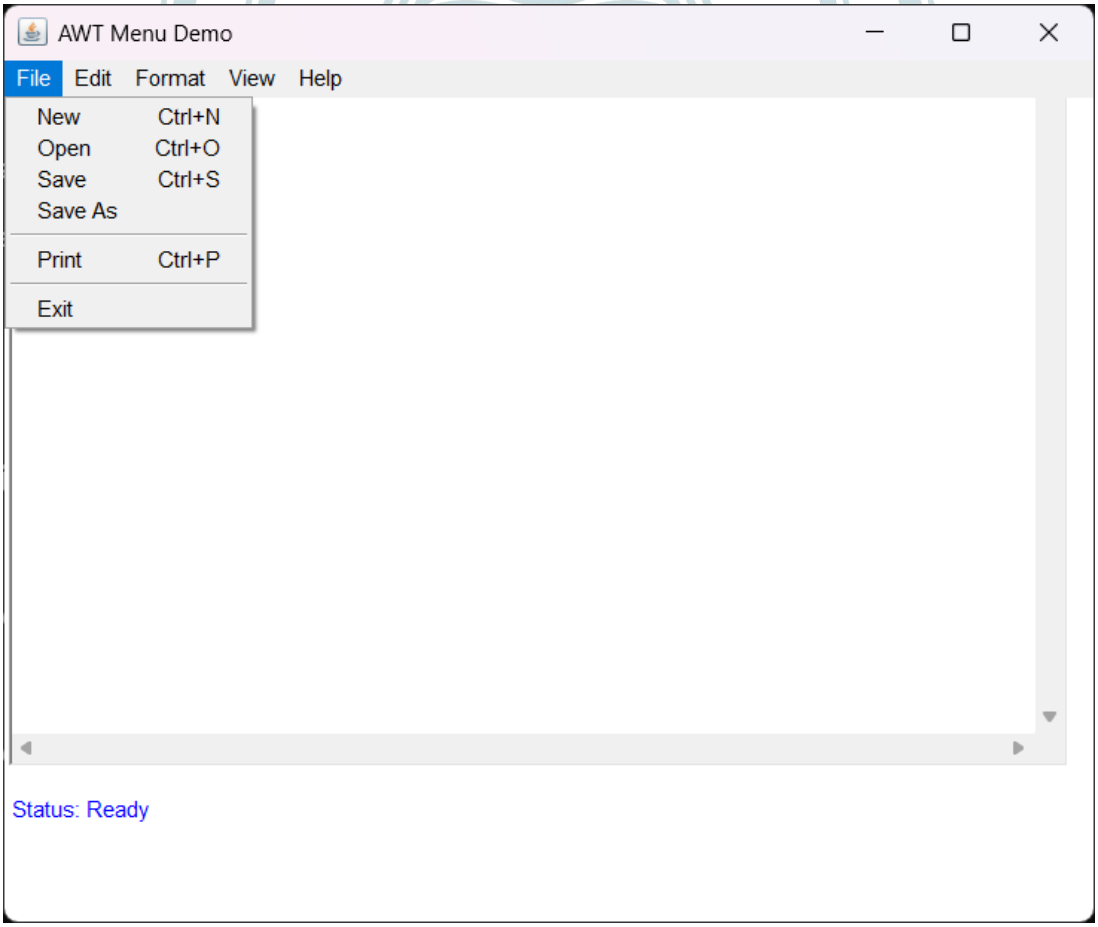
case "Full Screen":
   lStatus.setText("Status: Full Screen = " + miFullScreen.getState());
   break;

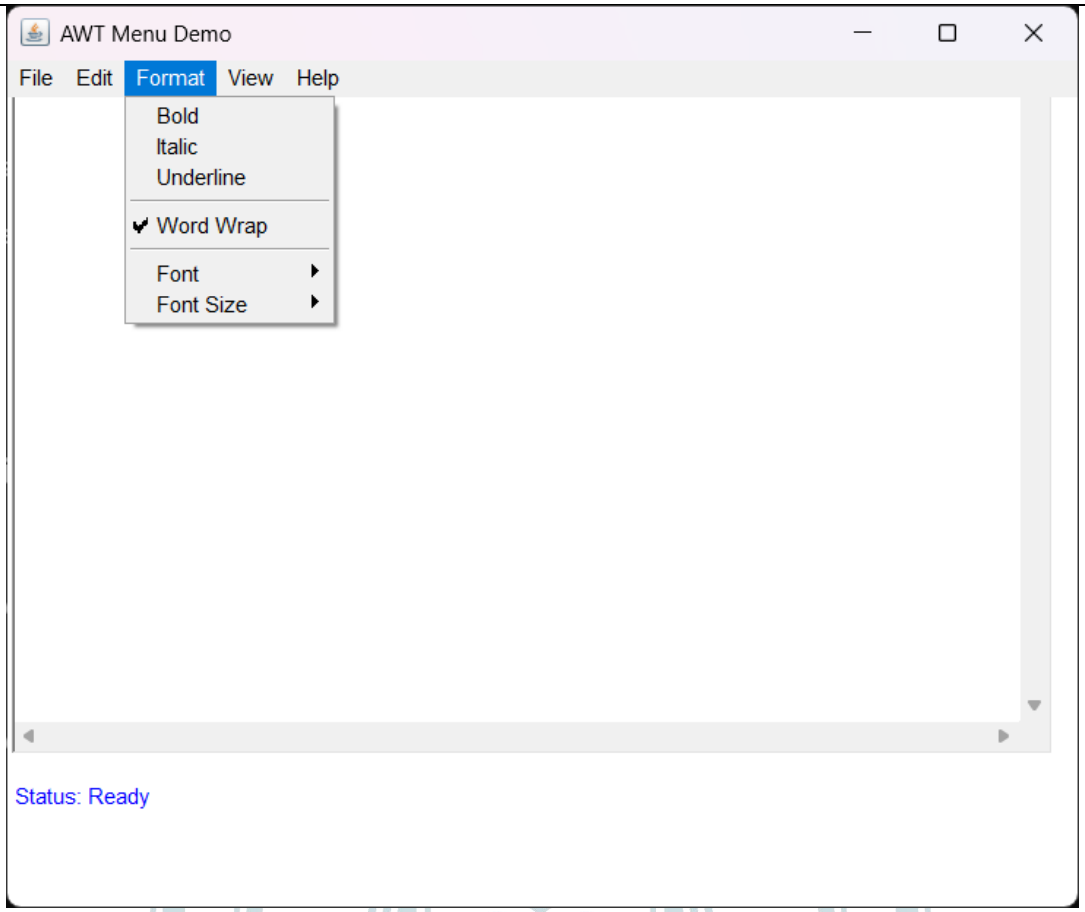
// Help menu
case "Help Topics":
   ta.append(">> Help Topics:\n" +
         "   1. Use File menu to create/open/save files.\n" +
         "   2. Use Edit menu to cut/copy/paste text.\n" +
         "   3. Use Format menu to change font/style.\n" +
         "   4. Use View menu to toggle toolbar/statusbar.\n");
   lStatus.setText("Status: Help Topics opened.");
   break;
```

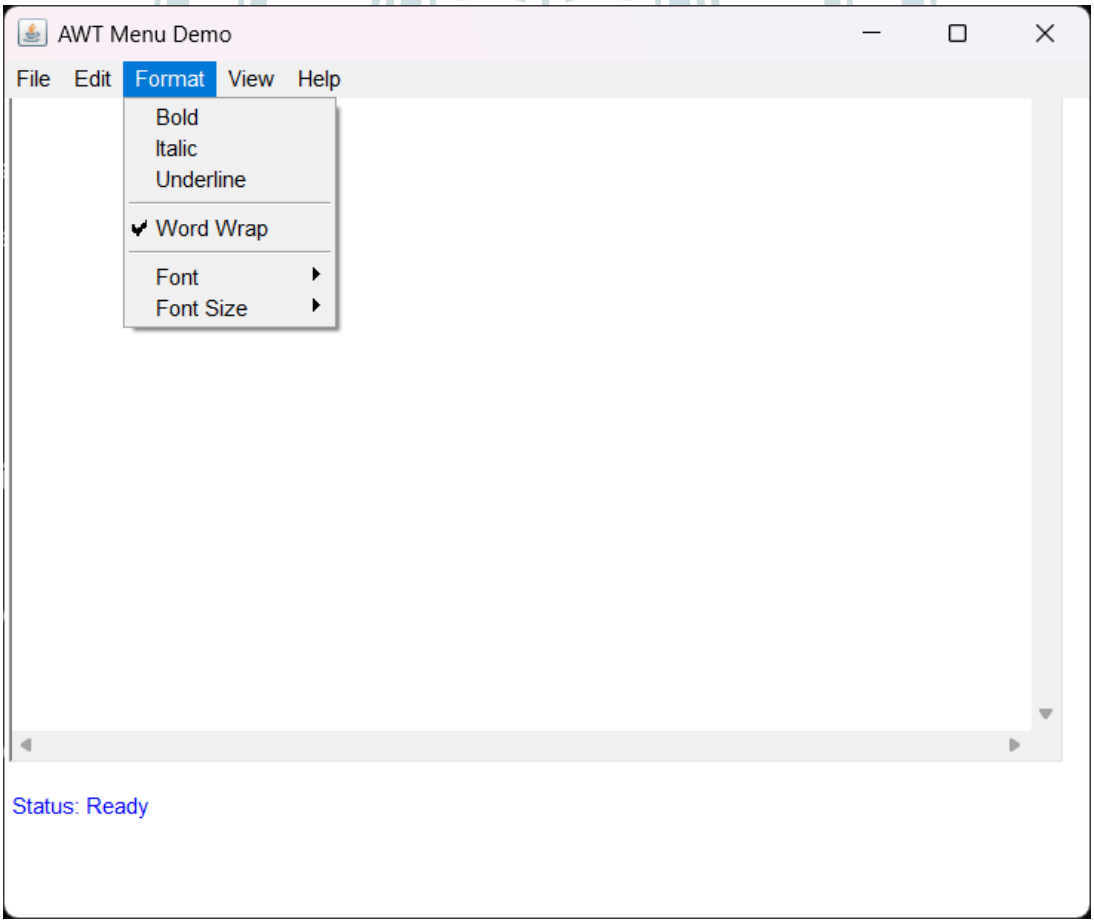
```
        case "About":
            ta.append(">> About:\n" +
                    "   AWT Menu Demo\n" +
                    "   Version 1.0\n" +
                    "   Demonstrates all AWT Menu functionalities.\n");
            lStatus.setText("Status: About opened.");
            break;
        }
    }

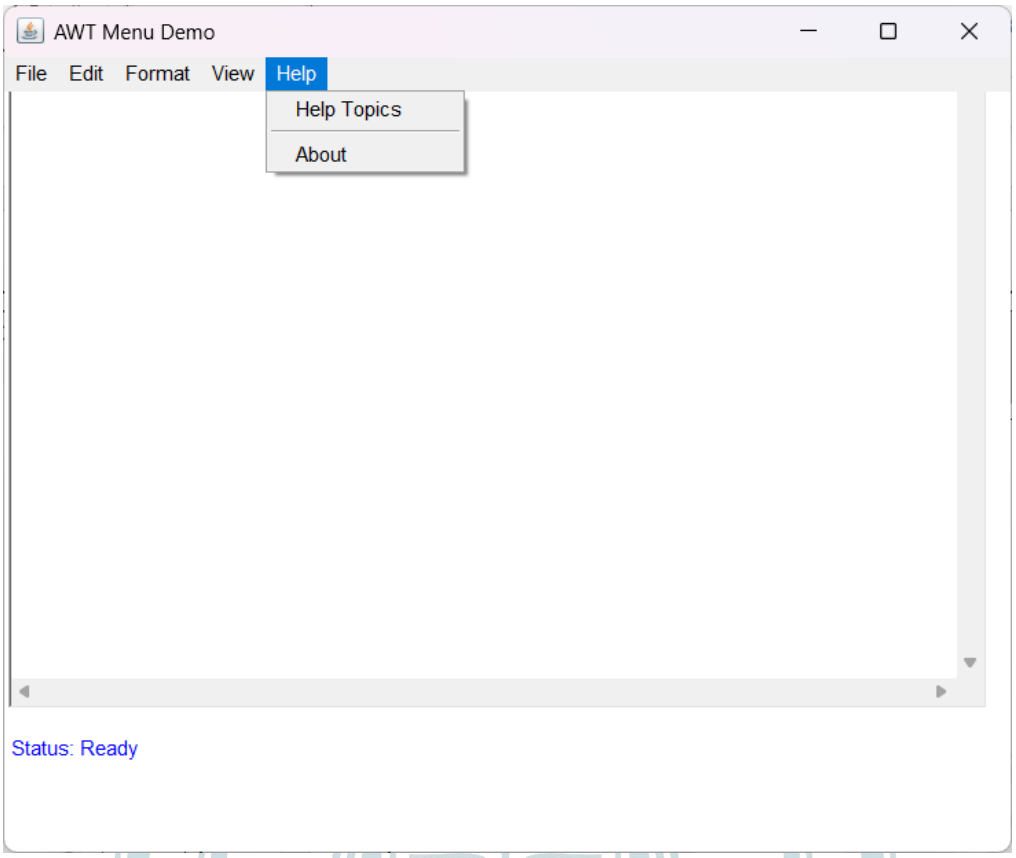
    // — main ————————————————————————————————————————————————
    public static void main(String[] args) {
        new PG61MenuDemo();
    }
}
```

Output




AWT Menu Demo
File
Edit
Format
View
Help
Bold
Italic
Underline
Word Wrap
Font
Font Size
Status: Ready


AWT Menu Demo
File
Edit
Format
View
Help
Bold
Italic
Underline
Word Wrap
Font
Font Size
Status: Ready


AWT Menu Demo
File
Edit
Format
View
Help
Help Topics
About
Status: Ready

Program No: 62	**Date:** 02-03-2026

Program Title: Project - E-Commerce Management & Shopping System

```
/*
E-Commerce Management & Shopping System
@ Ashbi Saju
Roll no: 16
Date: 04-03-2026
*/
```

CartFrame.java

```
package ecommercesystem;

import java.awt.*;
import java.sql.*;
import java.util.*;
import javax.swing.*;
import javax.swing.border.*;
import javax.swing.table.DefaultTableModel;

public class CartFrame extends JFrame {

   private JTable         productTable, cartTable;
   private DefaultTableModel productModel, cartModel;
   private JLabel         lblTotal, lblCustomerName;
   private Map<Integer, Double> productPrices = new HashMap<>();
   private double totalAmount     = 0;
   private int    currentCustomerId = -1;

   public CartFrame() { this(-1); }

   public CartFrame(int customerId) {
      setTitle("Shop");
      setSize(1100, 640);
      setLocationRelativeTo(null);
      setLayout(new BorderLayout());
      this.currentCustomerId = customerId;
      initComponents();
      loadProducts();
      loadCustomerName();
   }

   private void initComponents() {
      JPanel header = new JPanel(new BorderLayout());
```

```
header.setBackground(new Color(44, 62, 80));
header.setPreferredSize(new Dimension(0, 55));
header.setBorder(new EmptyBorder(0, 20, 0, 20));
JLabel lblH = new JLabel("Shop - Browse & Order");
lblH.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 17));
lblH.setForeground(Color.WHITE);

lblCustomerName = new JLabel("");
lblCustomerName.setFont(new Font(UIUtils.EMOJI_FONT, Font.PLAIN, 12));
lblCustomerName.setForeground(new Color(189, 195, 199));

header.add(lblH,          BorderLayout.WEST);
header.add(lblCustomerName, BorderLayout.EAST);
add(header, BorderLayout.NORTH);

// ── LEFT - Products ──
JPanel leftPanel = new JPanel(new BorderLayout());
leftPanel.setBackground(new Color(245, 247, 250));
leftPanel.setBorder(new EmptyBorder(14, 14, 14, 7));

JLabel lblP = new JLabel("Available Products");
lblP.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 13));
lblP.setForeground(new Color(44, 62, 80));
lblP.setBorder(new EmptyBorder(0, 0, 8, 0));

JLabel lblHint = new JLabel("Double-click to add to cart");
lblHint.setFont(new Font(UIUtils.EMOJI_FONT, Font.PLAIN, 10));
lblHint.setForeground(new Color(149, 165, 166));
lblHint.setBorder(new EmptyBorder(6, 0, 0, 0));

String[] pCols = {"ID","Name","Price","Stock","Category"};
productModel = new DefaultTableModel(pCols, 0) {
   public boolean isCellEditable(int r, int c) { return false; }
};
productTable = new JTable(productModel);
UIUtils.styleTable(productTable);
productTable.addMouseListener(new java.awt.event.MouseAdapter() {
   public void mouseClicked(java.awt.event.MouseEvent e) {
      if (e.getClickCount() == 2) addToCart();
   }
});

JScrollPane ps = new JScrollPane(productTable);
ps.setBorder(new LineBorder(new Color(220, 220, 220), 1));
ps.getViewport().setBackground(Color.WHITE);

leftPanel.add(lblP,   BorderLayout.NORTH);
```

```
leftPanel.add(ps,     BorderLayout.CENTER);
leftPanel.add(lblHint,BorderLayout.SOUTH);

// ── RIGHT - Cart ──
JPanel rightPanel = new JPanel(new BorderLayout());
rightPanel.setBackground(new Color(245, 247, 250));
rightPanel.setBorder(new EmptyBorder(14, 7, 14, 14));

JLabel lblC = new JLabel("Your Cart");
lblC.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 13));
lblC.setForeground(new Color(44, 62, 80));
lblC.setBorder(new EmptyBorder(0, 0, 8, 0));

String[] cCols = {"Product ID","Product Name","Qty","Unit Price","Subtotal"};
cartModel = new DefaultTableModel(cCols, 0) {
   public boolean isCellEditable(int r, int c) { return false; }
};
cartTable = new JTable(cartModel);
UIUtils.styleTable(cartTable);

JScrollPane cs = new JScrollPane(cartTable);
cs.setBorder(new LineBorder(new Color(220, 220, 220), 1));
cs.getViewport().setBackground(Color.WHITE);

// action bar
JPanel actionBar = new JPanel(new BorderLayout());
actionBar.setBackground(Color.WHITE);
actionBar.setBorder(new CompoundBorder(
   new MatteBorder(1, 0, 0, 0, new Color(230, 230, 230)),
   new EmptyBorder(10, 14, 10, 14)));

JPanel leftActions = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 0));
leftActions.setOpaque(false);
JButton btnRemove = actionBtn("Remove",    new Color(231, 76, 60));
JButton btnClear  = actionBtn("Clear Cart", new Color(149, 165, 166));
btnRemove.setPreferredSize(new Dimension(90, 32));
btnClear.setPreferredSize(new Dimension(90, 32));
leftActions.add(btnRemove);
leftActions.add(btnClear);

lblTotal = new JLabel("Total: Rs.0.00");
lblTotal.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 16));
lblTotal.setForeground(new Color(231, 76, 60));

actionBar.add(leftActions, BorderLayout.WEST);
actionBar.add(lblTotal,    BorderLayout.EAST);
```

```
    // place order bar
    JPanel orderBar = new JPanel(new BorderLayout());
    orderBar.setBackground(new Color(245, 247, 250));
    orderBar.setBorder(new EmptyBorder(10, 0, 0, 0));

    JButton btnPlace = new JButton("Place Order");
    btnPlace.setBackground(new Color(39, 174, 96));
    btnPlace.setForeground(Color.WHITE);
    btnPlace.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 14));
    btnPlace.setFocusPainted(false);
    btnPlace.setBorderPainted(false);
    btnPlace.setOpaque(true);
    btnPlace.setCursor(new Cursor(Cursor.HAND_CURSOR));
    btnPlace.setPreferredSize(new Dimension(140, 40));
    btnPlace.setBorder(new EmptyBorder(0, 0, 0, 0));
    btnPlace.addMouseListener(new java.awt.event.MouseAdapter() {
       public void mouseEntered(java.awt.event.MouseEvent e) {
          btnPlace.setBackground(new Color(39, 174, 96).brighter());
       }
       public void mouseExited(java.awt.event.MouseEvent e) {
          btnPlace.setBackground(new Color(39, 174, 96));
       }
    });

    orderBar.add(btnPlace, BorderLayout.EAST);

    JPanel rightBottom = new JPanel(new BorderLayout());
    rightBottom.setBackground(new Color(245, 247, 250));
    rightBottom.add(actionBar, BorderLayout.NORTH);
    rightBottom.add(orderBar,  BorderLayout.SOUTH);

    rightPanel.add(lblC,       BorderLayout.NORTH);
    rightPanel.add(cs,         BorderLayout.CENTER);
    rightPanel.add(rightBottom, BorderLayout.SOUTH);

    JSplitPane split = new JSplitPane(
       JSplitPane.HORIZONTAL_SPLIT, leftPanel, rightPanel);
    split.setDividerLocation(530);
    split.setDividerSize(4);
    split.setBorder(null);
    add(split, BorderLayout.CENTER);

    btnRemove.addActionListener(e -> removeFromCart());
    btnClear.addActionListener(e  -> { cartModel.setRowCount(0); updateTotal(); });
    btnPlace.addActionListener(e  -> placeOrder());
  }
```

```
private void loadCustomerName() {
    if (currentCustomerId == -1) {
        lblCustomerName.setText("Admin Mode");
        return;
    }
    try (Connection con = DBConnection.getConnection();
         PreparedStatement ps = con.prepareStatement(
            "SELECT name FROM customers WHERE id=?")) {
        ps.setInt(1, currentCustomerId);
        ResultSet rs = ps.executeQuery();
        if (rs.next())
            lblCustomerName.setText("Shopping as: " + rs.getString("name"));
    } catch (SQLException e) {}
}

private void loadProducts() {
    productModel.setRowCount(0);
    productPrices.clear();
    try (Connection con = DBConnection.getConnection();
         Statement st = con.createStatement();
         ResultSet rs = st.executeQuery(
            "SELECT * FROM products WHERE stock > 0 ORDER BY name")) {
        while (rs.next()) {
            int    id    = rs.getInt("id");
            double price = rs.getDouble("price");
            productPrices.put(id, price);
            productModel.addRow(new Object[]{
                id, rs.getString("name"), "Rs." + price,
                rs.getInt("stock"), rs.getString("category")
            });
        }
    } catch (SQLException e) {
        JOptionPane.showMessageDialog(this, e.getMessage());
    }
}

private void addToCart() {
    int row = productTable.getSelectedRow();
    if (row < 0) return;
    int    pid   = (int)    productModel.getValueAt(row, 0);
    String pname = (String) productModel.getValueAt(row, 1);
    double price = productPrices.get(pid);

    for (int i = 0; i < cartModel.getRowCount(); i++) {
        if ((int) cartModel.getValueAt(i, 0) == pid) {
            int qty = (int) cartModel.getValueAt(i, 2) + 1;
            cartModel.setValueAt(qty,        i, 2);
```

```
            cartModel.setValueAt(qty * price, i, 4);
            updateTotal(); return;
        }
    }
    cartModel.addRow(new Object[]{pid, pname, 1, price, price});
    updateTotal();
}

private void removeFromCart() {
    int row = cartTable.getSelectedRow();
    if (row >= 0) { cartModel.removeRow(row); updateTotal(); }
    else JOptionPane.showMessageDialog(this, "Select an item to remove!");
}

private void updateTotal() {
    totalAmount = 0;
    for (int i = 0; i < cartModel.getRowCount(); i++)
        totalAmount += (double) cartModel.getValueAt(i, 4);
    lblTotal.setText(String.format("Total: Rs.%.2f", totalAmount));
}

private void placeOrder() {
    if (cartModel.getRowCount() == 0) {
        JOptionPane.showMessageDialog(this, "Cart is empty!"); return;
    }
    if (currentCustomerId == -1) {
        JOptionPane.showMessageDialog(this,
            "No customer linked. Please log in as a customer."); return;
    }
    try (Connection con = DBConnection.getConnection()) {
        con.setAutoCommit(false);
        try {
            PreparedStatement ps = con.prepareStatement(
                "INSERT INTO orders (customer_id,total_amount,status) VALUES (?,?,'pending')",
                new String[]{"ID"});
            ps.setInt(1, currentCustomerId);
            ps.setDouble(2, totalAmount);
            ps.executeUpdate();

            ResultSet rs      = ps.getGeneratedKeys();
            int       orderId = rs.next() ? rs.getInt(1) : -1;

            for (int i = 0; i < cartModel.getRowCount(); i++) {
                int    pid   = (int)    cartModel.getValueAt(i, 0);
                int    qty   = (int)    cartModel.getValueAt(i, 2);
                double price = (double) cartModel.getValueAt(i, 3);
```

```
                PreparedStatement ps2 = con.prepareStatement(
                    "INSERT INTO order_items (order_id,product_id,quantity,unit_price) " +
                    "VALUES (?,?,?,?)");
                ps2.setInt(1, orderId); ps2.setInt(2, pid);
                ps2.setInt(3, qty);     ps2.setDouble(4, price);
                ps2.executeUpdate();

                PreparedStatement ps3 = con.prepareStatement(
                    "UPDATE products SET stock = stock - ? WHERE id=?");
                ps3.setInt(1, qty); ps3.setInt(2, pid);
                ps3.executeUpdate();
            }
            con.commit();
            JOptionPane.showMessageDialog(this,
                "Order #" + orderId + " placed!\nTotal: Rs." +
                String.format("%.2f", totalAmount),
                "Order Placed", JOptionPane.INFORMATION_MESSAGE);
            cartModel.setRowCount(0);
            updateTotal();
            loadProducts();
        } catch (SQLException ex) {
            con.rollback();
            JOptionPane.showMessageDialog(this, "Order failed: " + ex.getMessage());
        }
    } catch (SQLException e) {
        JOptionPane.showMessageDialog(this, e.getMessage());
    }
  }

  private JButton actionBtn(String text, Color bg) {
    JButton btn = new JButton(text);
    btn.setBackground(bg); btn.setForeground(Color.WHITE);
    btn.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 12));
    btn.setFocusPainted(false); btn.setBorderPainted(false);
    btn.setOpaque(true); btn.setCursor(new Cursor(Cursor.HAND_CURSOR));
    btn.setBorder(new EmptyBorder(6, 10, 6, 10));
    btn.addMouseListener(new java.awt.event.MouseAdapter() {
        public void mouseEntered(java.awt.event.MouseEvent e) {
btn.setBackground(bg.brighter()); }
        public void mouseExited(java.awt.event.MouseEvent e)  { btn.setBackground(bg); }
    });
    return btn;
  }
}
```

CustomerFrame.java

```
package ecommercesystem;

import java.awt.*;
import java.sql.*;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.table.DefaultTableModel;

public class CustomerFrame extends JFrame {

    private JTable table;
    private DefaultTableModel model;
    private JTextField txtName, txtEmail, txtPhone, txtAddress;
    private int selectedId = -1;

    public CustomerFrame() {
        setTitle("Customer Management");
        setSize(1000, 580);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());
        initComponents();
        loadCustomers();
    }

    private void initComponents() {
        add(UIUtils.createHeader("👥  Customer Management"), BorderLayout.NORTH);

        // LEFT
        JPanel leftPanel = new JPanel(new BorderLayout());
        leftPanel.setBackground(UIUtils.COLOR_BACKGROUND);
        leftPanel.setBorder(new EmptyBorder(15, 15, 15, 8));
        leftPanel.setPreferredSize(new Dimension(300, 0));

        JPanel card = UIUtils.createCard();
        card.setLayout(new GridBagLayout());

        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(6, 5, 6, 5);
        gbc.fill = GridBagConstraints.HORIZONTAL;
        gbc.weightx = 1;

        txtName    = UIUtils.createTextField();
        txtEmail   = UIUtils.createTextField();
        txtPhone   = UIUtils.createTextField();
        txtAddress = UIUtils.createTextField();
```

```
String[]     labels = {"Full Name", "Email Address", "Phone Number", "Address"};
JTextField[] fields = {txtName, txtEmail, txtPhone, txtAddress};

for (int i = 0; i < labels.length; i++) {
   gbc.gridy = i * 2; gbc.gridx = 0;
   JLabel lbl = new JLabel(labels[i]);
   lbl.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 12));
   lbl.setForeground(UIUtils.COLOR_TEXT);
   card.add(lbl, gbc);
   gbc.gridy = i * 2 + 1;
   fields[i].setPreferredSize(new Dimension(240, 34));
   card.add(fields[i], gbc);
}

JPanel btnPanel = new JPanel(new GridLayout(2, 2, 8, 8));
btnPanel.setOpaque(false);
btnPanel.setBorder(new EmptyBorder(15, 0, 0, 0));

JButton btnAdd    = UIUtils.createButton("➕ Add",   UIUtils.COLOR_SUCCESS);
JButton btnUpdate = UIUtils.createButton("✏ Update", UIUtils.COLOR_WARNING);
JButton btnDelete = UIUtils.createButton("🗑 Delete", UIUtils.COLOR_DANGER);
JButton btnClear  = UIUtils.createButton("Clear",    UIUtils.COLOR_SUBTLE);

btnPanel.add(btnAdd); btnPanel.add(btnUpdate);
btnPanel.add(btnDelete); btnPanel.add(btnClear);

gbc.gridy = labels.length * 2;
card.add(btnPanel, gbc);
leftPanel.add(card, BorderLayout.NORTH);

// RIGHT
JPanel rightPanel = new JPanel(new BorderLayout());
rightPanel.setBackground(UIUtils.COLOR_BACKGROUND);
rightPanel.setBorder(new EmptyBorder(15, 8, 15, 15));

JPanel searchPanel = new JPanel(new BorderLayout(8, 0));
searchPanel.setOpaque(false);
searchPanel.setBorder(new EmptyBorder(0, 0, 10, 0));
JTextField txtSearch = UIUtils.createTextField();
txtSearch.setPreferredSize(new Dimension(0, 36));

JButton btnSearch  = UIUtils.createButton("🔍 Search", UIUtils.COLOR_ACCENT);
JButton btnRefresh = UIUtils.createButton("↺ All",     UIUtils.COLOR_PRIMARY);
btnSearch.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 13));
btnRefresh.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 13));
```

```
JPanel sb = new JPanel(new FlowLayout(FlowLayout.RIGHT, 5, 0));
sb.setOpaque(false);
sb.add(btnSearch); sb.add(btnRefresh);

JLabel lblSearch = new JLabel("Search: ");
lblSearch.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 13));
lblSearch.setForeground(UIUtils.COLOR_TEXT);

searchPanel.add(lblSearch, BorderLayout.WEST);
searchPanel.add(txtSearch, BorderLayout.CENTER);
searchPanel.add(sb, BorderLayout.EAST);

String[] cols = {"ID", "Name", "Email", "Phone", "Address"};
model = new DefaultTableModel(cols, 0) {
   public boolean isCellEditable(int r, int c) { return false; }
};
table = new JTable(model);
UIUtils.styleTable(table);

JScrollPane scroll = new JScrollPane(table);
scroll.setBorder(BorderFactory.createLineBorder(UIUtils.COLOR_SUBTLE));
scroll.getViewport().setBackground(UIUtils.COLOR_WHITE);

rightPanel.add(searchPanel, BorderLayout.NORTH);
rightPanel.add(scroll, BorderLayout.CENTER);

table.getSelectionModel().addListSelectionListener(e -> {
   int row = table.getSelectedRow();
   if (row >= 0) {
      selectedId = (int) model.getValueAt(row, 0);
      txtName.setText((String) model.getValueAt(row, 1));
      txtEmail.setText((String) model.getValueAt(row, 2));
      txtPhone.setText((String) model.getValueAt(row, 3));
      txtAddress.setText((String) model.getValueAt(row, 4));
   }
});

btnAdd.addActionListener(e -> {
   if (txtName.getText().isEmpty()) { showError("Name is required!"); return; }
   try (Connection con = DBConnection.getConnection()) {
      PreparedStatement ps = con.prepareStatement(
         "INSERT INTO customers (name,email,phone,address) VALUES (?,?,?,?)");
      ps.setString(1, txtName.getText()); ps.setString(2, txtEmail.getText());
      ps.setString(3, txtPhone.getText()); ps.setString(4, txtAddress.getText());
      ps.executeUpdate();
      showSuccess("Customer added!"); clearForm(); loadCustomers();
   } catch (SQLException ex) { showError(ex.getMessage()); }
```

```
        });

        btnUpdate.addActionListener(e -> {
            if (selectedId == -1) { showError("Select a customer!"); return; }
            try (Connection con = DBConnection.getConnection()) {
                PreparedStatement ps = con.prepareStatement(
                    "UPDATE customers SET name=?,email=?,phone=?,address=? WHERE id=?");
                ps.setString(1, txtName.getText()); ps.setString(2, txtEmail.getText());
                ps.setString(3, txtPhone.getText()); ps.setString(4, txtAddress.getText());
                ps.setInt(5, selectedId);
                ps.executeUpdate();
                showSuccess("Customer updated!"); clearForm(); loadCustomers();
            } catch (SQLException ex) { showError(ex.getMessage()); }
        });

        btnDelete.addActionListener(e -> {
            if (selectedId == -1) { showError("Select a customer!"); return; }
            if (JOptionPane.showConfirmDialog(this, "Delete this customer?", "Confirm",
                    JOptionPane.YES_NO_OPTION) == JOptionPane.YES_OPTION) {
                try (Connection con = DBConnection.getConnection()) {
                    PreparedStatement ps = con.prepareStatement("DELETE FROM customers WHERE
id=?");
                    ps.setInt(1, selectedId); ps.executeUpdate();
                    clearForm(); loadCustomers();
                } catch (SQLException ex) { showError(ex.getMessage()); }
            }
        });

        btnClear.addActionListener(e -> clearForm());

        btnSearch.addActionListener(e -> {
            String kw = "%" + txtSearch.getText() + "%";
            model.setRowCount(0);
            try (Connection con = DBConnection.getConnection()) {
                PreparedStatement ps = con.prepareStatement(
                    "SELECT * FROM customers WHERE name LIKE ? OR email LIKE ? OR phone LIKE ?");
                ps.setString(1, kw); ps.setString(2, kw); ps.setString(3, kw);
                ResultSet rs = ps.executeQuery();
                while (rs.next()) {
                    model.addRow(new Object[]{
                        rs.getInt("id"), rs.getString("name"), rs.getString("email"),
                        rs.getString("phone"), rs.getString("address")
                    });
                }
            } catch (SQLException ex) { showError(ex.getMessage()); }
        });
```

```
        btnRefresh.addActionListener(e -> loadCustomers());

        add(leftPanel, BorderLayout.WEST);
        add(rightPanel, BorderLayout.CENTER);
    }

    private void loadCustomers() {
        model.setRowCount(0);
        try (Connection con = DBConnection.getConnection();
             Statement st = con.createStatement();
             ResultSet rs = st.executeQuery("SELECT * FROM customers ORDER BY id DESC")) {
            while (rs.next()) {
                model.addRow(new Object[]{
                    rs.getInt("id"), rs.getString("name"), rs.getString("email"),
                    rs.getString("phone"), rs.getString("address")
                });
            }
        } catch (SQLException e) { showError(e.getMessage()); }
    }

    private void clearForm() {
        selectedId = -1;
        txtName.setText(""); txtEmail.setText("");
        txtPhone.setText(""); txtAddress.setText("");
        table.clearSelection();
    }

    private void showError(String msg) {
        JOptionPane.showMessageDialog(this, msg, "Error", JOptionPane.ERROR_MESSAGE);
    }

    private void showSuccess(String msg) {
        JOptionPane.showMessageDialog(this, msg, "Success",
JOptionPane.INFORMATION_MESSAGE);
    }
}
```

CustomerOrderFrame.java

```
package ecommercesystem;

import java.awt.*;
import java.sql.*;
import javax.swing.*;
import javax.swing.border.*;
import javax.swing.table.*;
```

```
public class CustomerOrderFrame extends JFrame {

    private JTable          table;
    private DefaultTableModel model;
    private int             customerId;

    public CustomerOrderFrame(int customerId) {
        setTitle("My Orders");
        setSize(860, 540);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());
        this.customerId = customerId;
        initComponents();
        loadOrders();
    }

    private void initComponents() {
        JPanel header = new JPanel(new BorderLayout());
        header.setBackground(new Color(44, 62, 80));
        header.setPreferredSize(new Dimension(0, 55));
        header.setBorder(new EmptyBorder(0, 20, 0, 20));
        JLabel lblH = new JLabel("My Orders");
        lblH.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 17));
        lblH.setForeground(Color.WHITE);
        header.add(lblH, BorderLayout.WEST);
        add(header, BorderLayout.NORTH);

        String[] cols = {"Order ID","Order Date","Total (Rs.)","Status"};
        model = new DefaultTableModel(cols, 0) {
            public boolean isCellEditable(int r, int c) { return false; }
        };
        table = new JTable(model);
        UIUtils.styleTable(table);

        table.getColumnModel().getColumn(3).setCellRenderer(
            new DefaultTableCellRenderer() {
                public Component getTableCellRendererComponent(JTable t, Object val,
                        boolean sel, boolean foc, int r, int c) {
                    Component comp = super.getTableCellRendererComponent(
                        t, val, sel, foc, r, c);
                    setBorder(new EmptyBorder(0, 10, 0, 10));
                    setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 12));
                    if (!sel) {
                        comp.setBackground(UIUtils.statusColor(
                            val != null ? val.toString() : ""));
                        setForeground(new Color(44, 62, 80));
```

```
            }
            setHorizontalAlignment(CENTER);
            return comp;
        }
    });

    JScrollPane scroll = new JScrollPane(table);
    scroll.setBorder(new EmptyBorder(14, 14, 0, 14));
    scroll.getViewport().setBackground(Color.WHITE);

    // ── BOTTOM BAR ──
    JPanel bottomBar = new JPanel(new BorderLayout());
    bottomBar.setBackground(Color.WHITE);
    bottomBar.setBorder(new CompoundBorder(
        new MatteBorder(1, 0, 0, 0, new Color(230, 230, 230)),
        new EmptyBorder(12, 16, 12, 16)));

    JLabel lblInfo = new JLabel(
        "You can cancel orders that are not yet delivered.");
    lblInfo.setFont(new Font(UIUtils.EMOJI_FONT, Font.PLAIN, 11));
    lblInfo.setForeground(new Color(149, 165, 166));

    JButton btnCancel  = roundedBtn("Cancel Order", new Color(231, 76, 60),  new
Dimension(120, 34));
    JButton btnRefresh = roundedBtn("Refresh",     new Color(44,  62,  80), new
Dimension(88,  34));

    JPanel rightBtns = new JPanel(new FlowLayout(FlowLayout.RIGHT, 8, 0));
    rightBtns.setOpaque(false);
    rightBtns.add(btnCancel);
    rightBtns.add(btnRefresh);

    bottomBar.add(lblInfo,    BorderLayout.WEST);
    bottomBar.add(rightBtns, BorderLayout.EAST);

    JPanel center = new JPanel(new BorderLayout());
    center.setBackground(new Color(245, 247, 250));
    center.add(scroll,    BorderLayout.CENTER);
    center.add(bottomBar, BorderLayout.SOUTH);
    add(center, BorderLayout.CENTER);

    btnRefresh.addActionListener(e -> loadOrders());

    btnCancel.addActionListener(e -> {
        int row = table.getSelectedRow();
        if (row < 0) {
            JOptionPane.showMessageDialog(this, "Select an order to cancel!"); return;
```

```
        }
        int    orderId = (int)    model.getValueAt(row, 0);
        String status  = (String) model.getValueAt(row, 3);
        if ("delivered".equalsIgnoreCase(status) ||
            "cancelled".equalsIgnoreCase(status)) {
            JOptionPane.showMessageDialog(this,
                "Cannot cancel a " + status + " order!"); return;
        }
        if (JOptionPane.showConfirmDialog(this,
                "Cancel order #" + orderId + "?", "Confirm",
                JOptionPane.YES_NO_OPTION) == JOptionPane.YES_OPTION) {
            try (Connection con = DBConnection.getConnection()) {
                PreparedStatement ps = con.prepareStatement(
                    "UPDATE orders SET status='cancelled' WHERE id=?");
                ps.setInt(1, orderId);
                ps.executeUpdate();
                JOptionPane.showMessageDialog(this,
                    "Order #" + orderId + " cancelled.",
                    "Cancelled", JOptionPane.INFORMATION_MESSAGE);
                loadOrders();
            } catch (SQLException ex) {
                JOptionPane.showMessageDialog(this, ex.getMessage());
            }
        }
    });
}

private void loadOrders() {
    model.setRowCount(0);
    if (customerId == -1) {
        JOptionPane.showMessageDialog(this,
            "Customer record not found.", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }
    try (Connection con = DBConnection.getConnection();
         PreparedStatement ps = con.prepareStatement(
            "SELECT id, " +
            "TO_CHAR(order_date,'DD-MON-YYYY HH24:MI') as odate, " +
            "total_amount, status " +
            "FROM orders WHERE customer_id=? ORDER BY id DESC")) {
        ps.setInt(1, customerId);
        ResultSet rs  = ps.executeQuery();
        boolean   any = false;
        while (rs.next()) {
            any = true;
            model.addRow(new Object[]{
                rs.getInt("id"),
```

```
                rs.getString("odate"),
                "Rs." + String.format("%.2f", rs.getDouble("total_amount")),
                rs.getString("status")
            });
        }
        if (!any) {
            JOptionPane.showMessageDialog(this,
                "You have not placed any orders yet.",
                "No Orders", JOptionPane.INFORMATION_MESSAGE);
        }
    } catch (SQLException e) {
        JOptionPane.showMessageDialog(this, e.getMessage());
    }
}

private JButton roundedBtn(String text, Color bg, Dimension size) {
    JButton btn = new JButton(text) {
        protected void paintComponent(Graphics g) {
            Graphics2D g2 = (Graphics2D) g.create();
            g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
                                RenderingHints.VALUE_ANTIALIAS_ON);
            g2.setColor(getBackground());
            g2.fillRoundRect(0, 0, getWidth(), getHeight(), 12, 12);
            FontMetrics fm = g2.getFontMetrics();
            g2.setColor(getForeground());
            g2.setFont(getFont());
            int x = (getWidth()  - fm.stringWidth(getText())) / 2;
            int y = (getHeight() + fm.getAscent() - fm.getDescent()) / 2;
            g2.drawString(getText(), x, y);
            g2.dispose();
        }
    };
    btn.setBackground(bg);
    btn.setForeground(Color.WHITE);
    btn.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 12));
    btn.setFocusPainted(false);
    btn.setBorderPainted(false);
    btn.setContentAreaFilled(false);
    btn.setOpaque(false);
    btn.setCursor(new Cursor(Cursor.HAND_CURSOR));
    btn.setPreferredSize(size);
    btn.addMouseListener(new java.awt.event.MouseAdapter() {
        public void mouseEntered(java.awt.event.MouseEvent e) {
            btn.setBackground(bg.darker()); btn.repaint();
        }
        public void mouseExited(java.awt.event.MouseEvent e) {
            btn.setBackground(bg); btn.repaint();
```

```
        }
    });
    return btn;
  }
}
```

DashboardFram.java

```
package ecommercesystem;

import java.awt.*;
import java.awt.event.ActionListener;
import java.sql.*;
import javax.swing.*;
import javax.swing.border.*;

public class DashboardFrame extends JFrame {

  private String username, role;
  private int customerId = -1;

  public DashboardFrame(String username, String role, int customerId) {
    this.username   = username;
    this.role       = role;
    this.customerId = customerId;
    if (this.customerId == -1 && "customer".equalsIgnoreCase(role)) {
      this.customerId = resolveCustomerId(username);
    }
    setTitle("Dashboard");
    setSize(860, 540);
    setLocationRelativeTo(null);
    setDefaultCloseOperation(EXIT_ON_CLOSE);
    initComponents();
  }

  public DashboardFrame(String username, String role) {
    this(username, role, -1);
  }

  private int resolveCustomerId(String username) {
    try (Connection con = DBConnection.getConnection();
         PreparedStatement ps = con.prepareStatement(
           "SELECT c.id FROM customers c " +
           "JOIN users u ON LOWER(c.email) = LOWER(u.email) " +
           "WHERE u.username = ?")) {
        ps.setString(1, username);
```

```
            ResultSet rs = ps.executeQuery();
            if (rs.next()) return rs.getInt(1);
        } catch (SQLException e) {}
        return -1;
    }

    private void initComponents() {
        JPanel root = new JPanel(new BorderLayout(0, 0));
        root.setBackground(new Color(245, 247, 250));
        setContentPane(root);

        // ── TOPBAR ──
        JPanel topBar = new JPanel(new BorderLayout());
        topBar.setBackground(new Color(44, 62, 80));
        topBar.setPreferredSize(new Dimension(0, 56));
        topBar.setBorder(new EmptyBorder(0, 24, 0, 16));

        JLabel lblTitle = new JLabel("E-Commerce System");
        lblTitle.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 17));
        lblTitle.setForeground(Color.WHITE);

        JPanel topRight = new JPanel(new FlowLayout(FlowLayout.RIGHT, 8, 12));
        topRight.setOpaque(false);

        JLabel lblBadge = new JLabel(role.toUpperCase());
        lblBadge.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 10));
        lblBadge.setForeground(new Color(44, 62, 80));
        lblBadge.setBackground("admin".equalsIgnoreCase(role)
            ? new Color(241, 196, 15)
            : new Color(46, 204, 113));
        lblBadge.setOpaque(true);
        lblBadge.setBorder(new EmptyBorder(3, 8, 3, 8));

        JLabel lblUser = new JLabel(username);
        lblUser.setFont(new Font(UIUtils.EMOJI_FONT, Font.PLAIN, 13));
        lblUser.setForeground(new Color(189, 195, 199));

        // ── ROUNDED SIGN OUT BUTTON ──
        JButton btnLogout = roundedBtn("Sign out",
            new Color(231, 76, 60), new Dimension(88, 30));
        btnLogout.addActionListener(e -> {
            dispose(); new LoginFrame().setVisible(true);
        });

        topRight.add(lblBadge);
        topRight.add(lblUser);
        topRight.add(Box.createHorizontalStrut(4));
```

```
        topRight.add(btnLogout);

        topBar.add(lblTitle,  BorderLayout.WEST);
        topBar.add(topRight,  BorderLayout.EAST);

        // — BODY —
        JPanel body = new JPanel(new BorderLayout(0, 0));
        body.setBackground(new Color(245, 247, 250));
        body.setBorder(new EmptyBorder(24, 24, 24, 24));

        JLabel lblWelcome = new JLabel(
            "admin".equalsIgnoreCase(role)
            ? "Welcome back, " + username + "!"
            : "Hello, " + username + "!");
        lblWelcome.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 20));
        lblWelcome.setForeground(new Color(44, 62, 80));
        lblWelcome.setBorder(new EmptyBorder(0, 0, 18, 0));

        // — STATS —
        JPanel statsRow = new JPanel(new GridLayout(1,
            "admin".equalsIgnoreCase(role) ? 4 : 3, 14, 0));
        statsRow.setOpaque(false);

        if ("admin".equalsIgnoreCase(role)) {
            statsRow.add(statCard("Products",   getCount("SELECT COUNT(*) FROM products"),  new
Color(52,152,219), "Total products in store"));
            statsRow.add(statCard("Customers",  getCount("SELECT COUNT(*) FROM customers"),
new Color(39,174,96),  "Registered customers"));
            statsRow.add(statCard("Orders",    getCount("SELECT COUNT(*) FROM orders"),    new
Color(230,126,34), "All time orders"));
            statsRow.add(statCard("Revenue",    "Rs." + getRevenue(),                    new
Color(155,89,182), "Excluding cancelled"));
        } else {
            statsRow.add(statCard("Products",    getCount("SELECT COUNT(*) FROM products"), new
Color(52,152,219), "Available in store"));
            statsRow.add(statCard("My Orders",   String.valueOf(getCustomerOrderCount()),   new
Color(230,126,34), "Orders placed by you"));
            statsRow.add(statCard("Total Spent", "Rs." + getCustomerTotalSpent(),          new
Color(39,174,96),  "Your total spending"));
        }

        JLabel lblSection = new JLabel(
            "admin".equalsIgnoreCase(role) ? "Management" : "Quick Actions");
        lblSection.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 13));
        lblSection.setForeground(new Color(149, 165, 166));
        lblSection.setBorder(new EmptyBorder(22, 0, 10, 0));
```

```
        // ── NAV ──
        JPanel navRow = new JPanel(new GridLayout(1,
            "admin".equalsIgnoreCase(role) ? 3 : 2, 14, 0));
        navRow.setOpaque(false);

        if ("admin".equalsIgnoreCase(role)) {
            navRow.add(navCard("Products",  "Add, edit, delete products",    new Color(52,152,219),
e -> new ProductFrame().setVisible(true)));
            navRow.add(navCard("Customers", "View and manage customers",     new
Color(39,174,96),  e -> new CustomerFrame().setVisible(true)));
            navRow.add(navCard("Orders",    "Track and update orders",       new Color(230,126,34), e
-> new OrderFrame().setVisible(true)));
        } else {
            navRow.add(navCard("Shop",      "Browse and buy products",       new Color(52,152,219),
e -> new CartFrame(customerId).setVisible(true)));
            navRow.add(navCard("My Orders", "View your order history",       new Color(230,126,34),
e -> new CustomerOrderFrame(customerId).setVisible(true)));
        }

        JLabel lblFooter = new JLabel("E-Commerce Management System  |  v1.0");
        lblFooter.setFont(new Font(UIUtils.EMOJI_FONT, Font.PLAIN, 10));
        lblFooter.setForeground(new Color(189, 195, 199));
        lblFooter.setBorder(new EmptyBorder(16, 0, 0, 0));

        JPanel centerBody = new JPanel();
        centerBody.setLayout(new BoxLayout(centerBody, BoxLayout.Y_AXIS));
        centerBody.setOpaque(false);
        lblWelcome.setAlignmentX(Component.LEFT_ALIGNMENT);
        statsRow.setAlignmentX(Component.LEFT_ALIGNMENT);
        lblSection.setAlignmentX(Component.LEFT_ALIGNMENT);
        navRow.setAlignmentX(Component.LEFT_ALIGNMENT);
        lblFooter.setAlignmentX(Component.LEFT_ALIGNMENT);
        centerBody.add(lblWelcome);
        centerBody.add(statsRow);
        centerBody.add(lblSection);
        centerBody.add(navRow);
        centerBody.add(lblFooter);

        body.add(centerBody, BorderLayout.CENTER);
        root.add(topBar, BorderLayout.NORTH);
        root.add(body,   BorderLayout.CENTER);
    }

    private JPanel statCard(String label, String value, Color accent, String sub) {
        JPanel card = new JPanel(new BorderLayout());
        card.setBackground(Color.WHITE);
        card.setBorder(new CompoundBorder(
```

```
            new LineBorder(new Color(230, 230, 230), 1),
            new EmptyBorder(16, 18, 16, 18)));
        JPanel accentLine = new JPanel();
        accentLine.setBackground(accent);
        accentLine.setPreferredSize(new Dimension(0, 3));
        card.add(accentLine, BorderLayout.NORTH);
        JPanel content = new JPanel();
        content.setLayout(new BoxLayout(content, BoxLayout.Y_AXIS));
        content.setOpaque(false);
        content.setBorder(new EmptyBorder(10, 0, 0, 0));
        JLabel lblValue = new JLabel(value);
        lblValue.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 24));
        lblValue.setForeground(new Color(44, 62, 80));
        JLabel lblLabel = new JLabel(label);
        lblLabel.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 13));
        lblLabel.setForeground(new Color(44, 62, 80));
        lblLabel.setBorder(new EmptyBorder(4, 0, 2, 0));
        JLabel lblSub = new JLabel(sub);
        lblSub.setFont(new Font(UIUtils.EMOJI_FONT, Font.PLAIN, 10));
        lblSub.setForeground(new Color(149, 165, 166));
        content.add(lblValue);
        content.add(lblLabel);
        content.add(lblSub);
        card.add(content, BorderLayout.CENTER);
        return card;
    }

    private JPanel navCard(String title, String subtitle,
            Color accent, ActionListener al) {
        JPanel card = new JPanel(new BorderLayout());
        card.setBackground(Color.WHITE);
        card.setBorder(new CompoundBorder(
            new LineBorder(new Color(230, 230, 230), 1),
            new EmptyBorder(18, 20, 18, 20)));
        card.setCursor(new Cursor(Cursor.HAND_CURSOR));
        JPanel bar = new JPanel();
        bar.setBackground(accent);
        bar.setPreferredSize(new Dimension(4, 0));
        card.add(bar, BorderLayout.WEST);
        JPanel content = new JPanel();
        content.setLayout(new BoxLayout(content, BoxLayout.Y_AXIS));
        content.setOpaque(false);
        content.setBorder(new EmptyBorder(0, 14, 0, 0));
        JLabel lblTitle = new JLabel(title);
        lblTitle.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 15));
        lblTitle.setForeground(new Color(44, 62, 80));
        JLabel lblSub = new JLabel(subtitle);
```

```
    lblSub.setFont(new Font(UIUtils.EMOJI_FONT, Font.PLAIN, 11));
    lblSub.setForeground(new Color(149, 165, 166));
    lblSub.setBorder(new EmptyBorder(4, 0, 0, 0));
    JLabel lblArrow = new JLabel("→");
    lblArrow.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 16));
    lblArrow.setForeground(accent);
    content.add(lblTitle);
    content.add(lblSub);
    card.add(content,  BorderLayout.CENTER);
    card.add(lblArrow, BorderLayout.EAST);
    card.addMouseListener(new java.awt.event.MouseAdapter() {
      public void mouseEntered(java.awt.event.MouseEvent e) {
        card.setBackground(new Color(248, 250, 252)); card.repaint();
      }
      public void mouseExited(java.awt.event.MouseEvent e) {
        card.setBackground(Color.WHITE); card.repaint();
      }
      public void mouseClicked(java.awt.event.MouseEvent e) {
        al.actionPerformed(null);
      }
    });
    return card;
  }

  // ── ROUNDED BUTTON HELPER ──
  private JButton roundedBtn(String text, Color bg, Dimension size) {
    JButton btn = new JButton(text) {
      protected void paintComponent(Graphics g) {
        Graphics2D g2 = (Graphics2D) g.create();
        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
                RenderingHints.VALUE_ANTIALIAS_ON);
        g2.setColor(getBackground());
        g2.fillRoundRect(0, 0, getWidth(), getHeight(), 12, 12);
        FontMetrics fm = g2.getFontMetrics();
        g2.setColor(getForeground());
        g2.setFont(getFont());
        int x = (getWidth()  - fm.stringWidth(getText())) / 2;
        int y = (getHeight() + fm.getAscent() - fm.getDescent()) / 2;
        g2.drawString(getText(), x, y);
        g2.dispose();
      }
    };
    btn.setBackground(bg);
    btn.setForeground(Color.WHITE);
    btn.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 12));
    btn.setFocusPainted(false);
    btn.setBorderPainted(false);
```

```
        btn.setContentAreaFilled(false);
        btn.setOpaque(false);
        btn.setCursor(new Cursor(Cursor.HAND_CURSOR));
        btn.setPreferredSize(size);
        btn.addMouseListener(new java.awt.event.MouseAdapter() {
            public void mouseEntered(java.awt.event.MouseEvent e) {
                btn.setBackground(bg.darker()); btn.repaint();
            }
            public void mouseExited(java.awt.event.MouseEvent e) {
                btn.setBackground(bg); btn.repaint();
            }
        });
        return btn;
    }

    private String getCount(String sql) {
        try (Connection con = DBConnection.getConnection();
             Statement st  = con.createStatement();
             ResultSet rs  = st.executeQuery(sql)) {
            if (rs.next()) return String.valueOf(rs.getInt(1));
        } catch (SQLException e) {}
        return "0";
    }

    private String getRevenue() {
        try (Connection con = DBConnection.getConnection();
             Statement st  = con.createStatement();
             ResultSet rs  = st.executeQuery(
                 "SELECT NVL(SUM(total_amount),0) FROM orders WHERE status != 'cancelled'")) {
            if (rs.next()) return String.format("%.0f", rs.getDouble(1));
        } catch (SQLException e) {}
        return "0";
    }

    private int getCustomerOrderCount() {
        if (customerId == -1) return 0;
        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(
                 "SELECT COUNT(*) FROM orders WHERE customer_id=?")) {
            ps.setInt(1, customerId);
            ResultSet rs = ps.executeQuery();
            if (rs.next()) return rs.getInt(1);
        } catch (SQLException e) {}
        return 0;
    }

    private String getCustomerTotalSpent() {
```

```
        if (customerId == -1) return "0";
        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(
               "SELECT NVL(SUM(total_amount),0) FROM orders WHERE customer_id=?")) {
            ps.setInt(1, customerId);
            ResultSet rs = ps.executeQuery();
            if (rs.next()) return String.format("%.0f", rs.getDouble(1));
        } catch (SQLException e) {}
        return "0";
    }
}
```

LoginFrame.java

```
package ecommercesystem;

import java.awt.*;
import java.awt.event.*;
import java.sql.*;
import javax.swing.*;
import javax.swing.border.*;

public class LoginFrame extends JFrame {

    private JTextField     txtUsername;
    private JPasswordField txtPassword;
    private JLabel         lblStatus;

    public LoginFrame() {
        setTitle("E-Commerce System");
        setSize(420, 520);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setResizable(false);
        initComponents();
    }

    private void initComponents() {
        JPanel root = new JPanel(new GridBagLayout());
        root.setBackground(new Color(245, 247, 250));
        setContentPane(root);

        JPanel card = new JPanel();
        card.setLayout(new BoxLayout(card, BoxLayout.Y_AXIS));
        card.setBackground(Color.WHITE);
        card.setBorder(new CompoundBorder(
```

```
        new LineBorder(new Color(230, 230, 230), 1),
        new EmptyBorder(36, 36, 36, 36)));
    card.setPreferredSize(new Dimension(350, 440));

    JPanel accent = new JPanel();
    accent.setBackground(new Color(52, 152, 219));
    accent.setPreferredSize(new Dimension(0, 4));
    accent.setMaximumSize(new Dimension(Integer.MAX_VALUE, 4));

    JLabel lblTitle = new JLabel("Sign In");
    lblTitle.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 22));
    lblTitle.setForeground(new Color(44, 62, 80));
    lblTitle.setAlignmentX(Component.LEFT_ALIGNMENT);

    JLabel lblSub = new JLabel("Enter your credentials to continue");
    lblSub.setFont(new Font(UIUtils.EMOJI_FONT, Font.PLAIN, 12));
    lblSub.setForeground(new Color(149, 165, 166));
    lblSub.setAlignmentX(Component.LEFT_ALIGNMENT);

    txtUsername = UIUtils.createTextField();
    txtUsername.setMaximumSize(new Dimension(Integer.MAX_VALUE, 38));
    txtUsername.setAlignmentX(Component.LEFT_ALIGNMENT);

    txtPassword = UIUtils.createPasswordField();
    txtPassword.setMaximumSize(new Dimension(Integer.MAX_VALUE, 38));
    txtPassword.setAlignmentX(Component.LEFT_ALIGNMENT);

    JLabel lblU = fieldLabel("Username");
    JLabel lblP = fieldLabel("Password");

    // ── ROUNDED BUTTONS ──
    JButton btnLogin    = roundedBtn("Sign In",        new Color(52, 152, 219));
    JButton btnRegister = roundedBtn("Create an account", new Color(39, 174, 96));
    btnLogin.setMaximumSize(new Dimension(Integer.MAX_VALUE, 44));
    btnLogin.setAlignmentX(Component.LEFT_ALIGNMENT);
    btnRegister.setMaximumSize(new Dimension(Integer.MAX_VALUE, 40));
    btnRegister.setAlignmentX(Component.LEFT_ALIGNMENT);

    lblStatus = new JLabel(" ");
    lblStatus.setFont(new Font(UIUtils.EMOJI_FONT, Font.PLAIN, 11));
    lblStatus.setForeground(new Color(231, 76, 60));
    lblStatus.setAlignmentX(Component.LEFT_ALIGNMENT);

    btnLogin.addActionListener(e -> doLogin());
    btnRegister.addActionListener(e -> {
        dispose(); new RegisterFrame().setVisible(true);
    });
```

```
        txtPassword.addKeyListener(new KeyAdapter() {
            public void keyPressed(KeyEvent e) {
                if (e.getKeyCode() == KeyEvent.VK_ENTER) doLogin();
            }
        });

        card.add(accent);
        card.add(Box.createVerticalStrut(24));
        card.add(lblTitle);
        card.add(Box.createVerticalStrut(4));
        card.add(lblSub);
        card.add(Box.createVerticalStrut(24));
        card.add(lblU);
        card.add(Box.createVerticalStrut(5));
        card.add(txtUsername);
        card.add(Box.createVerticalStrut(14));
        card.add(lblP);
        card.add(Box.createVerticalStrut(5));
        card.add(txtPassword);
        card.add(Box.createVerticalStrut(20));
        card.add(btnLogin);
        card.add(Box.createVerticalStrut(8));
        card.add(btnRegister);
        card.add(Box.createVerticalStrut(10));
        card.add(lblStatus);

        root.add(card);
    }

    private void doLogin() {
        String username = txtUsername.getText().trim();
        String password = new String(txtPassword.getPassword()).trim();
        if (username.isEmpty() || password.isEmpty()) {
            lblStatus.setText("Please enter username and password!"); return;
        }
        try (Connection con = DBConnection.getConnection()) {
            PreparedStatement ps = con.prepareStatement(
                "SELECT * FROM users WHERE username=? AND password=?");
            ps.setString(1, username); ps.setString(2, password);
            ResultSet rs = ps.executeQuery();
            if (rs.next()) {
                String role  = rs.getString("role");
                String email = rs.getString("email");
                int custId   = -1;
                if ("customer".equalsIgnoreCase(role) && email != null) {
                    try (PreparedStatement cs = con.prepareStatement(
                            "SELECT id FROM customers WHERE LOWER(email)=LOWER(?)")) {
```

```
                cs.setString(1, email);
                ResultSet cr = cs.executeQuery();
                if (cr.next()) custId = cr.getInt(1);
              } catch (SQLException ex) {}
            }
            dispose();
            new DashboardFrame(username, role, custId).setVisible(true);
          } else {
            lblStatus.setText("Invalid username or password!");
            txtPassword.setText("");
          }
        } catch (SQLException ex) {
          lblStatus.setText("DB Error: " + ex.getMessage());
        }
    }

    private JLabel fieldLabel(String text) {
        JLabel l = new JLabel(text);
        l.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 12));
        l.setForeground(new Color(44, 62, 80));
        l.setAlignmentX(Component.LEFT_ALIGNMENT);
        return l;
    }

    private JButton roundedBtn(String text, Color bg) {
        JButton btn = new JButton(text) {
            protected void paintComponent(Graphics g) {
                Graphics2D g2 = (Graphics2D) g.create();
                g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
                        RenderingHints.VALUE_ANTIALIAS_ON);
                g2.setColor(getBackground());
                g2.fillRoundRect(0, 0, getWidth(), getHeight(), 12, 12);
                FontMetrics fm = g2.getFontMetrics();
                g2.setColor(getForeground());
                g2.setFont(getFont());
                int x = (getWidth()  - fm.stringWidth(getText())) / 2;
                int y = (getHeight() + fm.getAscent() - fm.getDescent()) / 2;
                g2.drawString(getText(), x, y);
                g2.dispose();
            }
        };
        btn.setBackground(bg);
        btn.setForeground(Color.WHITE);
        btn.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 13));
        btn.setFocusPainted(false);
        btn.setBorderPainted(false);
        btn.setContentAreaFilled(false);
```

```
        btn.setOpaque(false);
        btn.setCursor(new Cursor(Cursor.HAND_CURSOR));
        btn.setBorder(new EmptyBorder(10, 0, 10, 0));
        btn.addMouseListener(new java.awt.event.MouseAdapter() {
            public void mouseEntered(java.awt.event.MouseEvent e) {
                btn.setBackground(bg.darker()); btn.repaint();
            }
            public void mouseExited(java.awt.event.MouseEvent e) {
                btn.setBackground(bg); btn.repaint();
            }
        });
        return btn;
    }
}
```

OrderFrame.java

```
package ecommercesystem;

import java.awt.*;
import java.sql.*;
import javax.swing.*;
import javax.swing.border.*;
import javax.swing.table.*;

public class OrderFrame extends JFrame {

    private JTable           table;
    private DefaultTableModel model;
    private JComboBox<String> cmbStatus;
    private int              selectedId = -1;

    public OrderFrame() {
        setTitle("Order Management");
        setSize(980, 600);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());
        initComponents();
        loadOrders();
    }

    private void initComponents() {
        JPanel header = new JPanel(new BorderLayout());
        header.setBackground(new Color(44, 62, 80));
        header.setPreferredSize(new Dimension(0, 55));
        header.setBorder(new EmptyBorder(0, 20, 0, 20));
```

```
JLabel lblH = new JLabel("Order Management");
lblH.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 17));
lblH.setForeground(Color.WHITE);
header.add(lblH, BorderLayout.WEST);
add(header, BorderLayout.NORTH);

String[] cols = {"Order ID","Customer","Order Date","Total (Rs.)","Status"};
model = new DefaultTableModel(cols, 0) {
   public boolean isCellEditable(int r, int c) { return false; }
};
table = new JTable(model);
UIUtils.styleTable(table);

table.getColumnModel().getColumn(4).setCellRenderer(
   new DefaultTableCellRenderer() {
      public Component getTableCellRendererComponent(JTable t, Object val,
            boolean sel, boolean foc, int r, int c) {
         Component comp = super.getTableCellRendererComponent(
            t, val, sel, foc, r, c);
         setBorder(new EmptyBorder(0, 10, 0, 10));
         setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 12));
         if (!sel) {
            comp.setBackground(UIUtils.statusColor(
               val != null ? val.toString() : ""));
            setForeground(new Color(44, 62, 80));
         }
         setHorizontalAlignment(CENTER);
         return comp;
      }
   });

table.getSelectionModel().addListSelectionListener(e -> {
   int row = table.getSelectedRow();
   if (row >= 0) {
      selectedId = (int) model.getValueAt(row, 0);
      cmbStatus.setSelectedItem(model.getValueAt(row, 4).toString());
   }
});

JScrollPane scroll = new JScrollPane(table);
scroll.setBorder(new EmptyBorder(14, 14, 0, 14));
scroll.getViewport().setBackground(Color.WHITE);

// ── BOTTOM BAR ──
JPanel bottomBar = new JPanel(new BorderLayout());
bottomBar.setBackground(Color.WHITE);
bottomBar.setBorder(new CompoundBorder(
```

```
        new MatteBorder(1, 0, 0, 0, new Color(230, 230, 230)),
        new EmptyBorder(12, 16, 12, 16)));

    JPanel leftBar = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 0));
    leftBar.setOpaque(false);

    JLabel lblS = new JLabel("Change status:");
    lblS.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 12));
    lblS.setForeground(new Color(149, 165, 166));

    String[] statuses = {"pending","processing","shipped","delivered","cancelled"};
    cmbStatus = new JComboBox<>(statuses);
    cmbStatus.setFont(new Font(UIUtils.EMOJI_FONT, Font.PLAIN, 13));
    cmbStatus.setPreferredSize(new Dimension(150, 34));
    cmbStatus.setBorder(new LineBorder(new Color(220, 220, 220), 1));

    JButton btnUpdate  = roundedBtn("Update",  new Color(230, 126, 34), new Dimension(90,
34));
    JButton btnRefresh = roundedBtn("Refresh", new Color(44,  62,  80), new Dimension(88,
34));

    leftBar.add(lblS);
    leftBar.add(cmbStatus);
    leftBar.add(btnUpdate);
    leftBar.add(btnRefresh);

    // legend
    JPanel legend = new JPanel(new FlowLayout(FlowLayout.RIGHT, 8, 0));
    legend.setOpaque(false);
    legend.add(legendDot("Pending",    Color.WHITE));
    legend.add(legendDot("Processing", UIUtils.statusColor("processing")));
    legend.add(legendDot("Shipped",    UIUtils.statusColor("shipped")));
    legend.add(legendDot("Delivered",  UIUtils.statusColor("delivered")));
    legend.add(legendDot("Cancelled",  UIUtils.statusColor("cancelled")));

    bottomBar.add(leftBar, BorderLayout.WEST);
    bottomBar.add(legend,  BorderLayout.EAST);

    JPanel center = new JPanel(new BorderLayout());
    center.setBackground(new Color(245, 247, 250));
    center.add(scroll,    BorderLayout.CENTER);
    center.add(bottomBar, BorderLayout.SOUTH);
    add(center, BorderLayout.CENTER);

    btnUpdate.addActionListener(e -> {
        if (selectedId == -1) {
            JOptionPane.showMessageDialog(this, "Select an order!"); return;
```

```
        }
        try (Connection con = DBConnection.getConnection()) {
            PreparedStatement ps = con.prepareStatement(
                "UPDATE orders SET status=? WHERE id=?");
            ps.setString(1, (String) cmbStatus.getSelectedItem());
            ps.setInt(2, selectedId);
            ps.executeUpdate();
            JOptionPane.showMessageDialog(this, "Status updated!",
                "Success", JOptionPane.INFORMATION_MESSAGE);
            loadOrders();
        } catch (SQLException ex) {
            JOptionPane.showMessageDialog(this, ex.getMessage());
        }
    });

    btnRefresh.addActionListener(e -> loadOrders());
}

private JPanel legendDot(String label, Color color) {
    JPanel p = new JPanel(new FlowLayout(FlowLayout.LEFT, 4, 0));
    p.setOpaque(false);
    JPanel dot = new JPanel();
    dot.setBackground(color);
    dot.setPreferredSize(new Dimension(12, 12));
    dot.setBorder(new LineBorder(new Color(200, 200, 200), 1));
    JLabel lbl = new JLabel(label);
    lbl.setFont(new Font(UIUtils.EMOJI_FONT, Font.PLAIN, 10));
    lbl.setForeground(new Color(149, 165, 166));
    p.add(dot); p.add(lbl);
    return p;
}

private void loadOrders() {
    model.setRowCount(0);
    try (Connection con = DBConnection.getConnection();
         Statement st = con.createStatement();
         ResultSet rs = st.executeQuery(
            "SELECT o.id, NVL(c.name,'Unknown') as cname, " +
            "TO_CHAR(o.order_date,'DD-MON-YYYY HH24:MI') as odate, " +
            "o.total_amount, o.status " +
            "FROM orders o LEFT JOIN customers c ON o.customer_id=c.id " +
            "ORDER BY o.id DESC")) {
        while (rs.next()) {
            model.addRow(new Object[]{
                rs.getInt("id"), rs.getString("cname"),
                rs.getString("odate"), rs.getDouble("total_amount"),
                rs.getString("status")
```

```
            });
        }
      } catch (SQLException e) {
        JOptionPane.showMessageDialog(this, e.getMessage());
      }
    }

    private JButton roundedBtn(String text, Color bg, Dimension size) {
      JButton btn = new JButton(text) {
        protected void paintComponent(Graphics g) {
          Graphics2D g2 = (Graphics2D) g.create();
          g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
                  RenderingHints.VALUE_ANTIALIAS_ON);
          g2.setColor(getBackground());
          g2.fillRoundRect(0, 0, getWidth(), getHeight(), 12, 12);
          FontMetrics fm = g2.getFontMetrics();
          g2.setColor(getForeground());
          g2.setFont(getFont());
          int x = (getWidth()  - fm.stringWidth(getText())) / 2;
          int y = (getHeight() + fm.getAscent() - fm.getDescent()) / 2;
          g2.drawString(getText(), x, y);
          g2.dispose();
        }
      };
      btn.setBackground(bg);
      btn.setForeground(Color.WHITE);
      btn.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 12));
      btn.setFocusPainted(false);
      btn.setBorderPainted(false);
      btn.setContentAreaFilled(false);
      btn.setOpaque(false);
      btn.setCursor(new Cursor(Cursor.HAND_CURSOR));
      btn.setPreferredSize(size);
      btn.addMouseListener(new java.awt.event.MouseAdapter() {
        public void mouseEntered(java.awt.event.MouseEvent e) {
          btn.setBackground(bg.darker()); btn.repaint();
        }
        public void mouseExited(java.awt.event.MouseEvent e) {
          btn.setBackground(bg); btn.repaint();
        }
      });
      return btn;
    }
}
```

ProductFrame.java

```
package ecommercesystem;

import java.awt.*;
import java.sql.*;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.table.DefaultTableModel;

public class ProductFrame extends JFrame {

    private JTable table;
    private DefaultTableModel model;
    private JTextField txtName, txtDesc, txtPrice, txtStock, txtCategory, txtSearch;
    private int selectedId = -1;

    public ProductFrame() {
        setTitle("Product Management");
        setSize(1000, 620);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());
        initComponents();
        loadProducts();
    }

    private void initComponents() {
        add(UIUtils.createHeader("📦  Product Management"), BorderLayout.NORTH);

        // LEFT
        JPanel leftPanel = new JPanel(new BorderLayout());
        leftPanel.setBackground(UIUtils.COLOR_BACKGROUND);
        leftPanel.setBorder(new EmptyBorder(15, 15, 15, 8));
        leftPanel.setPreferredSize(new Dimension(300, 0));

        JPanel card = UIUtils.createCard();
        card.setLayout(new GridBagLayout());

        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(6, 5, 6, 5);
        gbc.fill = GridBagConstraints.HORIZONTAL;
        gbc.weightx = 1;

        txtName     = UIUtils.createTextField();
        txtDesc     = UIUtils.createTextField();
        txtPrice    = UIUtils.createTextField();
        txtStock    = UIUtils.createTextField();
```

```
txtCategory = UIUtils.createTextField();

String[]     labels = {"Product Name", "Description", "Price (₹)", "Stock", "Category"};
JTextField[] fields = {txtName, txtDesc, txtPrice, txtStock, txtCategory};

for (int i = 0; i < labels.length; i++) {
   gbc.gridy = i * 2; gbc.gridx = 0;
   JLabel lbl = new JLabel(labels[i]);
   lbl.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 12));
   lbl.setForeground(UIUtils.COLOR_TEXT);
   card.add(lbl, gbc);
   gbc.gridy = i * 2 + 1;
   fields[i].setPreferredSize(new Dimension(240, 34));
   card.add(fields[i], gbc);
}

JPanel btnPanel = new JPanel(new GridLayout(2, 2, 8, 8));
btnPanel.setOpaque(false);
btnPanel.setBorder(new EmptyBorder(15, 0, 0, 0));

JButton btnAdd    = UIUtils.createButton("➕ Add",    UIUtils.COLOR_SUCCESS);
JButton btnUpdate = UIUtils.createButton("✏ Update",  UIUtils.COLOR_WARNING);
JButton btnDelete = UIUtils.createButton("🗑 Delete",  UIUtils.COLOR_DANGER);
JButton btnClear  = UIUtils.createButton("Clear",     UIUtils.COLOR_SUBTLE);

btnPanel.add(btnAdd); btnPanel.add(btnUpdate);
btnPanel.add(btnDelete); btnPanel.add(btnClear);

gbc.gridy = labels.length * 2;
card.add(btnPanel, gbc);
leftPanel.add(card, BorderLayout.NORTH);

// RIGHT
JPanel rightPanel = new JPanel(new BorderLayout());
rightPanel.setBackground(UIUtils.COLOR_BACKGROUND);
rightPanel.setBorder(new EmptyBorder(15, 8, 15, 15));

JPanel searchPanel = new JPanel(new BorderLayout(8, 0));
searchPanel.setOpaque(false);
searchPanel.setBorder(new EmptyBorder(0, 0, 10, 0));
txtSearch = UIUtils.createTextField();
txtSearch.setPreferredSize(new Dimension(0, 36));

JButton btnSearch  = UIUtils.createButton("🔍 Search", UIUtils.COLOR_ACCENT);
JButton btnRefresh = UIUtils.createButton("↺ All",     UIUtils.COLOR_PRIMARY);
btnSearch.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 13));
btnRefresh.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 13));
```

```
    JPanel searchBtns = new JPanel(new FlowLayout(FlowLayout.RIGHT, 5, 0));
    searchBtns.setOpaque(false);
    searchBtns.add(btnSearch); searchBtns.add(btnRefresh);

    JLabel lblSearch = new JLabel("Search: ");
    lblSearch.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 13));
    lblSearch.setForeground(UIUtils.COLOR_TEXT);

    searchPanel.add(lblSearch, BorderLayout.WEST);
    searchPanel.add(txtSearch, BorderLayout.CENTER);
    searchPanel.add(searchBtns, BorderLayout.EAST);

    String[] cols = {"ID", "Name", "Description", "Price", "Stock", "Category"};
    model = new DefaultTableModel(cols, 0) {
       public boolean isCellEditable(int r, int c) { return false; }
    };
    table = new JTable(model);
    UIUtils.styleTable(table);

    JScrollPane scroll = new JScrollPane(table);
    scroll.setBorder(BorderFactory.createLineBorder(UIUtils.COLOR_SUBTLE));
    scroll.getViewport().setBackground(UIUtils.COLOR_WHITE);

    rightPanel.add(searchPanel, BorderLayout.NORTH);
    rightPanel.add(scroll, BorderLayout.CENTER);

    table.getSelectionModel().addListSelectionListener(e -> loadSelectedRow());
    btnAdd.addActionListener(e -> addProduct());
    btnUpdate.addActionListener(e -> updateProduct());
    btnDelete.addActionListener(e -> deleteProduct());
    btnClear.addActionListener(e -> clearForm());
    btnSearch.addActionListener(e -> searchProducts());
    btnRefresh.addActionListener(e -> loadProducts());

    add(leftPanel, BorderLayout.WEST);
    add(rightPanel, BorderLayout.CENTER);
  }

  private void loadProducts() {
    model.setRowCount(0);
    try (Connection con = DBConnection.getConnection();
         Statement st = con.createStatement();
         ResultSet rs = st.executeQuery("SELECT * FROM products ORDER BY id DESC")) {
        while (rs.next()) {
           model.addRow(new Object[]{
              rs.getInt("id"), rs.getString("name"), rs.getString("description"),
```

```
                "₹" + rs.getDouble("price"), rs.getInt("stock"), rs.getString("category")
            });
        }
    } catch (SQLException e) { showError(e.getMessage()); }
  }

  private void addProduct() {
    if (txtName.getText().isEmpty() || txtPrice.getText().isEmpty()) {
      showError("Name and Price are required!"); return;
    }
    try (Connection con = DBConnection.getConnection()) {
      PreparedStatement ps = con.prepareStatement(
        "INSERT INTO products (name,description,price,stock,category) VALUES (?,?,?,?,?)");
      ps.setString(1, txtName.getText());
      ps.setString(2, txtDesc.getText());
      ps.setDouble(3, Double.parseDouble(txtPrice.getText()));
      ps.setInt(4, txtStock.getText().isEmpty() ? 0 : Integer.parseInt(txtStock.getText()));
      ps.setString(5, txtCategory.getText());
      ps.executeUpdate();
      showSuccess("Product added successfully!");
      clearForm(); loadProducts();
    } catch (Exception e) { showError(e.getMessage()); }
  }

  private void updateProduct() {
    if (selectedId == -1) { showError("Select a product first!"); return; }
    try (Connection con = DBConnection.getConnection()) {
      PreparedStatement ps = con.prepareStatement(
        "UPDATE products SET name=?,description=?,price=?,stock=?,category=? WHERE
id=?");
      ps.setString(1, txtName.getText());
      ps.setString(2, txtDesc.getText());
      ps.setDouble(3, Double.parseDouble(txtPrice.getText()));
      ps.setInt(4, Integer.parseInt(txtStock.getText()));
      ps.setString(5, txtCategory.getText());
      ps.setInt(6, selectedId);
      ps.executeUpdate();
      showSuccess("Product updated!");
      clearForm(); loadProducts();
    } catch (Exception e) { showError(e.getMessage()); }
  }

  private void deleteProduct() {
    if (selectedId == -1) { showError("Select a product first!"); return; }
    if (JOptionPane.showConfirmDialog(this, "Delete this product?", "Confirm",
        JOptionPane.YES_NO_OPTION) == JOptionPane.YES_OPTION) {
      try (Connection con = DBConnection.getConnection()) {
```

```
            PreparedStatement ps = con.prepareStatement("DELETE FROM products WHERE
id=?");
            ps.setInt(1, selectedId);
            ps.executeUpdate();
            clearForm(); loadProducts();
          } catch (SQLException e) { showError(e.getMessage()); }
        }
    }

    private void searchProducts() {
        String kw = "%" + txtSearch.getText() + "%";
        model.setRowCount(0);
        try (Connection con = DBConnection.getConnection()) {
            PreparedStatement ps = con.prepareStatement(
                "SELECT * FROM products WHERE name LIKE ? OR category LIKE ?");
            ps.setString(1, kw); ps.setString(2, kw);
            ResultSet rs = ps.executeQuery();
            while (rs.next()) {
                model.addRow(new Object[]{
                    rs.getInt("id"), rs.getString("name"), rs.getString("description"),
                    "₹" + rs.getDouble("price"), rs.getInt("stock"), rs.getString("category")
                });
            }
        } catch (SQLException e) { showError(e.getMessage()); }
    }

    private void loadSelectedRow() {
        int row = table.getSelectedRow();
        if (row >= 0) {
            selectedId = (int) model.getValueAt(row, 0);
            txtName.setText((String) model.getValueAt(row, 1));
            txtDesc.setText((String) model.getValueAt(row, 2));
            txtPrice.setText(model.getValueAt(row, 3).toString().replace("₹", ""));
            txtStock.setText(String.valueOf(model.getValueAt(row, 4)));
            txtCategory.setText((String) model.getValueAt(row, 5));
        }
    }

    private void clearForm() {
        selectedId = -1;
        txtName.setText(""); txtDesc.setText(""); txtPrice.setText("");
        txtStock.setText(""); txtCategory.setText("");
        table.clearSelection();
    }

    private void showError(String msg) {
        JOptionPane.showMessageDialog(this, msg, "Error", JOptionPane.ERROR_MESSAGE);
```

```
    }

    private void showSuccess(String msg) {
        JOptionPane.showMessageDialog(this, msg, "Success",
JOptionPane.INFORMATION_MESSAGE);
    }
}
```

RegisterFrame.java

```
package ecommercesystem;

import java.awt.*;
import java.sql.*;
import javax.swing.*;
import javax.swing.border.*;

public class RegisterFrame extends JFrame {

    private JTextField     txtUsername, txtEmail, txtName, txtPhone, txtAddress;
    private JPasswordField txtPassword;
    private JLabel         lblStatus;

    public RegisterFrame() {
        setTitle("E-Commerce System - Register");
        setSize(440, 660);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(DISPOSE_ON_CLOSE);
        setResizable(false);
        initComponents();
    }

    private void initComponents() {
        JPanel root = new JPanel(new GridBagLayout());
        root.setBackground(new Color(245, 247, 250));
        setContentPane(root);

        JPanel card = new JPanel();
        card.setLayout(new BoxLayout(card, BoxLayout.Y_AXIS));
        card.setBackground(Color.WHITE);
        card.setPreferredSize(new Dimension(370, 600));

        JPanel accent = new JPanel();
        accent.setBackground(new Color(39, 174, 96));
        accent.setPreferredSize(new Dimension(0, 4));
        accent.setMaximumSize(new Dimension(Integer.MAX_VALUE, 4));
```

```
JLabel lblTitle = new JLabel("Create Account");
lblTitle.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 22));
lblTitle.setForeground(new Color(44, 62, 80));
lblTitle.setAlignmentX(Component.LEFT_ALIGNMENT);

JLabel lblSub = new JLabel("Fill in your details to register");
lblSub.setFont(new Font(UIUtils.EMOJI_FONT, Font.PLAIN, 12));
lblSub.setForeground(new Color(149, 165, 166));
lblSub.setAlignmentX(Component.LEFT_ALIGNMENT);

txtUsername = field(); txtPassword = passField();
txtEmail    = field(); txtName     = field();
txtPhone    = field(); txtAddress  = field();

String[]    labels = {"Username","Password","Email","Full Name","Phone","Address"};
JComponent[] fields = {txtUsername, txtPassword, txtEmail,
                txtName, txtPhone, txtAddress};

// ── ROUNDED BUTTONS ──
JButton btnRegister = roundedBtn("Register",     new Color(39, 174, 96));
JButton btnBack     = roundedBtn("Back to Sign In", new Color(52, 152, 219));
btnRegister.setMaximumSize(new Dimension(Integer.MAX_VALUE, 44));
btnRegister.setAlignmentX(Component.LEFT_ALIGNMENT);
btnBack.setMaximumSize(new Dimension(Integer.MAX_VALUE, 40));
btnBack.setAlignmentX(Component.LEFT_ALIGNMENT);

lblStatus = new JLabel(" ");
lblStatus.setFont(new Font(UIUtils.EMOJI_FONT, Font.PLAIN, 11));
lblStatus.setForeground(new Color(231, 76, 60));
lblStatus.setAlignmentX(Component.LEFT_ALIGNMENT);

btnRegister.addActionListener(e -> doRegister());
btnBack.addActionListener(e -> { dispose(); new LoginFrame().setVisible(true); });

card.add(accent);
card.add(Box.createVerticalStrut(20));
card.add(lblTitle);
card.add(Box.createVerticalStrut(4));
card.add(lblSub);
card.add(Box.createVerticalStrut(20));

for (int i = 0; i < labels.length; i++) {
   JLabel lbl = new JLabel(labels[i]);
   lbl.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 12));
   lbl.setForeground(new Color(44, 62, 80));
   lbl.setAlignmentX(Component.LEFT_ALIGNMENT);
```

```
            card.add(lbl);
            card.add(Box.createVerticalStrut(4));
            card.add(fields[i]);
            card.add(Box.createVerticalStrut(10));
        }

        card.add(Box.createVerticalStrut(8));
        card.add(btnRegister);
        card.add(Box.createVerticalStrut(6));
        card.add(btnBack);
        card.add(Box.createVerticalStrut(8));
        card.add(lblStatus);

        root.add(card);
    }

    private JTextField field() {
        JTextField f = UIUtils.createTextField();
        f.setMaximumSize(new Dimension(Integer.MAX_VALUE, 38));
        f.setAlignmentX(Component.LEFT_ALIGNMENT);
        return f;
    }

    private JPasswordField passField() {
        JPasswordField f = UIUtils.createPasswordField();
        f.setMaximumSize(new Dimension(Integer.MAX_VALUE, 38));
        f.setAlignmentX(Component.LEFT_ALIGNMENT);
        return f;
    }

    private void doRegister() {
        String username = txtUsername.getText().trim();
        String password = new String(txtPassword.getPassword()).trim();
        String email    = txtEmail.getText().trim();
        String name     = txtName.getText().trim();
        String phone    = txtPhone.getText().trim();
        String addr     = txtAddress.getText().trim();

        if (username.isEmpty() || password.isEmpty() ||
                email.isEmpty() || name.isEmpty()) {
            lblStatus.setText("Required fields cannot be empty"); return;
        }
        try (Connection con = DBConnection.getConnection()) {
            con.setAutoCommit(false);
            try {
                PreparedStatement ps = con.prepareStatement(
                    "INSERT INTO users (username,password,role,email) VALUES (?,?,?,?)");
```

```
            ps.setString(1, username); ps.setString(2, password);
            ps.setString(3, "customer"); ps.setString(4, email);
            ps.executeUpdate();

            PreparedStatement ps2 = con.prepareStatement(
               "INSERT INTO customers (name,email,phone,address) VALUES (?,?,?,?)",
               new String[]{"ID"});
            ps2.setString(1, name);  ps2.setString(2, email);
            ps2.setString(3, phone); ps2.setString(4, addr);
            ps2.executeUpdate();

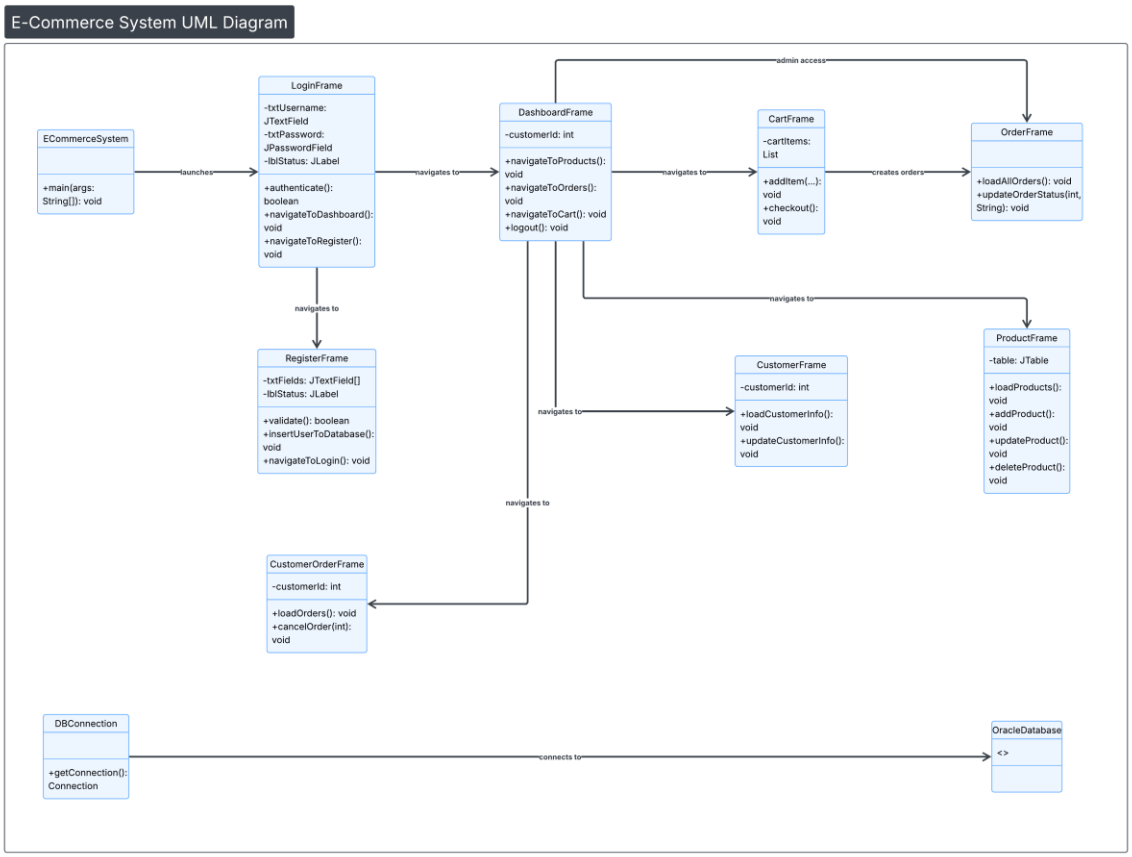
            int custId = -1;
            try (ResultSet gr = ps2.getGeneratedKeys()) {
               if (gr.next()) custId = gr.getInt(1);
            }
            con.commit();
            dispose();
            new DashboardFrame(username, "customer", custId).setVisible(true);
         } catch (SQLException ex) {
            con.rollback();
            lblStatus.setText("Failed: " + ex.getMessage());
         }
      } catch (SQLException ex) {
         lblStatus.setText("DB error: " + ex.getMessage());
      }
   }

   private JButton roundedBtn(String text, Color bg) {
      JButton btn = new JButton(text) {
         protected void paintComponent(Graphics g) {
            Graphics2D g2 = (Graphics2D) g.create();
            g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
                                RenderingHints.VALUE_ANTIALIAS_ON);
            g2.setColor(getBackground());
            g2.fillRoundRect(0, 0, getWidth(), getHeight(), 12, 12);
            FontMetrics fm = g2.getFontMetrics();
            g2.setColor(getForeground());
            g2.setFont(getFont());
            int x = (getWidth()  - fm.stringWidth(getText())) / 2;
            int y = (getHeight() + fm.getAscent() - fm.getDescent()) / 2;
            g2.drawString(getText(), x, y);
            g2.dispose();
         }
      };
      btn.setBackground(bg);
      btn.setForeground(Color.WHITE);
      btn.setFont(new Font(UIUtils.EMOJI_FONT, Font.BOLD, 13));
```

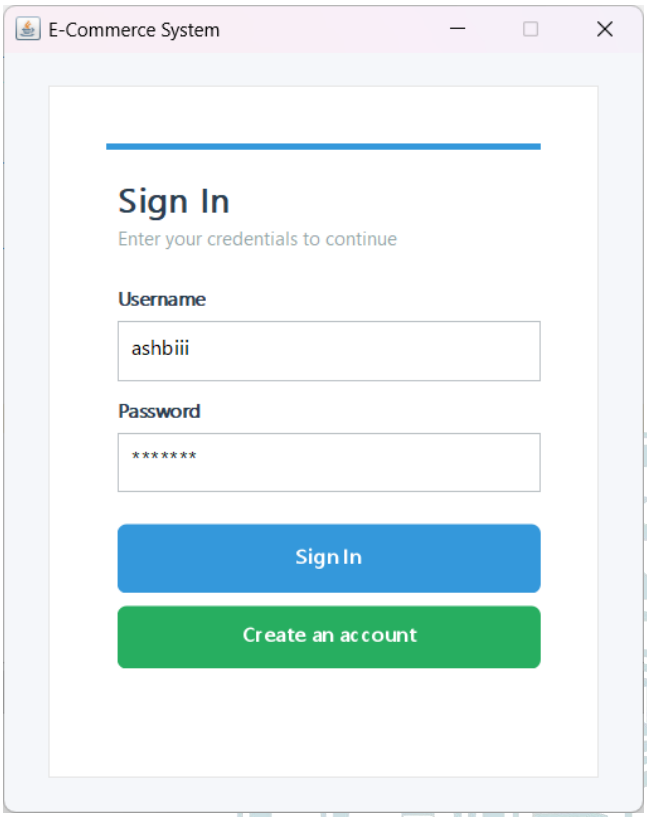
```
        btn.setFocusPainted(false);
        btn.setBorderPainted(false);
        btn.setContentAreaFilled(false);
        btn.setOpaque(false);
        btn.setCursor(new Cursor(Cursor.HAND_CURSOR));
        btn.setBorder(new EmptyBorder(10, 0, 10, 0));
        btn.addMouseListener(new java.awt.event.MouseAdapter() {
            public void mouseEntered(java.awt.event.MouseEvent e) {
                btn.setBackground(bg.darker()); btn.repaint();
            }
            public void mouseExited(java.awt.event.MouseEvent e) {
                btn.setBackground(bg); btn.repaint();
            }
        });
        return btn;
    }
}
```

Output

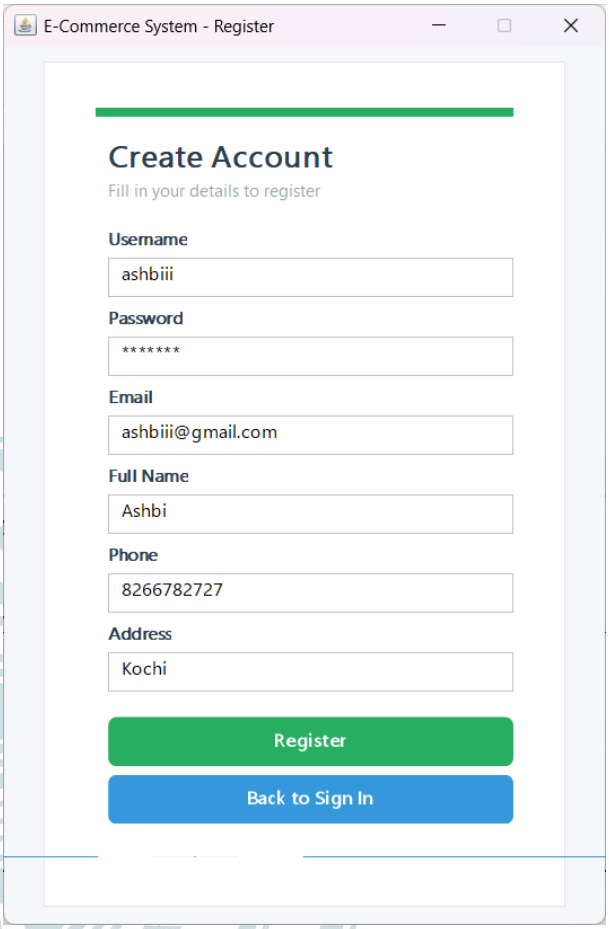
Class Diagram



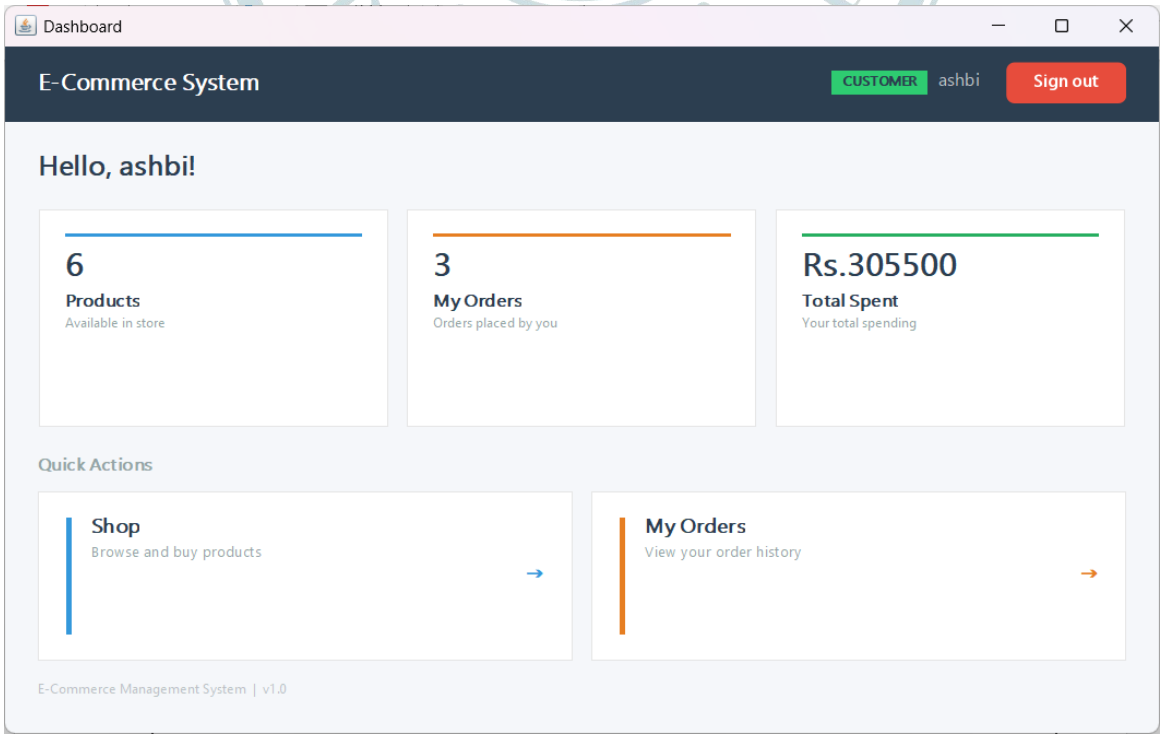
LoginFrame

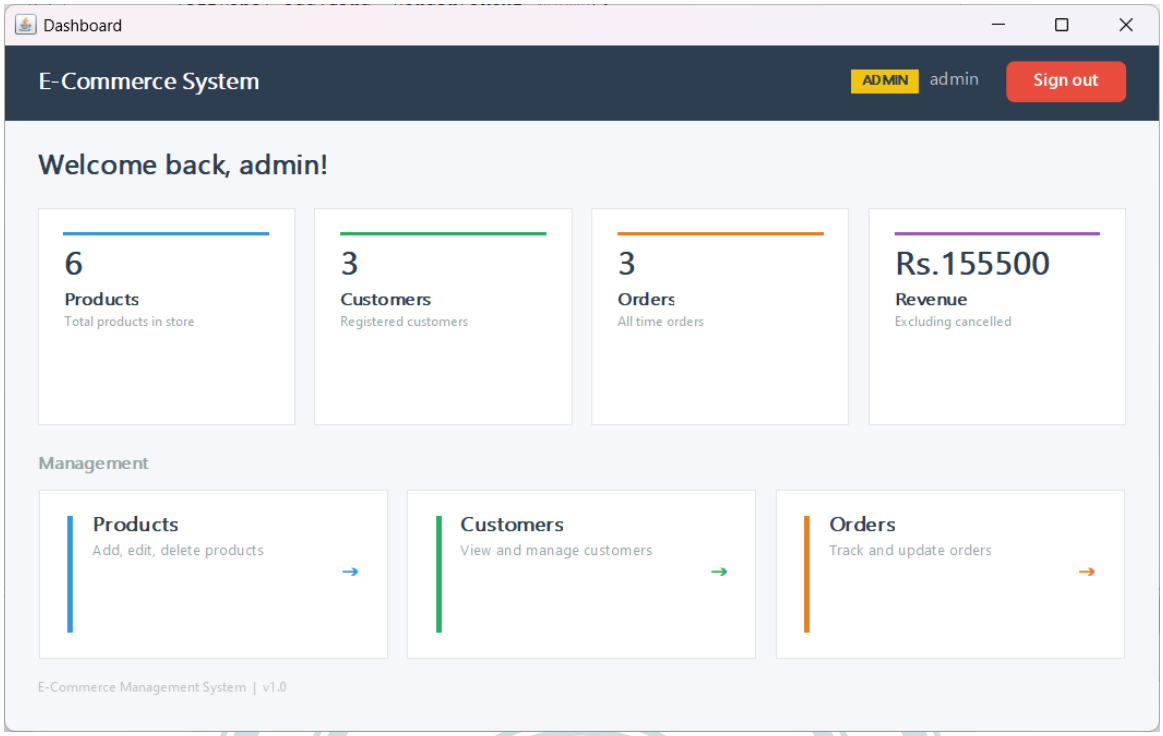


RegisterFrame

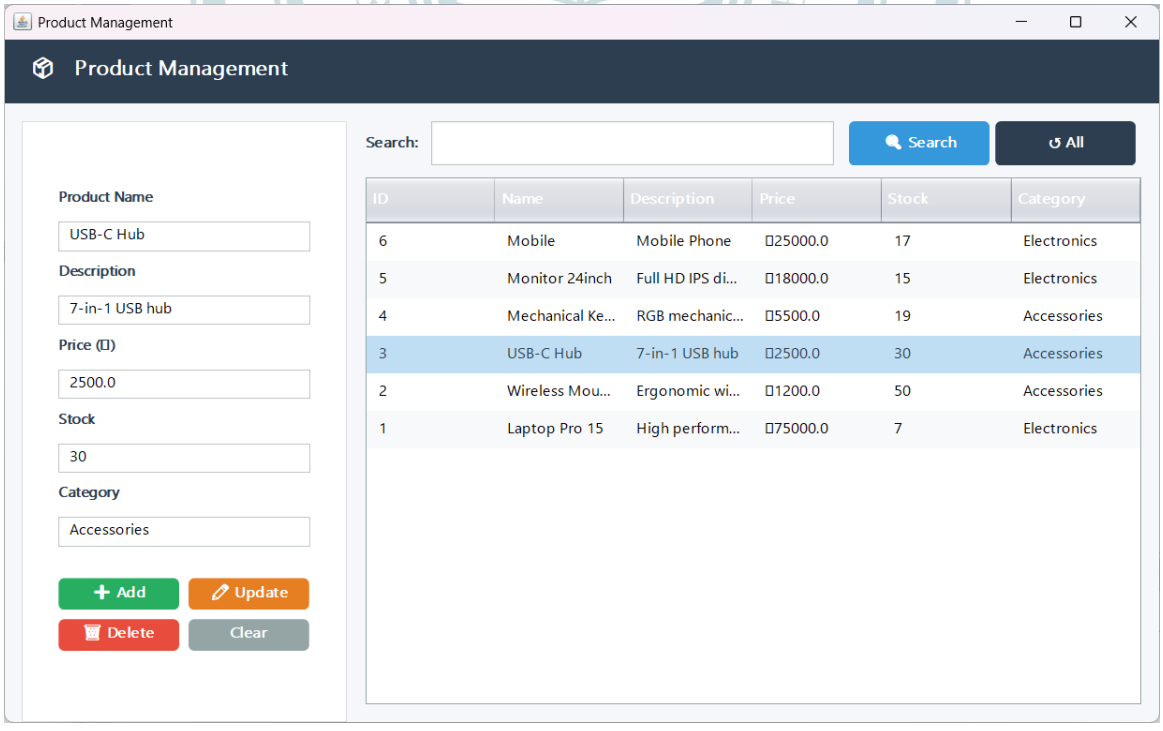


DashboardFrame

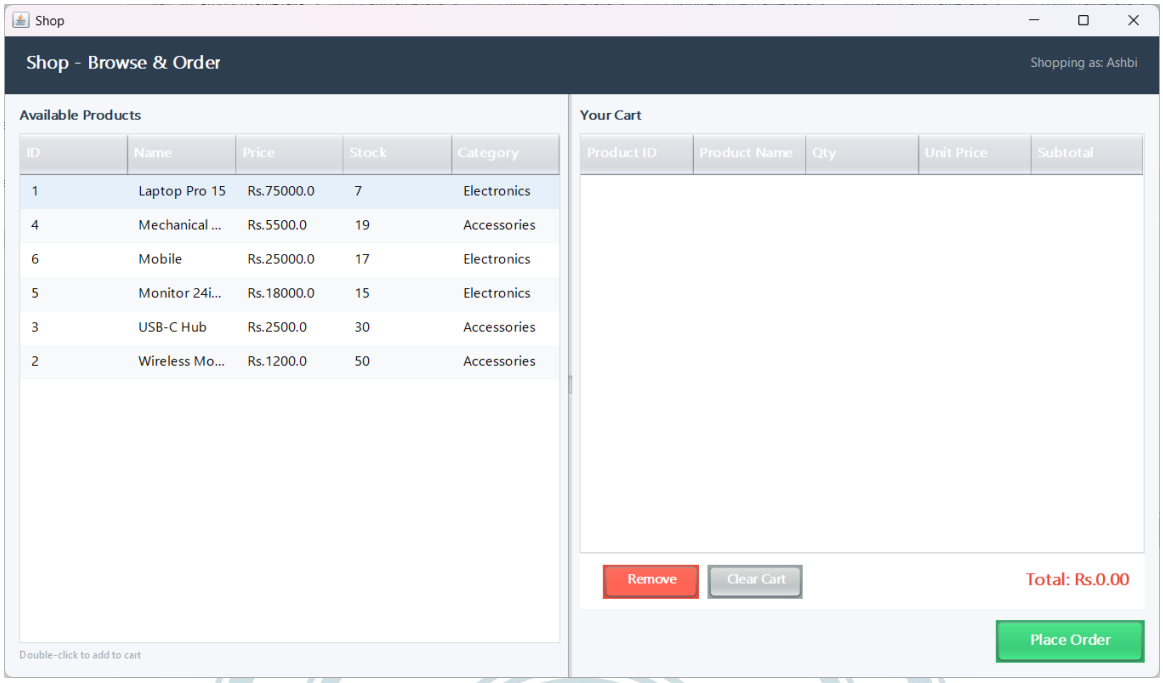




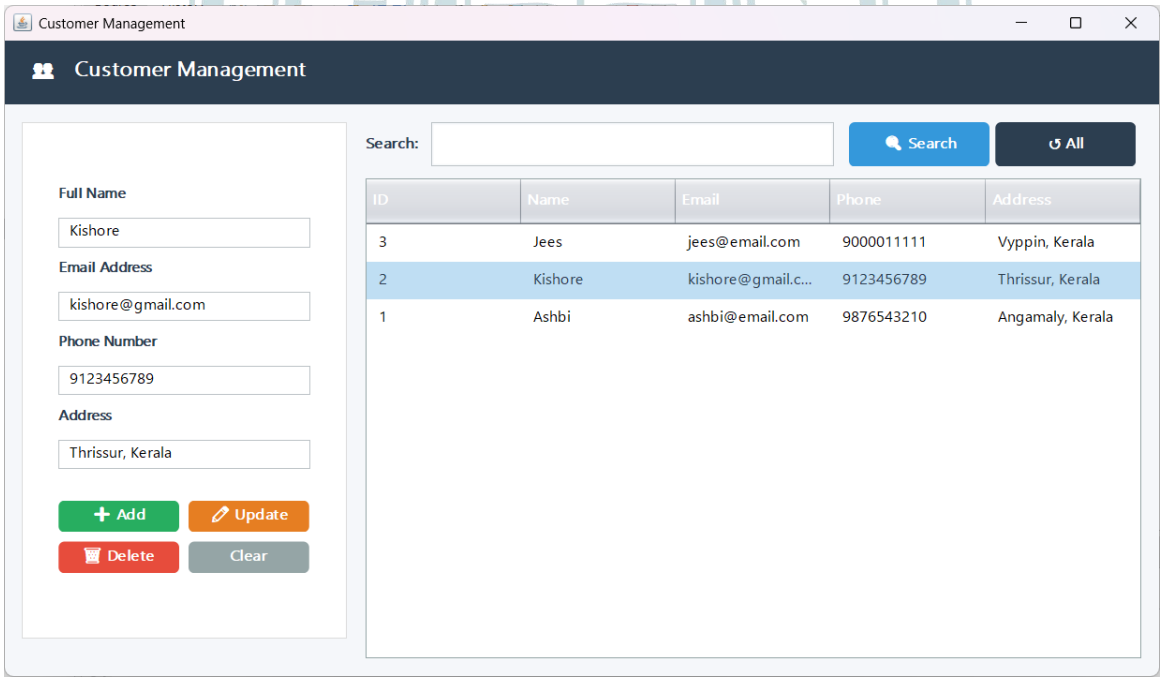
ProductFrame



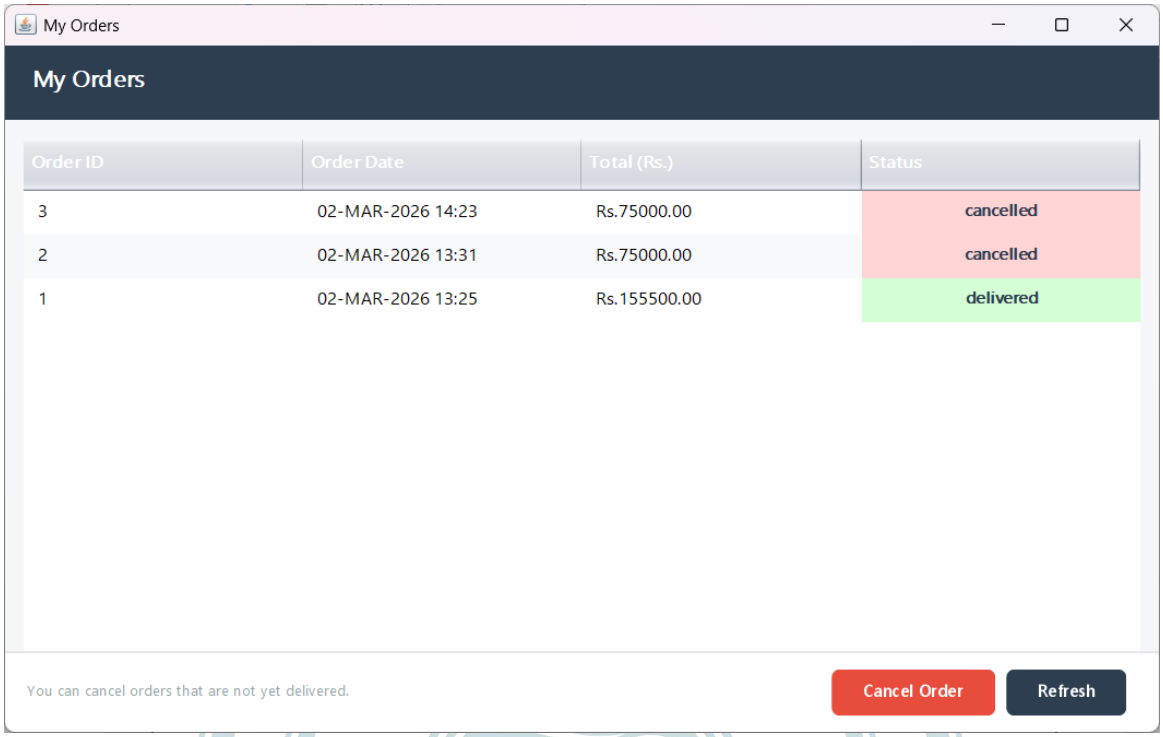
CartFrame



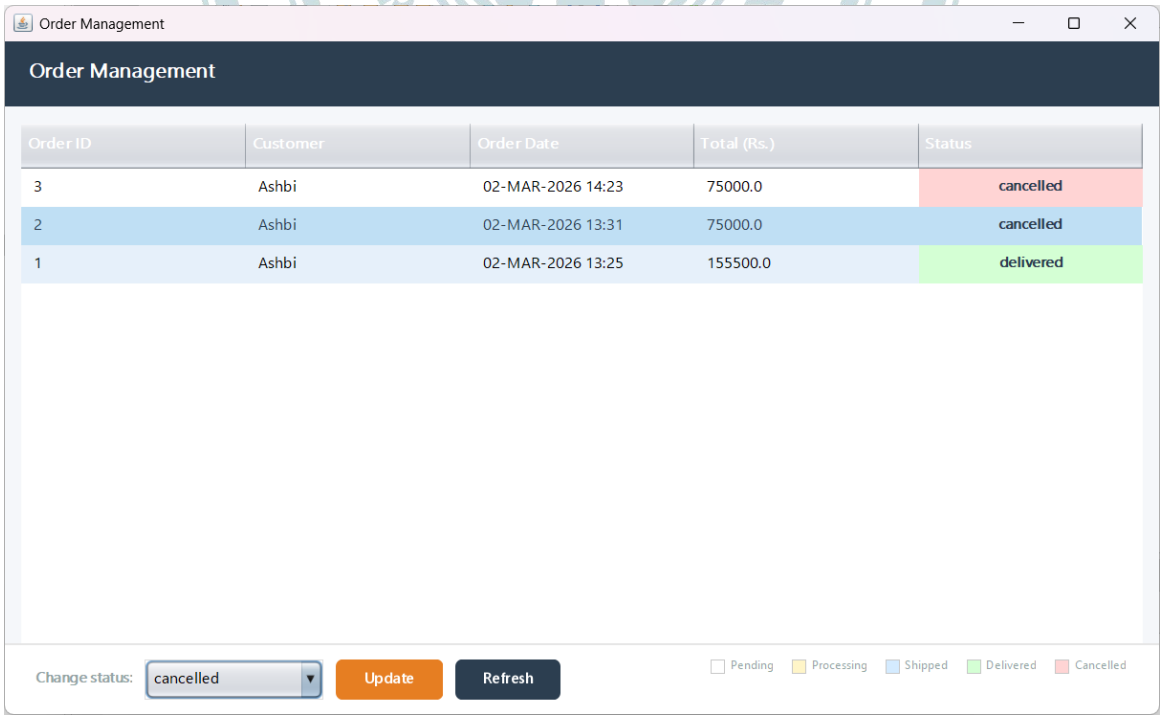
CustomerFrame



CustomerOrderFrame



OrderFrame



LEARN • SERVE • EXCEL
RAJAGIRI